

**ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



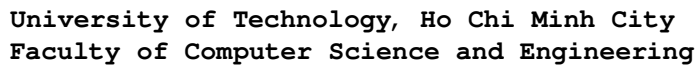
BÁO CÁO BÀI TẬP LỚN

HỆ ĐIỀU HÀNH (CO2018)

Simple Operating System

Giảng viên hướng dẫn : Đỗ Hoàng Quân
Sinh viên: Nguyễn Hữu Khánh - 2411524
Nguyễn Nguyên Hưng - 2411365
Nguyễn Tiến Nam - 2412188
Vũ Trí Khải - 2411566

Thành phố Hồ Chí Minh, 05/2026



STT	Họ và tên	MSSV	Nhiệm vụ	Đóng góp
1	Nguyễn Hữu Khánh	2411524	Triển khai cơ chế phân trang (Paging Address Translation), xử lý luồng bộ nhớ ảo (Virtual Memory).	100%
2	Nguyễn Nguyên Hưng	2411365	Xây dựng cơ chế đồng bộ hóa (Synchronization), tích hợp system call mới, và kiểm thử toàn hệ thống.	100%
3	Nguyễn Tiến Nam	2412188	Khởi tạo cấu trúc quản lý bộ nhớ vật lý (Physical Memory), xử lý cấp phát frame.	100%
4	Vũ Trí Khải	2411566	Cài đặt bộ định thời Multi-Level Queue (MLQ), phân tích biểu đồ Gantt.	100%

Nhận xét của GV:

Giảng viên <i>(Ký và ghi rõ họ, tên)</i>	Nhóm trưởng <i>(Ký và ghi rõ họ, tên)</i>
 Đỗ Hoàng Quân	 Vũ Trí Khải



Mục lục

1	Mở đầu	4
1.1	Bối cảnh và động lực	4
1.2	Mục tiêu bài tập	4
1.3	Kiến trúc tổng quan hệ thống	4
1.4	Phạm vi thực hiện của nhóm	5
1.5	Cấu trúc báo cáo	6
2	Scheduler	7
2.1	Mô tả thuật toán điều phối MLQ	7
2.1.1	Tổng quan	7
2.1.2	Cơ chế hàng đợi	7
2.1.3	Số lượng Slot (Quantum phân phối)	7
2.1.4	Cách thức chọn tiến trình (get_proc)	8
2.1.5	Chế độ Round-Robin trong cùng mức ưu tiên	8
2.2	Chi tiết triển khai mã nguồn	8
2.2.1	File queue.c	8
2.2.2	File sched.c	10
2.3	Biểu đồ Gantt và Phân tích kết quả	11
2.3.1	Testcase sched_0	11
2.3.2	Testcase sched	15
2.3.3	Testcase sched_1	17
2.4	Trả lời câu hỏi lý thuyết	20
2.4.1	Đánh giá tác động của các chính sách chi tiết trong MLQ	20
2.4.2	So sánh với các thuật toán lập lịch khác	21
2.4.3	Kết luận	21
2.4.4	Đồng bộ hóa (Synchronization) và Race Condition	21
3	Memory Management	23
3.1	Thiết lập và quản lý bộ nhớ vật lý	23
3.1.1	Cấu trúc dữ liệu và Khởi tạo	23
3.1.2	Cấp phát bộ nhớ ($\mathcal{O}(1)$) và Đồng bộ hóa	23
3.1.3	Thu hồi bộ nhớ ($\mathcal{O}(N)$) và Đánh đổi thiết kế	24
3.1.4	An toàn bộ nhớ và Công tác Gỡ lỗi	24
3.2	Quản lý không gian bộ nhớ ảo	25
3.2.1	Cấu trúc phân tầng: VMA và Region	25
3.2.2	Cơ chế nối rộng ranh giới và Căn chỉnh trang	25
3.2.3	Kiểm soát an toàn: Tránh chồng lấn và Hoàn tác (Rollback)	26
3.3	Cơ chế phân trang 5 cấp trên kiến trúc 64-bit	27
3.3.1	Cơ chế Canonical Address (Địa chỉ chuẩn tắc)	27
3.3.2	Phân giải địa chỉ ảo 57-bit	27
3.3.3	Chiến lược Cấp phát động (Sparse Allocation)	28
3.3.4	Ví dụ minh họa chi tiết quy trình dịch địa chỉ ảo (Address Walkthrough)	29
3.4	Thư viện quản lý và giao tiếp bộ nhớ (libmem)	30
3.4.1	Cấp phát và thu hồi không gian người dùng (User-space)	31
3.4.2	Xử lý lỗi trang (Page Fault) và cơ chế Swapping	32
3.4.3	Quản lý bộ nhớ đặc quyền (Kernel-space Memory)	33
3.4.4	Giao tiếp dữ liệu liên không gian (Cross-space Communication)	34
3.5	Minh họa trạng thái cấp phát vùng nhớ (Data Segment Allocation)	34
3.6	Cơ chế bảo mật và truy xuất bộ nhớ	35
3.6.1	Cơ chế định danh tiến trình thông qua PID	35
3.6.2	Mã nguồn (Static Analysis)	35



3.6.3	Đánh giá thiết kế	35
3.7	Cấp phát giả lập (Dummy Allocation) và Cơ chế Giao tiếp qua <code>vmap_pg_d_memset</code>	36
3.7.1	Luồng Điều phối và Giao tiếp Liên Ranh giới (System Call Dispatching)	36
3.7.2	Thuật toán Duyệt Cây Phân cấp và Khởi tạo Biến thể Giả lập	36
3.8	Các vấn đề cốt lõi trong Quản lý Bộ nhớ	37
3.8.1	Động lực kết hợp phân đoạn và phân trang	37
3.8.2	Lợi ích của phân trang phân cấp nhiều mức	37
3.8.3	Ưu điểm và nhược điểm của cấp phát bộ nhớ phân trang (paging) và cấp phát bộ nhớ liên tục (contiguous memory allocation)	38
3.9	Phân tích kết quả kiểm thử	39
3.9.1	Testcase <code>os_0_mlq_paging</code>	39
3.9.2	Testcase <code>os_1_mlq_paging</code>	43
3.9.3	Testcase <code>os_1_singleCPU_mlq</code>	48
3.9.4	Testcase <code>os_1_singleCPU_mlq_paging</code>	54
3.9.5	Testcase <code>os_2_mlq_paging</code>	59
3.9.6	Testcase <code>os_2_singleCPU_mlq_paging</code>	63
3.9.7	Tổng kết kết quả kiểm thử	67
3.10	Đánh giá và thống kê hiệu năng quản lý bộ nhớ	67
4	System Call	69
4.1	Tương tác giữa các mô-đun lưu trữ tên tiến trình, điều khiển hệ điều hành và quản lý hàng đợi	69
4.1.1	Cơ chế Quản lý và Nạp Tiến trình	69
4.1.2	Quy trình thực thi Syscall	69
4.2	Trả lời câu hỏi	70
4.3	Kết quả thực thi	71
4.3.1	<code>os_sc</code>	71
4.3.2	<code>os_syscall</code>	71
4.3.3	<code>os_syscall_list</code>	72
5	Synchronization	72
5.1	Critical section và Lock	72
5.2	Trả lời câu hỏi	73
6	Put It All Together	74
6.1	Testcase <code>os_1_mlq_paging_small_1K</code>	74
6.2	Testcase <code>os_1_mlq_paging_small_4K</code>	79
7	Kết luận	85
7.1	Mức độ hoàn thành mục tiêu	85
7.2	Những khó khăn kỹ thuật và giải pháp kiểm thử	85
7.3	Hướng phát triển và tối ưu hệ thống trong tương lai	85

1 Mở đầu

1.1 Bối cảnh và động lực

Hệ điều hành là nền tảng cốt lõi của mọi hệ thống máy tính hiện đại, đóng vai trò trung gian trong việc quản lý hai tài nguyên quan trọng nhất: CPU (hoặc bộ đồng xử lý) và bộ nhớ RAM. Dù các khái niệm như bộ định thời, quản lý không gian địa chỉ hay đồng bộ hóa tiến trình thường được tiếp cận ở góc độ lý thuyết, để hiểu sâu cơ chế hoạt động của chúng, sinh viên cần trực tiếp thiết kế và thao tác ở mức hệ thống.

Bài tập lớn môn Hệ điều hành (CO2018) đặt ra một thách thức thực tế: xây dựng và hoàn thiện một hệ điều hành mô phỏng — **Simple Operating System** — dựa trên bộ khung mã nguồn được cung cấp. Quá trình này giúp chuyển hóa các khái niệm trừu tượng thành mã nguồn cụ thể, từ đó cho phép quan sát trực quan cách một hệ điều hành cấp phát tài nguyên, bảo vệ hệ thống thông qua chế độ hoạt động kép (*dual-mode*) và duy trì tính ổn định khi nhiều tiến trình chạy song song.

1.2 Mục tiêu bài tập

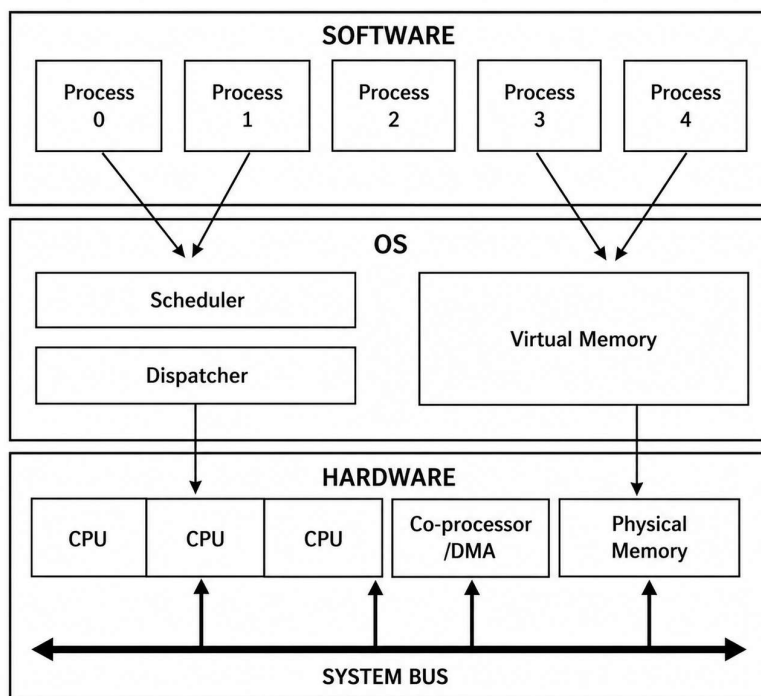
Mục tiêu cốt lõi của bài tập là mô phỏng thành công các thành phần trọng yếu của một hệ điều hành đơn giản. Cụ thể, hệ thống tập trung vào ba mô-đun chính sau:

- **Bộ định thời (Scheduler):** Cài đặt thuật toán *Multi-Level Queue* (MLQ) được lấy cảm hứng từ hạt nhân Linux, nhằm điều phối các tiến trình trên nhiều CPU khả dụng. Thuật toán sử dụng nhiều hàng đợi ưu tiên (lên đến $MAX_PRIO = 140$ mức), trong đó số lượng "slot" CPU cấp phát cho một hàng đợi được tính toán trực tiếp dựa trên công thức $MAX_PRIO - prio$.
- **Quản lý bộ nhớ (Memory Management):** Xây dựng cơ chế phân trang (*paging*) nhằm thiết lập ánh xạ từ địa chỉ ảo sang địa chỉ vật lý, đảm bảo không gian bộ nhớ của mỗi tiến trình được cô lập hoàn toàn. Module này yêu cầu cài đặt các thao tác cấp phát, đọc/ghi, và đặc biệt là cơ chế hoán đổi (*swapping*) để luân chuyển dữ liệu giữa RAM và các thiết bị SWAP khi bộ nhớ chính cạn kiệt.
- **Đồng bộ hóa (Synchronization):** Phân tích và xác định các tài nguyên dùng chung trong môi trường đa xử lý, từ đó triển khai các cơ chế khóa (*locking*) phù hợp nhằm ngăn chặn xung đột dữ liệu khi nhiều tiến trình cùng truy cập đồng thời.

Bên cạnh đó, sinh viên còn thực hành thiết kế giao diện lập trình thông qua việc cài đặt các **system call**, tạo cầu nối an toàn giữa không gian người dùng và các tác vụ đặc quyền của hạt nhân.

1.3 Kiến trúc tổng quan hệ thống

Kiến trúc của Simple Operating System được phân tách rõ ràng để quản lý hiệu quả phần cứng mô phỏng. Theo thiết kế tổng thể, hai thành phần cốt lõi đảm nhận việc vận hành hệ thống là *Scheduler/Dispatcher* (quyết định tiến trình nào chạy trên CPU nào) và *Virtual Memory* (quản lý không gian nhớ và quá trình dịch địa chỉ). Hình 1 minh họa mối quan hệ giữa các lớp phần mềm, nhân OS, và phần cứng ảo trong hệ thống.



Hình 1: Kiến trúc tổng quan của Simple Operating System

Để đảm bảo tính bảo mật và kiểm soát truy cập, hệ điều hành được vận hành dưới **chế độ hoạt động kép** (*dual-mode operation*). Được hỗ trợ bởi một mode bit trên phần cứng, hệ thống phân định rạch ròi giữa mã nguồn người dùng (*user mode*) và các đoạn mã đặc quyền của nhân (*kernel mode*). Cơ chế này bảo đảm rằng các chương trình ứng dụng không thể tự ý thực thi các lệnh nhạy cảm, đồng thời phải giao tiếp với hạt nhân (chẳng hạn như thao tác đọc/ghi vùng nhớ) thông qua các hàm trung gian chuẩn mực như `copy_from_user` hoặc `copy_to_user`.

1.4 Phạm vi thực hiện của nhóm

Trong khuôn khổ giới hạn của đề án, nhóm đã tập trung hoàn thiện và kiểm thử thành công các hạng mục sau:

- Hoàn thiện luồng hoạt động của MLQ Scheduler thông qua việc lập trình các hàm `enqueue()`, `dequeue()` để thao tác với hàng đợi ưu tiên, và `get_proc()` để chọn tiến trình hợp lệ tiếp theo theo chính sách phân phối thời gian.
- Thiết lập và quản lý cấu trúc không gian bộ nhớ tiến trình, từ việc xử lý các vùng nhớ (`vm_area`), các phân vùng nhỏ (`vm_rgn`) đến việc duy trì bảng cấp phát tự do (`vm_freerg_list`).
- Triển khai cơ chế dịch địa chỉ hỗ trợ hệ thống 32-bit và mở rộng quy mô lên hệ thống phân trang đa cấp 64-bit (*5-level paging*), tuân thủ định dạng địa chỉ canonical để tách biệt hoàn toàn không gian của user và kernel.
- Hiện thực hóa các thao tác quản lý bộ nhớ nâng cao như cấp phát (`alloc`, `kmalloc`), giải phóng (`free`), và đặc biệt là vòng đời hoán đổi trang vật lý giữa MEMRAM và MEMSWP.
- Thêm mới các system call vào bảng `syscall.tbl`, cấu trúc các hàm xử lý (`sys_XXXhandler`) và xác minh hoạt động của chúng qua các ứng dụng người dùng tự định nghĩa.

1.5 Cấu trúc báo cáo

Phần còn lại của báo cáo được tổ chức theo luồng logic của hệ thống, cụ thể như sau:

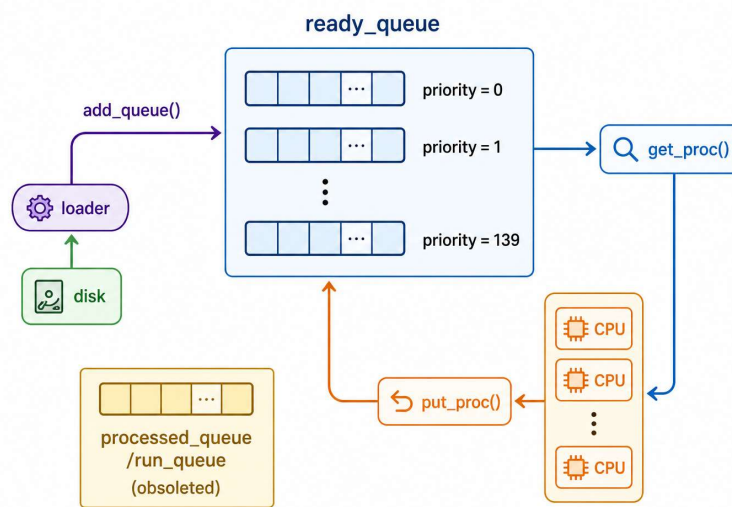
- **Phần 2 – Bộ định thời (Scheduler):** Trình bày chi tiết cấu trúc dữ liệu của hàng đợi, cách thức hoạt động của thuật toán MLQ, việc xử lý độ ưu tiên (default và live priority), và minh họa quá trình điều phối CPU qua biểu đồ Gantt.
- **Phần 3 – Quản lý bộ nhớ (Memory Management):** Phân tích thiết kế của bộ nhớ ảo, từ tổ chức layout không gian tiến trình, cơ chế dịch địa chỉ đa cấp, đến thuật toán hoán đổi trang (swapping) để tối ưu hóa việc sử dụng RAM và SWAP vật lý.
- **Phần 4 – System Call & Đồng bộ hóa (Synchronization):** Giới thiệu quy trình tích hợp các lời gọi hệ thống mới để thao tác với bộ nhớ, đồng thời đánh giá vai trò của mutex trong việc giải quyết bài toán tương tranh trên các tài nguyên dùng chung.
- **Phần 7 – Kết luận:** Đánh giá mức độ hoàn thành các mục tiêu đề ra, tổng hợp những khó khăn kỹ thuật trong quá trình hiện thực hóa 64-bit address scheme và hướng phát triển tối ưu hệ thống trong tương lai.

2 Scheduler

2.1 Mô tả thuật toán điều phối MLQ

2.1.1 Tổng quan

Thuật toán Multi-Level Queue (MLQ) trong hệ thống này được thiết kế để quản lý các tiến trình với nhiều mức độ ưu tiên khác nhau, nhằm tối ưu hóa việc phân bổ tài nguyên CPU. Hình dưới đây minh họa hoạt động tổng thể của bộ điều phối trong hệ thống.



Hình 2: Sơ đồ hoạt động của bộ điều phối trong hệ thống

2.1.2 Cơ chế hàng đợi

Hệ thống hỗ trợ tối đa MAX_PRIO (140) cấp độ ưu tiên. Mỗi cấp độ ưu tiên được quản lý bởi một hàng đợi sẵn sàng (`mlq_ready_queue`) riêng biệt. Cấu trúc này cho phép hệ thống phân loại rõ ràng các tiến trình theo mức độ quan trọng, từ các tác vụ tương tác thời gian thực (ưu tiên cao, $prio = 0$) đến các tác vụ nền chạy ngầm (ưu tiên thấp, $prio = 139$).

2.1.3 Số lượng Slot (Quantum phân phối)

Mỗi hàng đợi được cấp một số lượng “slot” (lượt chạy) tương ứng với mức độ ưu tiên. Công thức tính như sau:

$$slot[prio] = MAX_PRIO - prio$$

Cơ chế này đảm bảo: độ ưu tiên càng cao (giá trị $prio$ càng nhỏ) thì số lượng slot nhận được càng lớn, tức là tiến trình được cấp quyền dùng CPU nhiều lần hơn trong một chu kỳ.

Ví dụ với $MAX_PRIO = 140$:

prio	0	1	...	139
slot	140	139	...	1

2.1.4 Cách thức chọn tiến trình (`get_proc`)

Hàm `get_mlq_proc()` thực hiện các bước sau để lấy tiến trình kế tiếp:

1. Duyệt từ hàng đợi có độ ưu tiên cao nhất (`prio = 0`) đến thấp nhất (`prio = MAX_PRIO - 1`).
2. Nếu hàng đợi tại mức `prio` không trống và còn slot (`slot[prio] > 0`), tiến trình ở đầu hàng đợi đó được lấy ra (`dequeue`) để thực thi. Slot của mức đó giảm đi 1.
3. Nếu **tất cả** các hàng đợi có tiến trình đều đã hết slot, hệ thống thực hiện **reset toàn bộ slot** về giá trị mặc định `MAX_PRIO - prio` cho mọi mức ưu tiên, rồi bắt đầu chu kỳ mới.
4. Tiến trình được chọn sẽ được đưa vào `running_list` để theo dõi.

Cơ chế reset slot đảm bảo rằng hệ thống không bị bế tắc: ngay cả khi các tiến trình ưu tiên cao chiếm dụng hết slot của mình, các tiến trình ưu tiên thấp hơn vẫn sẽ được phục vụ trong chu kỳ tiếp theo.

2.1.5 Chế độ Round-Robin trong cùng mức ưu tiên

Trong cùng một mức ưu tiên, các tiến trình được điều phối theo kiểu **Round-Robin**: mỗi tiến trình được chạy trong một khoảng thời gian `time_slice`. Khi hết `time_slice` mà tiến trình chưa hoàn thành, nó bị đẩy trở lại **cuối hàng đợi** ưu tiên tương ứng thông qua hàm `put_proc()`.

2.2 Chi tiết triển khai mã nguồn

2.2.1 File `queue.c`

`enqueue()`

Thêm một PCB vào cuối mảng `proc[]` của hàng đợi và tăng `size` lên 1. Nếu hàng đợi đã đầy (vượt `MAX_QUEUE_SIZE`), thao tác thêm bị bỏ qua để tránh tràn bộ nhớ (*buffer overflow*).

```
1 void enqueue(struct queue_t *q, struct pcb_t *proc)
2 {
3     if (q->size < MAX_QUEUE_SIZE) {
4         q->proc[q->size] = proc;
5         q->size++;
6     }
7 }
```

`dequeue()`

Hàm hoạt động khác nhau tùy chế độ biên dịch:

- **Chế độ MLQ** (`#ifdef MLQ_SCHED`): Lấy tiến trình ở **đầu hàng đợi** (chỉ mục 0), sau đó dịch chuyển toàn bộ các tiến trình phía sau lên một vị trí để duy trì cấu trúc **FIFO** (First-In First-Out). Điều này đảm bảo tính công bằng Round-Robin trong cùng một mức ưu tiên.
- **Chế độ mặc định**: Duyệt toàn bộ hàng đợi để tìm tiến trình có giá trị `priority` nhỏ nhất (ưu tiên cao nhất), lấy ra và dịch mảng để lấp đầy chỗ trống.

```
1 struct pcb_t *dequeue(struct queue_t *q)
2 {
3     if (empty(q)) return NULL;
4
5     #ifdef MLQ_SCHED
6         /* FIFO: luôn lấy phần tử đầu hàng đợi */
7         struct pcb_t *proc = q->proc[0];
8         for (int i = 0; i < q->size - 1; i++)
9             q->proc[i] = q->proc[i + 1];
10        q->size--;
11    #else
12        /* Tìm tiến trình có độ ưu tiên cao nhất */
13        int highestPriorityIdx = 0;
14        for (int i = 1; i < q->size; i++)
15            if (q->proc[i]->priority < q->proc[highestPriorityIdx]->priority)
16                highestPriorityIdx = i;
17        struct pcb_t *proc = q->proc[highestPriorityIdx];
18        for (int i = highestPriorityIdx; i < q->size - 1; i++)
19            q->proc[i] = q->proc[i + 1];
20        q->size--;
21    #endif
22    return proc;
23 }
```

purgequeue()

Tìm kiếm và xóa một tiến trình cụ thể khỏi hàng đợi (dùng để xóa tiến trình khỏi running_list khi nó hết time_slice và cần được đưa trở lại hàng đợi sẵn sàng).

```
1 struct pcb_t *purgequeue(struct queue_t *q, struct pcb_t *proc)
2 {
3     /* TODO: remove a specific item from queue
4      * */
5
6     if (empty(q) || proc == NULL) return NULL;
7
8     int i=0;
9     int found=0;
10    struct pcb_t * removedProc = NULL;
11
12    for (i=0; i < q->size; i++){
13        if(q->proc[i] == proc){
14            found = 1;
15            removedProc = q->proc[i];
16            break;
17        }
18    }
19
20    if(found){
21        for( int j =i; j < q->size-1 ; j++){
22            q->proc[j] = q->proc[j+1];
23        }
24        q->size--;
25        return removedProc;
26    }
27    return NULL;
28 }
```

2.2.2 File sched.c

get_proc()

Hàm này sử dụng hàm `get_mlq_proc()` để duyệt mảng hàng đợi sẵn sàng theo thứ tự mức ưu tiên tăng dần, từ mức 0 đến `MAX_PRIO - 1`. Toàn bộ quá trình truy xuất và cập nhật hàng đợi được bảo vệ bởi mutex `queue_lock`, nhờ đó tránh xung đột dữ liệu giữa các luồng truy cập đồng thời.

```
1 struct pcb_t *get_mlq_proc(void)
2 {
3     struct pcb_t *proc = NULL;
4     pthread_mutex_lock(&queue_lock);
5
6     for (int prio = 0; prio < MAX_PRIO; prio++) {
7         if (!empty(&mlq_ready_queue[prio]) && slot[prio] > 0) {
8             proc = dequeue(&mlq_ready_queue[prio]);
9             slot[prio]--;
10            break;
11        }
12    }
13
14    if (proc != NULL)
15        enqueue(&running_list, proc);
16
17    /* Reset slot neu tat ca hang doi co tien trinh da het luot */
18    if (proc == NULL && queue_empty() == -1) {
19        for (int prio = 0; prio < MAX_PRIO; prio++)
20            slot[prio] = MAX_PRIO - prio;
21    }
22
23    pthread_mutex_unlock(&queue_lock);
24    return proc;
25 }
```

put_proc()

Khi một tiến trình sử dụng hết lát thời gian, hàm `put_proc()` sẽ gọi `put_mlq_proc()` để đưa tiến trình trở lại hàng đợi sẵn sàng. Trong hàm này, tiến trình được cập nhật vào các cấu trúc quản lý hàng đợi của kernel như `ready_queue`, `mlq_ready_queue` và `running_list`. Toàn bộ thao tác trên hàng đợi được bảo vệ bởi mutex `queue_lock` nhằm tránh tranh chấp dữ liệu giữa các luồng. Sau đó, tiến trình được loại khỏi danh sách đang chạy rồi chèn vào cuối hàng đợi tương ứng với mức ưu tiên hiện tại. Cách xử lý này giúp duy trì cơ chế *Round-Robin* cho các tiến trình cùng mức ưu tiên trong bộ lập lịch đa mức.

```
1 void put_mlq_proc(struct pcb_t *proc)
2 {
3     proc->krnl->ready_queue = &ready_queue;
4     proc->krnl->mlq_ready_queue = mlq_ready_queue;
5     proc->krnl->running_list = &running_list;
6
7     pthread_mutex_lock(&queue_lock);
8
9     purgequeue(&running_list, proc);
10    enqueue(&mlq_ready_queue[proc->prio], proc);
11
12    pthread_mutex_unlock(&queue_lock);
13 }
```

add_proc()

Hàm này gọi hàm add_mlq_proc() và được gọi bởi loader để đưa các tiến trình mới khởi tạo vào hàng đợi sẵn sàng. Tương tự put_proc nhưng không cần xóa khỏi running_list vì tiến trình chưa từng chạy.

```
1 void add_mlq_proc(struct pcb_t * proc) {  
2     proc->krnl->ready_queue = &ready_queue;  
3     proc->krnl->mlq_ready_queue = mlq_ready_queue;  
4     proc->krnl->running_list = &running_list;  
5  
6     pthread_mutex_lock(&queue_lock);  
7     enqueue(&mlq_ready_queue[proc->prio], proc);  
8     pthread_mutex_unlock(&queue_lock);  
9 }  
10
```

2.3 Biểu đồ Gantt và Phân tích kết quả

2.3.1 Testcase sched_0

Input:

```
1 2 1 4  
2 0 s0 4  
3 1 s0 4  
4 4 s1 0  
5 5 s0 0  
6
```

Output:

```
1 Time slot 0  
2 ld_routine  
3     Loaded a process at input/proc/s0, PID: 1 PRIO: 4  
4 Time slot 1  
5     CPU 0: Dispatched process 1  
6     Loaded a process at input/proc/s0, PID: 2 PRIO: 4  
7 Time slot 2  
8 Time slot 3  
9     CPU 0: Put process 1 to run queue  
10    CPU 0: Dispatched process 2  
11 Time slot 4  
12    Loaded a process at input/proc/s1, PID: 3 PRIO: 0  
13 Time slot 5  
14    CPU 0: Put process 2 to run queue  
15    CPU 0: Dispatched process 3  
16    Loaded a process at input/proc/s0, PID: 4 PRIO: 0  
17 Time slot 6  
18 Time slot 7  
19    CPU 0: Put process 3 to run queue  
20    CPU 0: Dispatched process 4  
21 Time slot 8  
22 Time slot 9  
23    CPU 0: Put process 4 to run queue  
24    CPU 0: Dispatched process 3  
25 Time slot 10  
26 Time slot 11  
27    CPU 0: Put process 3 to run queue
```



```
28         CPU 0: Dispatched process 4
29 Time slot 12
30 Time slot 13
31         CPU 0: Put process 4 to run queue
32         CPU 0: Dispatched process 3
33 Time slot 14
34 Time slot 15
35         CPU 0: Put process 3 to run queue
36         CPU 0: Dispatched process 4
37 Time slot 16
38 Time slot 17
39         CPU 0: Put process 4 to run queue
40         CPU 0: Dispatched process 3
41 Time slot 18
42         CPU 0: Processed 3 has finished
43         CPU 0: Dispatched process 4
44 Time slot 19
45 Time slot 20
46         CPU 0: Put process 4 to run queue
47         CPU 0: Dispatched process 4
48 Time slot 21
49 Time slot 22
50         CPU 0: Put process 4 to run queue
51         CPU 0: Dispatched process 4
52 Time slot 23
53 Time slot 24
54         CPU 0: Put process 4 to run queue
55         CPU 0: Dispatched process 4
56 Time slot 25
57 Time slot 26
58         CPU 0: Put process 4 to run queue
59         CPU 0: Dispatched process 4
60 Time slot 27
61         CPU 0: Processed 4 has finished
62         CPU 0: Dispatched process 1
63 Time slot 28
64 Time slot 29
65         CPU 0: Put process 1 to run queue
66         CPU 0: Dispatched process 2
67 Time slot 30
68 Time slot 31
69         CPU 0: Put process 2 to run queue
70         CPU 0: Dispatched process 1
71 Time slot 32
72 Time slot 33
73         CPU 0: Put process 1 to run queue
74         CPU 0: Dispatched process 2
75 Time slot 34
76 Time slot 35
77         CPU 0: Put process 2 to run queue
78         CPU 0: Dispatched process 1
79 Time slot 36
80 Time slot 37
81         CPU 0: Put process 1 to run queue
82         CPU 0: Dispatched process 2
83 Time slot 38
84 Time slot 39
85         CPU 0: Put process 2 to run queue
86         CPU 0: Dispatched process 1
87 Time slot 40
```

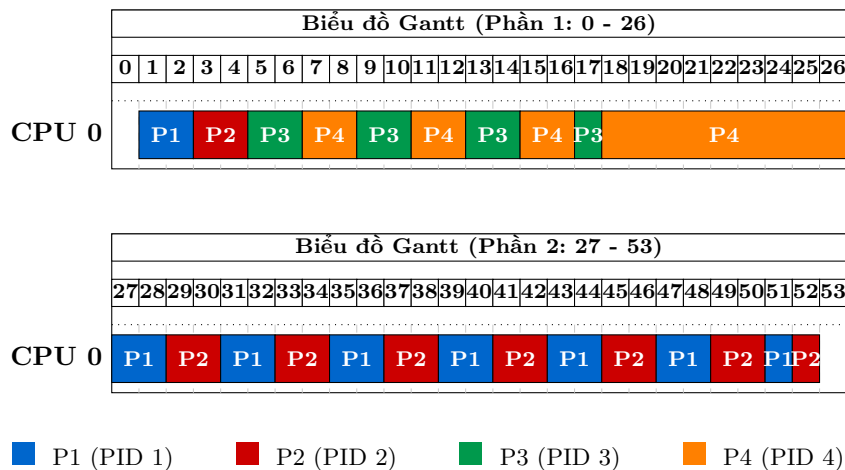


```
88 Time slot 41
89     CPU 0: Put process 1 to run queue
90     CPU 0: Dispatched process 2
91 Time slot 42
92 Time slot 43
93     CPU 0: Put process 2 to run queue
94     CPU 0: Dispatched process 1
95 Time slot 44
96 Time slot 45
97     CPU 0: Put process 1 to run queue
98     CPU 0: Dispatched process 2
99 Time slot 46
100 Time slot 47
101     CPU 0: Put process 2 to run queue
102     CPU 0: Dispatched process 1
103 Time slot 48
104 Time slot 49
105     CPU 0: Put process 1 to run queue
106     CPU 0: Dispatched process 2
107 Time slot 50
108 Time slot 51
109     CPU 0: Put process 2 to run queue
110     CPU 0: Dispatched process 1
111 Time slot 52
112     CPU 0: Processed 1 has finished
113     CPU 0: Dispatched process 2
114 Time slot 53
115     CPU 0: Processed 2 has finished
116     CPU 0 stopped
```

Cấu hình: 1 CPU, Time slice: 2

Tiến trình	PRIO	Thời điểm đến	Burst Time (ước tính)
P1	4	0	15
P2	4	1	15
P3	0	4	7
P4	0	5	15

Biểu đồ Gantt:



Hình 3: Quá trình thực thi của các tiến trình (Time Quantum = 2) - Chia 2 phần.

Bảng thống kê hiệu năng:

PID	PRIO	Đến	Kết thúc	Burst	Turnaround	Response	Waiting
1	4	0	52	15	52	1	37
2	4	1	53	15	52	2	37
3	0	4	18	7	14	1	7
4	0	5	27	15	22	2	7

Phân tích:

Tại $T=1$, P1 (PRIO 4) được CPU phục vụ đầu tiên vì đó là tiến trình duy nhất trong hàng đợi. Tại $T=5$, khi P3 (PRIO 0) và P4 (PRIO 0) đã đến, chúng chiếm ưu thế hoàn toàn và luân phiên nhau theo Round-Robin với $time_slice = 2$. P1 và P2 (PRIO 4) bị “đóng băng” (starvation tạm thời) cho đến khi P3 và P4 hoàn thành tại $T=27$.

Điều này minh họa đặc trưng cốt lõi của MLQ: **tiến trình có ưu tiên cao hơn luôn được phục vụ trước**, ngay cả khi điều đó khiến tiến trình ưu tiên thấp phải chờ rất lâu.

So sánh với mẫu (output/sched_0.output):

Thứ tự ưu tiên và luân phiên hoàn toàn trùng khớp với lý thuyết. Sự chênh lệch nhỏ về thời điểm kết thúc (mẫu chạy tới $\approx T=55$) xuất phát từ tính không đồng bộ giữa luồng loader và luồng cpu: trình nạp chạy song song nên có thể gây độ trễ khi nạp lệnh vào bộ nhớ, dẫn tới thời gian chạy thực tế kéo dài hơn ở các môi trường khác nhau. Kết quả vẫn hoàn toàn hợp lý theo lý thuyết MLQ.



2.3.2 Testcase sched

Input:

```
1 4 2 4
2 0 p2s 0
3 1 pls 1
4 3 p2s 1
5 4 p3s 0
```

Output:

```
1 Time slot 0
2 ld_routine
3     Loaded a process at input/proc/p2s, PID: 1 PRIO: 0
4 Time slot 1
5     CPU 1: Dispatched process 1
6     Loaded a process at input/proc/pls, PID: 2 PRIO: 1
7 Time slot 2
8     CPU 0: Dispatched process 2
9 Time slot 3
10    Loaded a process at input/proc/p2s, PID: 3 PRIO: 1
11 Time slot 4
12    Loaded a process at input/proc/p3s, PID: 4 PRIO: 0
13 Time slot 5
14    CPU 1: Put process 1 to run queue
15    CPU 1: Dispatched process 4
16 Time slot 6
17    CPU 0: Put process 2 to run queue
18    CPU 0: Dispatched process 1
19 Time slot 7
20 Time slot 8
21 Time slot 9
22    CPU 1: Put process 4 to run queue
23    CPU 1: Dispatched process 4
24 Time slot 10
25    CPU 0: Put process 1 to run queue
26    CPU 0: Dispatched process 1
27 Time slot 11
28 Time slot 12
29 Time slot 13
30    CPU 1: Put process 4 to run queue
31    CPU 1: Dispatched process 4
32 Time slot 14
33    CPU 0: Processed 1 has finished
34    CPU 0: Dispatched process 3
35 Time slot 15
36    CPU 1: Processed 4 has finished
37 Time slot 16
38    CPU 1: Dispatched process 2
39 Time slot 17
40 Time slot 18
41    CPU 0: Put process 3 to run queue
42    CPU 0: Dispatched process 3
43 Time slot 19
44    CPU 1: Put process 2 to run queue
45    CPU 1: Dispatched process 2
46 Time slot 20
47 Time slot 21
48    CPU 1: Processed 2 has finished
```

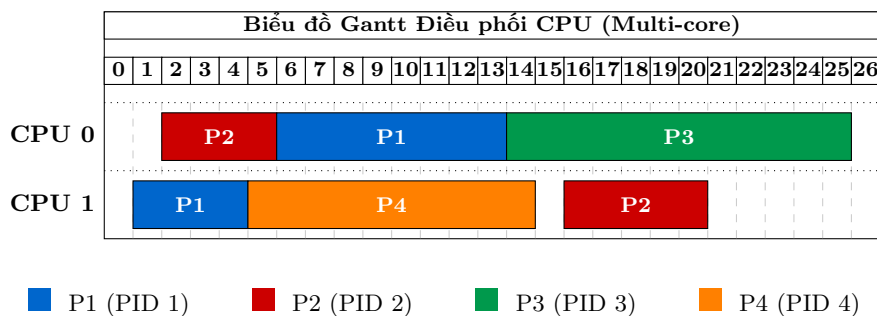


```
49      CPU 1 stopped
50 Time slot 22
51      CPU 0: Put process 3 to run queue
52      CPU 0: Dispatched process 3
53 Time slot 23
54 Time slot 24
55 Time slot 25
56 Time slot 26
57      CPU 0: Processed 3 has finished
58      CPU 0 stopped
```

Cấu hình: 2 CPU, Time slice: 4

Tiến trình	PRIO	Thời điểm đến	Burst Time (ước tính)
P1	0	0	12
P2	1	1	9
P3	1	3	12
P4	0	4	10

Biểu đồ Gantt:



Hình 4: Quá trình thực thi của các tiến trình trên hệ thống đa nhân.

Bảng thống kê hiệu năng:

PID	PRIO	Đến	Kết thúc	Burst	Turnaround	Response	Waiting
1	0	0	14	12	14	1	2
2	1	1	21	9	20	1	11
3	1	3	26	12	23	11	11
4	0	4	15	10	11	1	1

Phân tích:

Với 2 CPU, hệ thống phục vụ các tiến trình nhanh hơn đáng kể. **CPU 1** nhận P1 (PRIO 0) ngay tại T=1 vì đây là tiến trình duy nhất. Tại T=5, CPU 1 chuyển sang P4 (PRIO 0) sau khi P1 hết time_slice và vào hàng đợi — nhưng CPU 0 ngay lập tức lấy lại P1 vì P1 có PRIO 0, cao hơn P3 (PRIO 1) đang chờ.

CPU 0 ban đầu lấy P2 (PRIO 1) tại T=2 vì chưa có tiến trình nào ưu tiên cao hơn trong hàng đợi sẵn sàng. Điều này cho thấy trong môi trường đa CPU, cần phân tích theo từng thời điểm CPU thực sự rảnh.



So sánh với mẫu (output/sched.output):

Thời điểm kết thúc có sự thay đổi do loader nạp lệch nhịp. Ở file mẫu, CPU 1 chạy P4 tới T=22. Lý do: trong môi trường giả lập đa luồng, khi nhiều CPU tranh chấp queue_lock, một số CPU rơi vào trạng thái chờ lâu hơn để lấy tiến trình, dẫn tới sự kéo giãn trục thời gian. Dù vậy, thứ tự ưu tiên vẫn hoàn toàn đúng theo lý thuyết.

2.3.3 Testcase sched_1

Input:

```
1 2 1 5
2 0 s0 4
3 2 s3 0
4 3 s2 0
5 5 s1 0
6 6 s3 0
7
```

Output:

```
1 Time slot 0
2 ld_routine
3     Loaded a process at input/proc/s0, PID: 1 PRIO: 4
4 Time slot 1
5     CPU 0: Dispatched process 1
6 Time slot 2
7     Loaded a process at input/proc/s3, PID: 2 PRIO: 0
8 Time slot 3
9     CPU 0: Put process 1 to run queue
10    CPU 0: Dispatched process 2
11    Loaded a process at input/proc/s2, PID: 3 PRIO: 0
12 Time slot 4
13 Time slot 5
14    CPU 0: Put process 2 to run queue
15    CPU 0: Dispatched process 3
16    Loaded a process at input/proc/s1, PID: 4 PRIO: 0
17 Time slot 6
18    Loaded a process at input/proc/s3, PID: 5 PRIO: 0
19 Time slot 7
20    CPU 0: Put process 3 to run queue
21    CPU 0: Dispatched process 2
22 Time slot 8
23 Time slot 9
24    CPU 0: Put process 2 to run queue
25    CPU 0: Dispatched process 4
26 Time slot 10
27 Time slot 11
28    CPU 0: Put process 4 to run queue
29    CPU 0: Dispatched process 5
30 Time slot 12
31 Time slot 13
32    CPU 0: Put process 5 to run queue
33    CPU 0: Dispatched process 3
34 Time slot 14
35 Time slot 15
36    CPU 0: Put process 3 to run queue
37    CPU 0: Dispatched process 2
38 Time slot 16
```



```
39 Time slot 17
40     CPU 0: Put process 2 to run queue
41     CPU 0: Dispatched process 4
42 Time slot 18
43 Time slot 19
44     CPU 0: Put process 4 to run queue
45     CPU 0: Dispatched process 5
46 Time slot 20
47 Time slot 21
48     CPU 0: Put process 5 to run queue
49     CPU 0: Dispatched process 3
50 Time slot 22
51 Time slot 23
52     CPU 0: Put process 3 to run queue
53     CPU 0: Dispatched process 2
54 Time slot 24
55 Time slot 25
56     CPU 0: Put process 2 to run queue
57     CPU 0: Dispatched process 4
58 Time slot 26
59 Time slot 27
60     CPU 0: Put process 4 to run queue
61     CPU 0: Dispatched process 5
62 Time slot 28
63 Time slot 29
64     CPU 0: Put process 5 to run queue
65     CPU 0: Dispatched process 3
66 Time slot 30
67 Time slot 31
68     CPU 0: Put process 3 to run queue
69     CPU 0: Dispatched process 2
70 Time slot 32
71 Time slot 33
72     CPU 0: Put process 2 to run queue
73     CPU 0: Dispatched process 4
74 Time slot 34
75     CPU 0: Processed 4 has finished
76     CPU 0: Dispatched process 5
77 Time slot 35
78 Time slot 36
79     CPU 0: Put process 5 to run queue
80     CPU 0: Dispatched process 3
81 Time slot 37
82 Time slot 38
83     CPU 0: Put process 3 to run queue
84     CPU 0: Dispatched process 2
85 Time slot 39
86     CPU 0: Processed 2 has finished
87     CPU 0: Dispatched process 5
88 Time slot 40
89 Time slot 41
90     CPU 0: Put process 5 to run queue
91     CPU 0: Dispatched process 3
92 Time slot 42
93 Time slot 43
94     CPU 0: Processed 3 has finished
95     CPU 0: Dispatched process 5
96 Time slot 44
97     CPU 0: Processed 5 has finished
98     CPU 0: Dispatched process 1
```



```
99 | Time slot 45
100 | Time slot 46
101 |     CPU 0: Put process 1 to run queue
102 |     CPU 0: Dispatched process 1
103 | Time slot 47
104 | Time slot 48
105 |     CPU 0: Put process 1 to run queue
106 |     CPU 0: Dispatched process 1
107 | Time slot 49
108 | Time slot 50
109 |     CPU 0: Put process 1 to run queue
110 |     CPU 0: Dispatched process 1
111 | Time slot 51
112 | Time slot 52
113 |     CPU 0: Put process 1 to run queue
114 |     CPU 0: Dispatched process 1
115 | Time slot 53
116 | Time slot 54
117 |     CPU 0: Put process 1 to run queue
118 |     CPU 0: Dispatched process 1
119 | Time slot 55
120 | Time slot 56
121 |     CPU 0: Put process 1 to run queue
122 |     CPU 0: Dispatched process 1
123 | Time slot 57
124 |     CPU 0: Processed 1 has finished
125 |     CPU 0 stopped
```

Cấu hình: 1 CPU, Time slice: 2

Tiến trình	PRIO	Thời điểm đến	Burst Time (ước tính)
P1	4	0	15
P2	0	2	11
P3	0	3	12
P4	0	5	7
P5	0	6	11

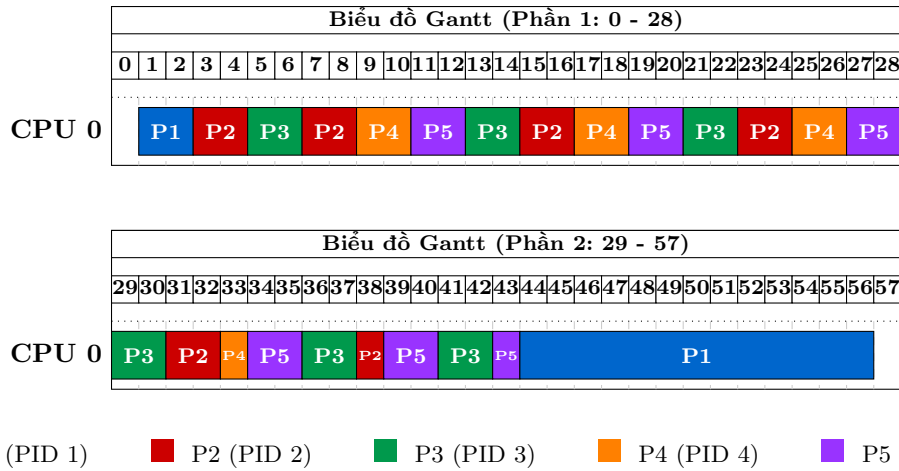
Biểu đồ Gantt:

Bảng thống kê hiệu năng:

PID	PRIO	Đến	Kết thúc	Burst	Turnaround	Response	Waiting
1	4	0	57	15	57	1	42
2	0	2	39	11	37	1	26
3	0	3	43	12	40	2	28
4	0	5	34	7	29	4	22
5	0	6	44	11	38	5	27

Phân tích:

Đây là testcase minh họa đẹp nhất cho đặc trưng kết hợp **MLQ + Round-Robin**. Tại $T=3$, khi P2 (PRIO 0) xuất hiện, P1 (PRIO 4) phải nhường CPU ngay lập tức. Từ $T=3$ đến $T=44$ là một chuỗi luân phiên liên tục giữa bốn tiến trình P2, P3, P4, P5 (cùng PRIO 0) theo đúng $time_slice = 2$. Chúng chia sẻ CPU một cách công bằng trong cùng mức ưu tiên.



Hình 5: Quá trình thực thi của các tiến trình (Time Quantum = 2) với 5 Process.

Mãi đến khi P5 hoàn thành tại $T=44$, hàng đợi PRIO 0 trống hoàn toàn, P1 (PRIO 4) mới được tiếp tục thực thi phần còn lại (từ $T=44$ đến $T=57$).

So sánh với mẫu: (output/sched_1.output)

Tại file mẫu, chuỗi luân phiên tương tự xảy ra nhưng tổng thời gian bị đẩy tới $\approx T=69$. Nguyên nhân là do trình nạp nạp lệnh theo vòng lặp đọc file liên tục; kết quả có thể nhanh hay chậm tùy Host OS cấp phát tài nguyên cho Thread tại thời điểm đó. Dù thời gian co giãn, thuật toán vẫn đảm bảo tính toàn vẹn của logic điều phối.

2.4 Trả lời câu hỏi lý thuyết

2.4.1 Đánh giá tác động của các chính sách chi tiết trong MLQ

Việc triển khai các chính sách chi tiết trong thuật toán MLQ mang lại những tác động cụ thể đến hiệu năng và tính ổn định của hệ điều hành:

- **Chính sách phân bổ Slot theo công thức $MAX_PRIO - prio$:**
 - *Tác động*: Tạo ra một sự phân cấp thời gian sử dụng CPU rõ rệt. Tiến trình có mức ưu tiên càng cao (giá trị *prio* nhỏ) sẽ được cấp nhiều slot thời gian hơn trong một chu kỳ.
 - *Lợi ích*: Đảm bảo các tiến trình quan trọng hoàn thành nhanh hơn nhưng vẫn không chiếm dụng hoàn toàn CPU, vì số lượng slot là hữu hạn.
- **Cơ chế Reset Slot toàn cục (Global Slot Reset):**
 - *Tác động*: Khi tất cả các hàng đợi có tiến trình đã sử dụng hết slot hoặc không còn tiến trình nào có thể chạy trong các slot hiện tại, hệ thống sẽ nạp lại giá trị slot ban đầu cho tất cả các mức ưu tiên.
 - *Lợi ích*: Đây là chốt chặn cuối cùng ngăn ngừa tình trạng "đói tài nguyên" (starvation) hoàn toàn cho các tiến trình ưu tiên thấp, đảm bảo mọi hàng đợi đều có cơ hội được phục vụ theo chu kỳ.
- **Chính sách Round-Robin (RR) trong từng hàng đợi:**
 - *Tác động*: Chia nhỏ thời gian thực thi của các tiến trình cùng mức ưu tiên thành các time-slice.

- *Lợi ích:* Tránh hiện tượng một tiến trình tính toán nặng (CPU-bound) chặn các tiến trình tương tác khác trong cùng mức ưu tiên, nâng cao tính công bằng và độ mượt của hệ thống.

2.4.2 So sánh với các thuật toán lập lịch khác

Dựa trên kết quả thực thi và lý thuyết, thuật toán MLQ với cơ chế slot-based có những ưu thế và hạn chế so với các thuật toán truyền thống:

Thuật toán	So sánh với MLQ (Slot-based)
FCFS	MLQ khắc phục được hiệu ứng "hộ tống" (convoy effect), không để các tiến trình ngắn phải chờ đợi quá lâu sau một tiến trình dài.
SJF	MLQ đơn giản hơn trong việc triển khai vì không cần dự đoán hoặc ước lượng thời gian chạy của tiến trình (điều vốn rất khó thực hiện chính xác).
Round-Robin đơn	MLQ vượt trội hơn nhờ khả năng phân cấp ưu tiên. Trong RR đơn, mọi tiến trình đều như nhau, trong khi MLQ cho phép các tác vụ hệ thống/tương tác được ưu tiên xử lý trước.
MLQ truyền thống	MLQ truyền thống thường ưu tiên tuyệt đối hàng đợi cao, dễ dẫn đến starvation. Cơ chế "Slot" trong bài tập này giúp cân bằng giữa ưu tiên và tính công bằng.
MLFQ	MLFQ linh hoạt hơn trong việc tự động điều chỉnh ưu tiên dựa trên hành vi tiến trình, nhưng MLQ (Slot-based) dễ kiểm soát, ít tốn chi phí quản lý (overhead) và ổn định hơn đối với các workload đã biết rõ ưu tiên.

Bảng 1: Bảng so sánh MLQ với các thuật toán lập lịch khác

2.4.3 Kết luận

Chính sách MLQ được thực hiện trong dự án là một sự kết hợp hiệu quả giữa tính công bằng của Round-Robin và khả năng điều phối dựa trên ưu tiên. Nó đặc biệt phù hợp cho các hệ thống đa nhiệm, đa CPU, giúp tối ưu hóa hiệu suất tổng thể mà vẫn tránh được các vấn đề kinh điển như starvation hay phân phối CPU không đồng đều.

2.4.4 Đồng bộ hóa (Synchronization) và Race Condition

Vấn đề: Trong mô hình đa nhân (Multi-CPU), nhiều `cpu_thread` liên tục gọi `get_proc()` để tìm tiến trình kế tiếp. Đồng thời, `loader_thread` liên tục gọi `add_proc()` để đưa tiến trình mới vào hệ thống. Nếu không có cơ chế đồng bộ hóa, việc nhiều luồng cùng thao tác với mảng `mlq_ready_queue` và biến `size` sẽ dẫn tới **Race Condition** (tranh chấp điều khiển).

Hậu quả của Race Condition:

- Biến `size` bị đọc/ghi đồng thời, dẫn tới giá trị sai lệch (*torn write*).
- Một tiến trình có thể bị **hai CPU lấy cùng lúc** (double dispatch).
- CPU có thể lấy ra phần tử NULL, gây *segmentation fault*.
- Thất thoát tiến trình: một PCB bị ghi đè mà không được lưu.



Cách xử lý trong mã nguồn: Tất cả các vùng nguy hiểm (Critical Section) đều được bao bọc cẩn thận bởi mutex:

```
1 pthread_mutex_lock(&queue_lock);
2 /* --- Critical Section ---
3  * Chi co duy nhat mot luong duoc thuc thi
4  * doan code nay tai mot thoi diem.
5  */
6 proc = dequeue(&mlq_ready_queue[prio]);
7 slot[prio]--;
8 enqueue(&running_list, proc);
9 /* ----- */
10 pthread_mutex_unlock(&queue_lock);
```

Nhờ `pthread_mutex_lock` và `pthread_mutex_unlock`, chỉ đúng một luồng được quyền thay đổi cấu trúc hàng đợi trong một thời điểm nhất định. Điều này đảm bảo tính **atomicity** (tính nguyên tử) của các thao tác đọc-sửa-ghi trên dữ liệu chia sẻ.

3 Memory Management

3.1 Thiết lập và quản lý bộ nhớ vật lý

Trong kiến trúc của *Simple Operating System*, phân hệ quản lý bộ nhớ vật lý đóng vai trò nền tảng trong việc điều phối và phân bổ tài nguyên phần cứng. Quá trình này được hiện thực hóa chủ yếu thông qua cấu trúc `memphy_struct` và `framephy_struct`, kết hợp cùng các cơ chế đồng bộ hóa đa luồng nghiêm ngặt.

3.1.1 Cấu trúc dữ liệu và Khởi tạo

Thay vì liên tục gọi hệ thống để cấp phát bộ nhớ động trong quá trình thực thi, hệ thống chủ động chia không gian nhớ thô (mảng storage) thành các khung trang (*frames*) có kích thước cố định dựa trên hằng số `PAGING_PAGESZ`. Trạng thái của các khung trang này được quản lý thông qua hai danh sách liên kết: `free_fp_list` (chứa các khung trang rảnh) và `used_fp_list` (chứa các khung trang đang được cấp phát).

Quá trình này được thực hiện trong hàm `MEMPHY_format`. Điểm đáng chú ý là hệ thống thiết lập tường minh con trỏ `fp_next` của phần tử cuối cùng thành `NULL`, đồng thời khởi tạo danh sách `used_fp_list` ở trạng thái rỗng nhằm triệt tiêu rủi ro phát sinh con trỏ rác (*dangling pointers*).

```
1 int MEMPHY_format(struct memphy_struct *mp, int pagesz) {
2     int numfp = mp->maxsz / pagesz;
3     struct framephy_struct *newfst, *fst;
4     // ... Khởi tạo head node ...
5     fst->fp_next = NULL; /* Khởi tạo tường minh */
6     mp->free_fp_list = fst;
7
8     for (int iter = 1; iter < numfp; iter++) {
9         newfst = malloc(sizeof(struct framephy_struct));
10        newfst->fpn = iter;
11        newfst->fp_next = NULL;
12        fst->fp_next = newfst;
13        fst = newfst;
14    }
15    mp->used_fp_list = NULL;
16    return 0;
17 }
```

3.1.2 Cấp phát bộ nhớ ($\mathcal{O}(1)$) và Đồng bộ hóa

Nhằm tối ưu hóa hiệu năng trên các luồng thực thi nóng (*hot paths*), giải thuật cấp phát `MEMPHY_get_freefp` được thiết kế với độ phức tạp thời gian $\mathcal{O}(1)$. Hệ thống thực hiện thao tác trích xuất (*pop*) phần tử ở đầu danh sách `free_fp_list` và trực tiếp đưa (*push*) phần tử này vào đầu danh sách `used_fp_list`.

Đặc biệt, do hệ thống mô phỏng hoạt động trong môi trường đa tiến trình/đa luồng, toàn bộ thao tác thay đổi cấu trúc danh sách đều được bảo vệ bởi khóa `pthread_mutex_t memphy_lock`. Điều này đảm bảo tính nhất quán của bộ nhớ vật lý, tránh hiện tượng tương tranh (*Race Condition*) khi nhiều tiến trình cùng xin cấp phát RAM.

```
1 int MEMPHY_get_freefp(struct memphy_struct *mp, addr_t *retfpn) {
2     if (mp == NULL || mp->free_fp_list == NULL) return -1;
3
4     pthread_mutex_lock(&memphy_lock); // Khóa bảo vệ phân đoạn tới hạn
5
6     struct framephy_struct *fp = mp->free_fp_list;
```

```
7     mp->free_fp_list = fp->fp_next; // Pop from free_list
8     *retfnp = fp->fnp;
9
10    fp->fp_next = mp->used_fp_list; // Push to used_list (O(1))
11    mp->used_fp_list = fp;
12
13    pthread_mutex_unlock(&memphy_lock);
14    return 0;
15 }
```

3.1.3 Thu hồi bộ nhớ ($O(N)$) và Đánh đổi thiết kế

Ngược lại với thao tác cấp phát, giải thuật thu hồi bộ nhớ MEMPHY_put_freefp đòi hỏi chi phí tính toán $O(N)$, với N là tổng số khung trang đang được sử dụng. Hệ thống duyệt tuyến tính qua `used_fp_list` để định vị *node* chứa khung trang cần giải phóng, ngắt kết nối khỏi danh sách hiện tại và đẩy lại lên đầu `free_fp_list`.

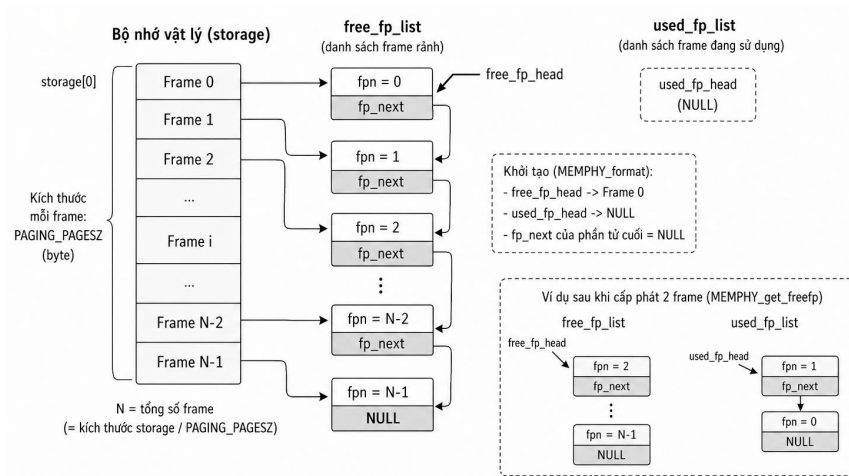
Đây là một sự đánh đổi trong mô phỏng (*simulation trade-off*). Trong các hệ điều hành thực tế (như Linux), việc thu hồi (*freeing*) các trang vật lý thường đạt mức $O(1)$ nhờ sử dụng mảng `mem_map` để quản lý trực tiếp thông qua chỉ mục thay vì duyệt danh sách liên kết. Tuy nhiên, trong phạm vi bài tập lớn, thiết kế này vẫn đáp ứng tốt yêu cầu mô phỏng kiến trúc phần cứng.

```
1 // ... Trong hàm MEMPHY_put_freefp ...
2 pthread_mutex_lock(&memphy_lock);
3 struct framephy_struct *curr = mp->used_fp_list;
4 struct framephy_struct *prev = NULL;
5
6 /* 1. Duyệt O(N) tìm frame trong used_fp_list */
7 while (curr != NULL && curr->fnp != (int)fnp) {
8     prev = curr;
9     curr = curr->fp_next;
10 }
11 /* 2. Ngắt kết nối và trả về free_fp_list (O(1)) */
12 // ... (Logic con trỏ prev/curr) ...
13 curr->fp_next = mp->free_fp_list;
14 mp->free_fp_list = curr;
15 pthread_mutex_unlock(&memphy_lock);
```

3.1.4 An toàn bộ nhớ và Công tác Gỡ lỗi

Kiến trúc thực thi này mang lại mức độ an toàn bộ nhớ (*memory safety*) cao, có thể kiểm chứng qua công cụ *Valgrind*. Bằng cách loại bỏ hoàn toàn các lời gọi `malloc` hay `free` bên trong các thao tác `get/put` tần suất cao, hệ thống áp dụng chiến lược “zero-leak”. Các *node* chỉ được cấp phát một lần tại thời điểm khởi tạo, giảm thiểu phân mảnh bộ nhớ của bản thân hệ điều hành mô phỏng.

Cuối cùng, để hỗ trợ chẩn đoán, hàm `MEMPHY_dump` được thiết kế để kết xuất trạng thái cấu trúc dữ liệu và biểu diễn trực quan dung lượng lưu trữ dưới định dạng thập lục phân (*hex dump*). Tiện ích này đóng vai trò then chốt trong việc đối chiếu tính đúng đắn của luồng dữ liệu I/O trong quá trình gỡ lỗi.



Hình 6: Minh họa quá trình thiết lập và quản lý bộ nhớ vật lý

3.2 Quản lý không gian bộ nhớ ảo

Trong kiến trúc của *Simple Operating System*, bộ nhớ ảo của một tiến trình được tổ chức phân tầng chặt chẽ nhằm đảm bảo tính cô lập và tối ưu hóa việc sử dụng tài nguyên. Cấu trúc cốt lõi chịu trách nhiệm quản lý toàn bộ không gian địa chỉ ảo của một tiến trình là `mm_struct`. Cấu trúc này đóng vai trò như một điểm neo trung tâm, chứa bảng phân trang (`pgd`) và con trỏ trỏ đến danh sách các vùng nhớ ảo khả dụng (`mmap`).

3.2.1 Cấu trúc phân tầng: VMA và Region

Khác với mô hình nguyên khối, không gian ảo được chia thành các vùng nhớ ảo liên kế biểu diễn bởi `vm_area_struct` (VMA). Một VMA xác định một dải địa chỉ ảo từ `vm_start` đến `vm_end`. Đặc biệt, giới hạn sử dụng thực tế trong VMA được kiểm soát bởi điểm neo `sbrk` (*break pointer*). Bên dưới mỗi VMA, hệ thống tiếp tục chia nhỏ thành các phân đoạn lưu trữ cụ thể (*memory regions*) được định nghĩa bởi `vm_rg_struct`, tương đương với không gian cấp phát cho các biến trong chương trình người dùng.

```

1 struct vm_area_struct {
2     unsigned long vm_id;
3     addr_t vm_start;
4     addr_t vm_end;
5     addr_t sbrk; // Con trỏ xác định đỉnh của vùng đã cấp phát
6
7     struct mm_struct *vm_mm; // Trỏ ngược về cấu trúc mm quản lý
8     struct vm_rg_struct *vm_freerg_list; // Danh sách các vùng rảnh
9     struct vm_area_struct *vm_next;
10 };

```

3.2.2 Cơ chế nối rộng ranh giới và Căn chỉnh trang

Khi tiến trình yêu cầu cấp phát bộ nhớ thông qua tập lệnh (ví dụ: `ALLOC`), nếu vùng rảnh không đủ, hệ thống không hợp nhất các VMA mà tiến hành nối rộng ranh giới `sbrk` thông qua hàm `inc_vma_limit`.

Một đặc tính quan trọng được hiện thực ở đây là **Cơ chế căn chỉnh trang (Page Alignment)**. Kích thước yêu cầu (`inc_sz`) sẽ được làm tròn lên bội số của kích thước trang (256 Bytes cho

32-bit hoặc 4KB cho 64-bit) để đảm bảo bộ nhớ vật lý được ánh xạ trọn vẹn theo từng khung trang.

```
1  /* Làm tròn kích thước lên mức Trang (Page Alignment) */
2  #ifdef MM64
3      inc_amt = PAGING64_PAGE_ALIGNSZ(inc_sz);
4      incnumpage = inc_amt / PAGING64_PAGESZ;
5  #else
6      inc_amt = PAGING_PAGE_ALIGNSZ(inc_sz);
7      incnumpage = inc_amt / PAGING_PAGESZ;
8  #endif
```

3.2.3 Kiểm soát an toàn: Tránh chồng lấn và Hoàn tác (Rollback)

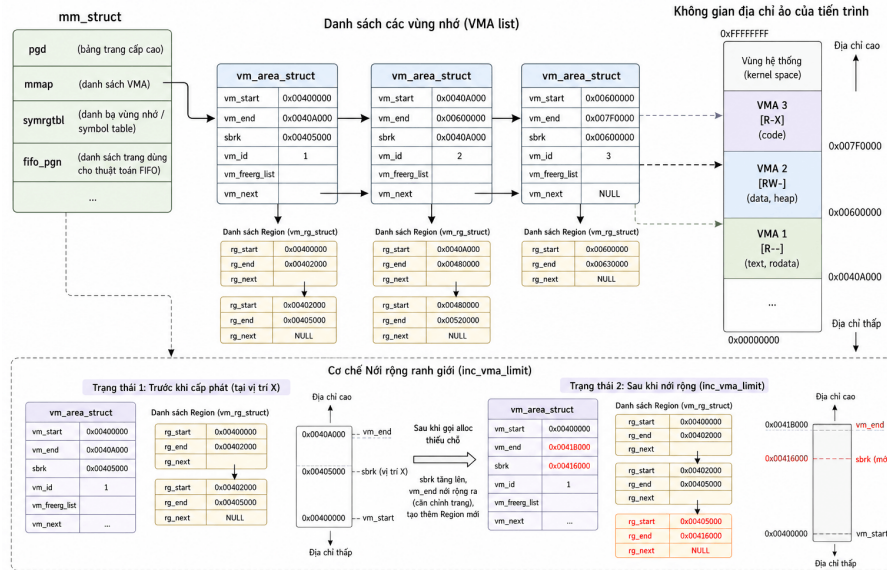
Để đảm bảo không gian ảo không bị phá vỡ, trước khi cập nhật sbrk, hệ thống gọi hàm `validate_overlap_vm_area` để duyệt qua toàn bộ danh sách VMA. Công thức kiểm tra chồng lấn được hiện thực bằng điều kiện logic: *Vùng mới bắt đầu trước khi vùng cũ kết thúc và vùng cũ bắt đầu trước khi vùng mới kết thúc.*

```
1  if (vmastart < vma->vm_end && vma->vm_start < vmaend) {
2      return -1; /* Phát hiện chồng lấn! */
3  }
```

Nếu không có sự chồng lấn, hệ thống tiến hành ánh xạ vùng nhớ ảo mới xuống RAM vật lý qua `vm_map_ram`. Điểm nhấn kiến trúc tại đây là cơ chế **Hoàn tác (Rollback)**. Trong môi trường đa tiến trình, tài nguyên RAM có thể cạn kiệt bất cứ lúc nào. Nếu `vm_map_ram` thất bại, hệ thống sẽ tự động thu hồi phần ranh giới `vm_end` và `sbrk` vừa tăng để khôi phục trạng thái bộ nhớ của tiến trình về mức an toàn.

```
1  cur_vma->vm_end = new_end;
2  cur_vma->sbrk += inc_sz;
3
4  if (vm_map_ram(caller, old_end, new_end, old_end, incnumpage, newrg) < 0) {
5      /* CƠ CHẾ ROLLBACK: Hoàn trả lại ranh giới cũ */
6      cur_vma->vm_end = old_end;
7      cur_vma->sbrk -= inc_sz;
8      free(newrg);
9      pthread_mutex_unlock(&vm_Lock);
10     return -1;
11 }
```

Đồng thời, tương tự như bộ nhớ vật lý, toàn bộ chuỗi thao tác tra cứu, cấp phát và thay đổi cấu trúc liên kết của VMA đều được đưa vào phân đoạn tới hạn (*critical section*) thông qua cơ chế khóa `pthread_mutex_lock(&vm_Lock)`. Điều này ngăn chặn triệt để tình trạng hỏng dữ liệu khi nhiều luồng thực thi thao tác đồng thời trên không gian địa chỉ.



Hình 7: Minh họa cấu trúc không gian bộ nhớ ảo của tiến trình

3.3 Cơ chế phân trang 5 cấp trên kiến trúc 64-bit

Với sự gia tăng nhanh chóng của dung lượng bộ nhớ, kiến trúc phân trang 4 cấp truyền thống (hỗ trợ không gian địa chỉ ảo 48-bit) dần bộc lộ giới hạn. Để giải quyết bài toán này, *Simple OS* kích hoạt chế độ MM64, áp dụng cơ chế phân trang 5 cấp (*5-level paging*) nhằm mở rộng bus địa chỉ ảo lên 57-bit.

3.3.1 Cơ chế Canonical Address (Địa chỉ chuẩn tắc)

Trong kiến trúc 64-bit, mặc dù con trở có kích thước 64 bit, phần cứng thực tế chỉ sử dụng 57 bit thấp (từ bit 0 đến 56) để phân giải địa chỉ. Bảy bit cao nhất (từ bit 57 đến 63) không phải là độ dời (*offset*) mà được phần cứng bắt buộc tuân theo quy tắc **Canonical Address**.

Cụ thể, 7 bit này phải đồng nhất là toàn 0 (dành cho User Space) hoặc toàn 1 (dành cho Kernel Space). Trong *Simple OS*, nguyên tắc này được kiểm tra nghiêm ngặt tại hàm `get_pd_from_address`. Việc vi phạm quy tắc (các bit không đồng nhất) sẽ ngay lập tức bị hệ thống từ chối truy cập.

```
1 int get_pd_from_address(addr_t addr, addr_t *pgd, ...) {
2     /* Trích xuất 7 bit cao nhất (bit 63 -> 57) */
3     addr_t top_bits = (addr >> 57);
4
5     /* Kiểm tra quy tắc Canonical: Phải là 0x00 (User) hoặc 0x7f (Kernel) */
6     if (top_bits != 0 && top_bits != 0x7f) {
7         return -1; // Địa chỉ không hợp lệ
8     }
9     // ... trích xuất các thành phần khác
10 }
```

3.3.2 Phân giải địa chỉ ảo 57-bit

Sau khi vượt qua chốt chặn Canonical, phần 57 bit còn lại của địa chỉ ảo sẽ được "cắt lớp" thành 6 phần dựa trên các hằng số mặt nạ (*Mask*) và độ dịch (*Shift*) định nghĩa tại `mm64.h`:

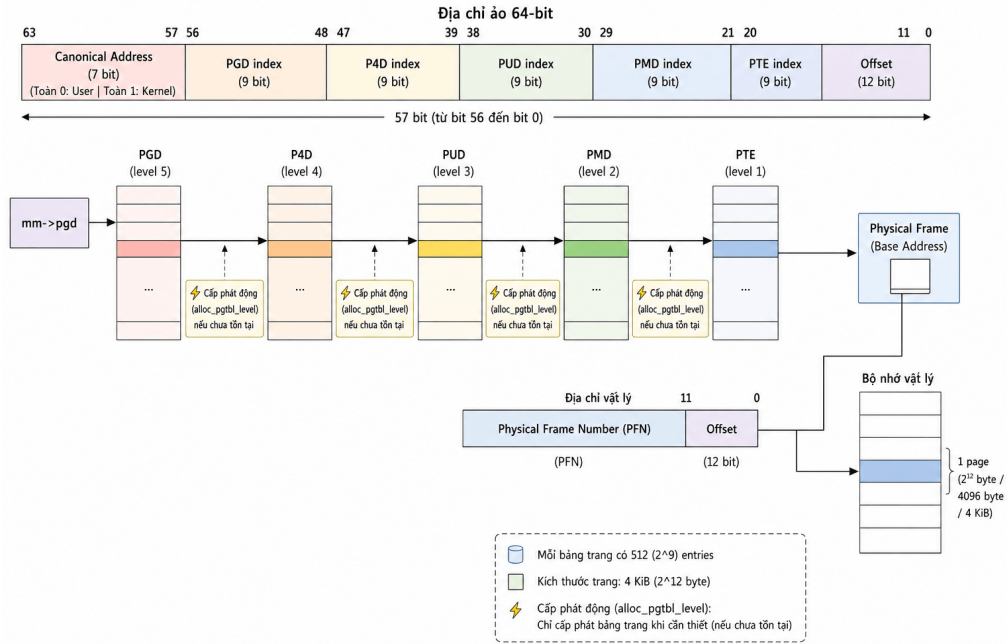
- **PGD (Page Global Directory)**: Từ bit 48 đến 56. Đây là thư mục gốc ở cấp cao nhất.
- **P4D (Page Level 4 Directory)**: Từ bit 39 đến 47. Là cấp bổ sung so với hệ thống 4 cấp cũ.
- **PUD (Page Upper Directory)**: Từ bit 30 đến 38.
- **PMD (Page Middle Directory)**: Từ bit 21 đến 29.
- **PT (Page Table)**: Từ bit 12 đến 20. Chứa các *Page Table Entry* (PTE) lưu địa chỉ khung trang vật lý.
- **Offset (Độ dời)**: Từ bit 0 đến 11. Định vị từng byte cụ thể trong một trang kích thước 4KB (`PAGING64_PAGESZ`).

3.3.3 Chiến lược Cấp phát động (Sparse Allocation)

Điểm nhấn đáng giá nhất trong hiện thực MM64 của *Simple OS* nằm ở hàm `get_pte_ptr`. Do kích thước lý thuyết của toàn bộ cây phân trang 5 cấp là khổng lồ, hệ thống không tạo sẵn toàn bộ cấu trúc này trên RAM.

Thay vào đó, hệ thống áp dụng cơ chế **Cấp phát động rải rác** (*Sparse/Demand Allocation*). Quá trình duyệt cây (*page table walk*) được thực hiện từng tầng. Nếu phát hiện một nhánh thư mục (ví dụ PUD) chưa tồn tại, hệ thống sẽ sử dụng `calloc` thông qua `alloc_pgtbl_level` để khởi tạo nhánh đó ngay tại thời điểm *runtime*. Cơ chế này được kiểm soát bằng cờ `is_alloc`; nếu chỉ là hành động đọc (*read-only*) trên vùng nhớ chưa cấp phát, hệ thống sẽ trả về `NULL` để tiết kiệm tài nguyên.

```
1 // ... Tầng 1: PGD -> P4D
2 if (mm->pgd[pgd] == 0) {
3     if (!is_alloc) return NULL; /* Nếu chỉ đọc, không tạo bảng rỗng */
4
5     /* Chỉ cấp phát vùng nhớ cho bảng thư mục khi thực sự cần thiết */
6     mm->pgd[pgd] = (uint64_t)alloc_pgtbl_level(mm);
7     if (mm->pgd[pgd] == 0) return NULL; // Lỗi hết RAM
8 }
9 uint64_t *p4d_tbl = (uint64_t *)mm->pgd[pgd];
10 // ... Tiếp tục đệ quy xuống các tầng PUD, PMD, PT
```



Hình 8: Minh họa cơ chế phân trang 5 cấp trên kiến trúc 64-bit

3.3.4 Ví dụ minh họa chi tiết quy trình dịch địa chỉ ảo (Address Walkthrough)

Để hiện thực hóa và chứng minh tính đúng đắn của mô hình phân trang 5 cấp trên kiến trúc 64-bit theo đúng yêu cầu kiểm thử trong đặc tả đề án, nhóm tiến hành phân tích kịch bản phân rã một địa chỉ ảo tuyến tính cụ thể dưới đây:

$$A = 0x39FFE1D9C9000$$

Dựa trên cấu hình phân chia trường bit hệ thống (trang có kích thước `PAGING64_PAGESZ = 4KB`, tương ứng với 12 bit độ dời), địa chỉ 64-bit này được bộ quản lý bộ nhớ (MMU) giả lập bóc tách thành các chỉ mục (*index*) để tra cứu cấu trúc cây phân tầng thông qua các phép toán logic bit-wise:

- Cấp độ 5 - Page Global Directory (PGD Index):** Chiếm giữ dải bit từ vị trí số 56 đến 48 (độ rộng 9 bit). Phép toán trích xuất chỉ mục:

$$\text{Index}_{\text{PGD}} = (0x39FFE1D9C9000 \gg 48) \& 0x1FF = 0x03$$

Ý nghĩa: Kết quả dẫn hướng hệ thống truy cập vào mục phân tầng số 0x03 trong mảng con trỏ gốc `mm_struct->pgd`.

- Cấp độ 4 - Page Level 4 Directory (P4D Index):** Chiếm giữ dải bit từ vị trí số 47 đến 39 (độ rộng 9 bit). Phép toán trích xuất chỉ mục:

$$\text{Index}_{\text{P4D}} = (0x39FFE1D9C9000 \gg 39) \& 0x1FF = 0x13F$$

Ý nghĩa: MMU sử dụng chỉ mục 0x13F để tìm kiếm vị trí bảng thư mục cấp 4 kế tiếp (`p4d_tbl`).

- Cấp độ 3 - Page Upper Directory (PUD Index):** Chiếm giữ dải bit từ vị trí số 38 đến 30 (độ rộng 9 bit). Phép toán trích xuất chỉ mục:

$$\text{Index}_{\text{PUD}} = (0x39FFE1D9C9000 \gg 30) \& 0x1FF = 0x1F8$$

Ý nghĩa: Định vị chính xác điểm vào thư mục cấp cao tại vị trí chỉ mục 0x1F8 của pud_tbl.

4. **Cấp độ 2 - Page Middle Directory (PMD Index):** Chiếm giữ dải bit từ vị trí số 29 đến 21 (độ rộng 9 bit). Phép toán trích xuất chỉ mục:

$$\text{Index}_{\text{PMD}} = (0x39FFE1D9C9000 \gg 21) \& 0x1FF = 0x0EC$$

Ý nghĩa: Điểm neo định tuyến đến vị trí 0x0EC trên bảng thư mục trung gian cấp 2 (pmd_tbl).

5. **Cấp độ 1 - Page Table (PT Index):** Chiếm giữ dải bit từ vị trí số 20 đến 12 (độ rộng 9 bit), cấu hình tương ứng với macro `PAGING64_ADDR_PT_MASK`. Phép toán trích xuất chỉ mục:

$$\text{Index}_{\text{PT}} = (0x39FFE1D9C9000 \gg 12) \& 0x1FF = 0x1C9$$

Ý nghĩa: Chỉ mục này trở thành vào phần tử thứ 0x1C9 của bảng trang mục tiêu (pt_tbl) để tiến hành đọc cấu trúc thông tin của mục bảng phân trang (PTE).

6. **Độ dời thực tế bên trong trang (Page Offset):** Nằm ở 12 bit cuối cùng (từ bit 11 đến bit 0). Phép toán cô lập độ dời:

$$\text{Offset} = 0x39FFE1D9C9000 \& 0xFFF = 0x000$$

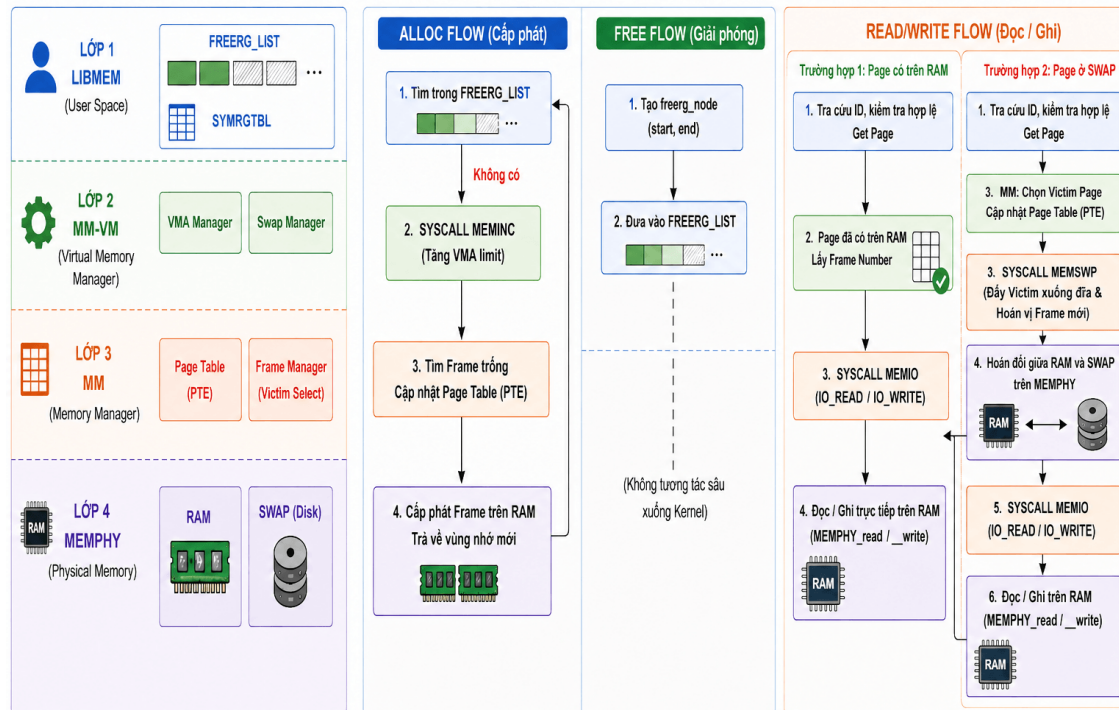
Phân Cấp	Dải Bit	Công Thức Logic	Chỉ Mục (Hex)
PGD (Cấp 5)	56 – 48	$(\text{addr} \gg 48) \& 0x1FF$	0x03
P4D (Cấp 4)	47 – 39	$(\text{addr} \gg 39) \& 0x1FF$	0x13F
PUD (Cấp 3)	38 – 30	$(\text{addr} \gg 30) \& 0x1FF$	0x1F8
PMD (Cấp 2)	29 – 21	$(\text{addr} \gg 21) \& 0x1FF$	0x0EC
PT (Cấp 1)	20 – 12	$(\text{addr} \gg 12) \& 0x1FF$	0x1C9
OFFSET	11 – 0	$\text{addr} \& 0xFFF$	0x000

Bảng 2: Bảng phân rã cấu trúc địa chỉ ảo 0x39FFE1D9C9000

Quy trình bẻ gãy địa chỉ ảo tuyến tính thành tập hợp chỉ mục có độ rộng cố định 9 bit chính là nền tảng cốt lõi giúp bộ nhớ MMU giả lập thực hiện các bước duyệt cây (*page table walk*) chính xác, hỗ trợ quản lý bộ nhớ thừa thớt hiệu quả trên không gian 64-bit.

3.4 Thư viện quản lý và giao tiếp bộ nhớ (libmem)

Đóng vai trò là lớp trung gian (*interface*) giữa ứng dụng người dùng và nhân hệ điều hành, `libmem.c` hiện thực hóa các API cấp phát ảo, dịch địa chỉ và xử lý lỗi trang. Quá trình giao tiếp này tuân thủ nguyên tắc cách ly không gian: mọi tương tác vật lý đều phải đi qua hệ thống `SYSCALL`. Đoạn mã này cung cấp một cái nhìn tổng quan về kiến trúc luồng dữ liệu.



Hình 9: Kiến trúc phân tầng và luồng tương tác tổng quan của hệ thống quản lý bộ nhớ

3.4.1 Cấp phát và thu hồi không gian người dùng (User-space)

Khi gọi `liballoc`, quá trình cấp phát không lập tức chiếm dụng RAM. Hàm `__alloc` ưu tiên tìm kiếm khoảng trống trong danh sách rảnh `vm_freerg_list`. Nếu thất bại, hệ thống gửi yêu cầu mở rộng giới hạn `brk` xuống Kernel thông qua `SYSCALL 17 (SYSMEM_INC_OP)`. Điểm đáng lưu ý theo đặc tả hệ thống là con trỏ tiến trình (PCB) không được truyền trực tiếp qua ranh giới User-Kernel mà Kernel phải tự nội suy dựa trên PID.

Tại hàm thu hồi `__free`, vùng nhớ không bị giải phóng khỏi RAM phần cứng mà chỉ được xóa khỏi danh bạ `symrgtbl` và đưa trở lại rổ `vm_freerg_list` để tái sử dụng. Đặc biệt, mọi thao tác thay đổi cấu trúc danh sách trong `libmem` đều được bảo vệ nghiêm ngặt bằng `pthread_mutex_lock(&mmvm_lock)` nhằm tránh hiện tượng tương tranh (*Race Condition*) trong môi trường đa luồng.

```

1  int __alloc(struct pcb_t *caller, int vmaid, int rgid, addr_t size,
2  addr_t *alloc_addr) {
3      // ...
4      pthread_mutex_lock(&mmvm_lock); /* Bảo vệ phân đoạn tới hạn */
5
6      if (get_free_vmrg_area(caller, vmaid, size, &rgnode) == 0) {
7          // ... Cập nhật symrgtbl
8          pthread_mutex_unlock(&mmvm_lock);
9          return 0;
10     }
11     // ... Thực hiện Syscall mở rộng limit
12     pthread_mutex_unlock(&mmvm_lock);
13 }

```

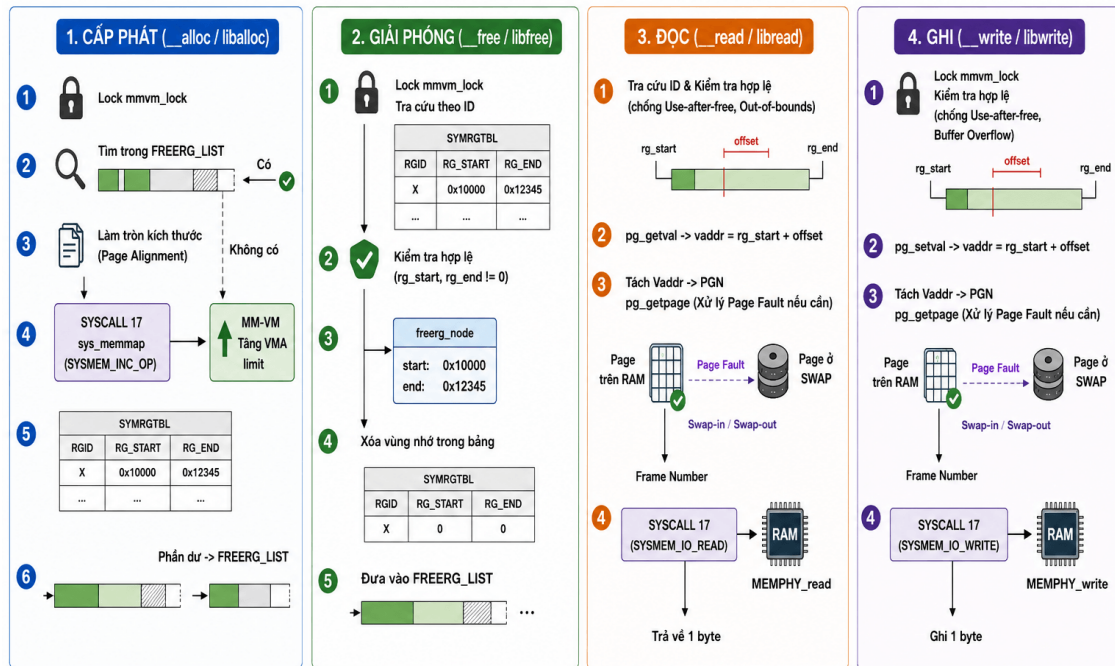
3.4.2 Xử lý lỗi trang (Page Fault) và cơ chế Swapping

Thao tác truy xuất bộ nhớ (`__read`, `__write`) được ủy thác cho `pg_getpage`. Hàm này kiểm tra bit *Present* của PTE; nếu trang chưa được nạp, một sự kiện lỗi trang (*Page Fault*) sẽ xảy ra và cơ chế hoán đổi (*Swapping*) được kích hoạt.

Nếu bộ nhớ RAM đầy, hệ thống áp dụng **thuật toán thay thế trang FIFO (First-In-First-Out)**. Tính đúng đắn của chu trình FIFO này được đảm bảo thông qua hai pha xử lý song song:

- **Pha nạp trang (Enqueue):** Mỗi khi một trang mới được nạp vào RAM, hàm `enlist_pgn_node` sẽ khởi tạo và chèn node của trang đó vào **đầu** danh sách liên kết `fifo_pgn` (thao tác Prepend với độ phức tạp $O(1)$). Nhờ cơ chế chèn vào đầu này, các trang được nạp càng sớm sẽ càng bị đẩy lùi dần về phía đuôi của danh sách.
- **Pha thu hồi (Dequeue):** Khi cần chọn "nạn nhân", hàm `find_victim_page` sẽ duyệt đến phần tử **cuối cùng** của danh sách. Vì các phần tử mới luôn được đẩy vào từ đầu, node ở cuối cùng chắc chắn là trang có mặt trong RAM lâu nhất. Trang nạn nhân này bị tước quyền, đẩy xuống đĩa qua `SYMEM_SWP_OP`, và nhường khung RAM cho trang mới.

```
1  /* 1. Pha nạp trang: Chèn node vào đầu danh sách (Prepend) */
2  int enlist_pgn_node(struct pgn_t **plist, addr_t pgn) {
3      struct pgn_t *pnode = malloc(sizeof(struct pgn_t));
4      pnode->pgn = pgn;
5
6      pnode->pg_next = *plist; /* Trỏ node mới vào head hiện tại */
7      *plist = pnode;         /* Cập nhật head bằng node mới */
8      return 0;
9  }
10
11 /* 2. Pha thu hồi: Lấy node ở cuối danh sách (Pop back) */
12 int find_victim_page(struct mm_struct *mm, addr_t *retpgn) {
13     struct pgn_t *pg = mm->fifo_pgn;
14     struct pgn_t *prev = NULL;
15
16     /* Duyệt đến node cuối cùng (node cũ nhất) */
17     while (pg->pg_next) {
18         prev = pg;
19         pg = pg->pg_next;
20     }
21     *retpgn = pg->pgn; /* Ghi nhận số hiệu trang nạn nhân */
22
23     /* Cắt node khỏi danh sách và giải phóng */
24     if (prev == NULL) mm->fifo_pgn = NULL;
25     else prev->pg_next = NULL;
26
27     free(pg);
28     return 0;
29 }
```



Hình 10: Lưu đồ chi tiết các thao tác Cấp phát, Thu hồi, Đọc và Ghi trong không gian người dùng

3.4.3 Quản lý bộ nhớ đặc quyền (Kernel-space Memory)

Trái ngược với User-space, bộ nhớ Kernel (`__kmalloc`) được cấp phát với quyền ưu tiên cao nhất. Quá trình này ánh xạ thẳng địa chỉ ảo vào khung RAM và thiết lập bit *Present* vĩnh viễn (*Pinned Pages*). Các trang của Kernel không bao giờ bị tham gia vào quá trình hoán đổi (Swap-out) nhằm đảm bảo tốc độ phản hồi $O(1)$ tuyệt đối, một yêu cầu bắt buộc khi xử lý Ngắt (*Interrupts*) hoặc *System Calls*. Bên cạnh đó, địa chỉ *vaddr* khởi tạo cho Kernel cũng được định tuyến để phù hợp với chuẩn *Canonical Address* của chế độ 64-bit.

Đặc biệt, đối với các cấu trúc dữ liệu nội bộ có kích thước cố định, hệ thống quản lý qua `libkmem_cache_alloc`. Mặc dù tên gọi mang hơi hướng *Slab Allocator*, hiện thực thực tế lại sử dụng **Bump Allocator** kết hợp **Memory Pool**.

```

1  addr_t __kmem_cache_alloc(...) {
2      // ...
3      /* Lấy địa chỉ đầu tiên đang trống */
4      addr_t slot_addr = pool->storage;
5
6      /* Tịnh tiến (Bump) con trỏ kho lên ô tiếp theo */
7      pool->storage += pool->align;
8      pool->size -= pool->align;
9      // ...
10 }
```

Cách tiếp cận "tịnh tiến con trỏ" này loại bỏ hoàn toàn chi phí tìm kiếm khoảng trống. Dù nhược điểm của Bump Allocator là không thể tái sử dụng bộ nhớ (*fragmentation* nếu có lệnh free), nhưng do các đối tượng cấu trúc OS (như bảng Pool) thường tồn tại suốt vòng đời hệ thống, sự đánh đổi này là hoàn toàn hợp lý để đổi lấy hiệu năng $O(1)$.

3.4.4 Giao tiếp dữ liệu liên không gian (Cross-space Communication)

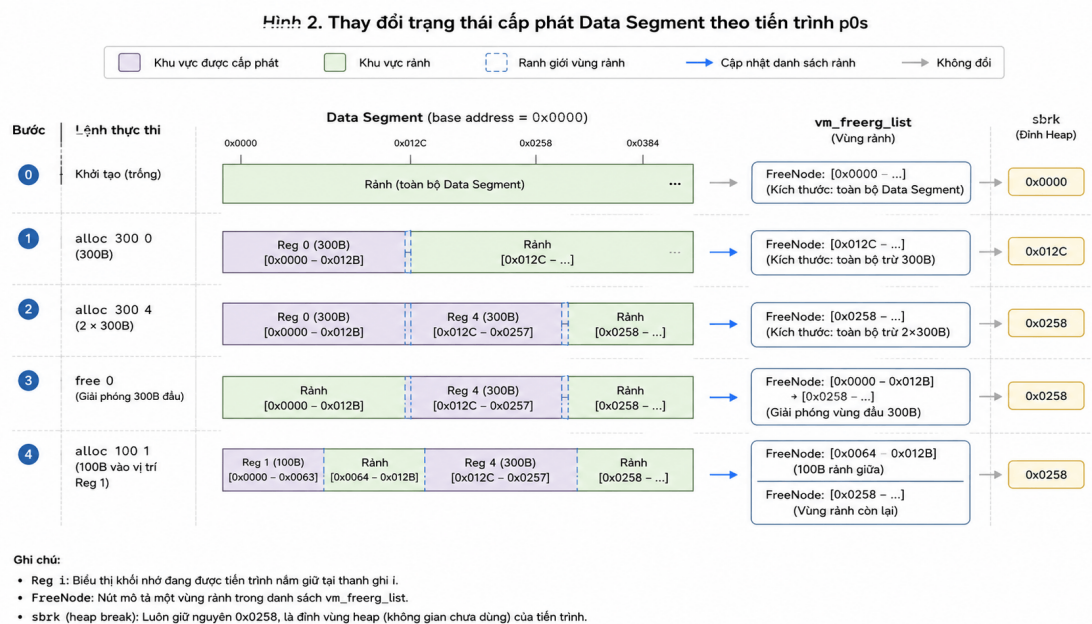
Do kiến trúc bảo vệ phân vạch (*dual-mode*), User bị cấm truy cập địa chỉ Kernel. Việc trao đổi dữ liệu phải qua `libkmem_copy_from_user` và `copy_to_user`.

Sự bất đối xứng về đặc quyền thể hiện rõ trong mã nguồn: Tại `__read_kernel_mem`, nhân OS tự tin dùng quyền tối cao để truy cập thẳng phần cứng qua hàm `MEMPHY_read` mà không cần xác thực. Ngược lại, khi OS phải ghi đè lên bộ nhớ người dùng thông qua `__write_user_mem`, nó vẫn phải tuân thủ việc gọi `pg_getpage` như một tiến trình bình thường để phòng trường hợp dữ liệu User đã bị đẩy xuống đĩa (*Swapped out*), đảm bảo không gây ra tình trạng đọc/ghi sai địa chỉ.

3.5 Minh họa trạng thái cấp phát vùng nhớ (Data Segment Allocation)

Để làm rõ cơ chế quản lý không gian địa chỉ ảo bên trong một tiến trình, nhóm trích xuất chuỗi lệnh thao tác bộ nhớ từ file cấu hình tiến trình `p0s` nhằm minh họa sự biến đổi trạng thái của các cấu trúc dữ liệu cốt lõi: bảng quản lý vùng nhớ đang dùng (`symrgtbl`), danh sách vùng rảnh (`vm_freerg_list`) và con trỏ đỉnh Heap (`sbrk`).

Giả sử địa chỉ cơ sở (*base address*) của vùng Data Segment bắt đầu từ `0x0000`.



Hình 11: Thay đổi trạng thái cấp phát Data Segment theo tiến trình p0s

Phân tích cơ chế tái sử dụng (Memory Reuse): Thông qua bảng trạng thái trên, hệ thống đã chứng minh được tính hiệu quả của giải thuật quản lý bộ nhớ cục bộ. Ở lệnh `alloc 100 1`, thay vì tiếp tục tịnh tiến con trỏ `sbrk` lên cao hơn gây lãng phí, hệ thống đã chủ động duyệt danh sách `vm_freerg_list`, tìm thấy khối nhớ rảnh 300 Bytes (từ lệnh `free 0` trước đó). Nó tiến hành "cắt" 100 Bytes để cấp phát cho `Reg 1`, và cập nhật lại khối rảnh còn 200 Bytes. Kết quả là đỉnh Heap (`sbrk`) vẫn giữ nguyên ở mức `0x0258`, triệt tiêu hiện tượng phân mảnh ngoại vi và tối đa hóa không gian khả dụng của tiến trình.

3.6 Cơ chế bảo mật và truy xuất bộ nhớ

Để đảm bảo tính toàn vẹn và ngăn chặn các hành vi truy xuất trái phép vào vùng nhớ Kernel, kiến trúc của hệ thống tuân thủ nghiêm ngặt mô hình tách biệt không gian người dùng (User Space) và không gian nhân (Kernel Space).

3.6.1 Cơ chế định danh tiến trình thông qua PID

Thay vì cho phép tiến trình người dùng truyền trực tiếp con trỏ cấu trúc điều khiển PCB (*Process Control Block*), hệ thống yêu cầu mọi thao tác bộ nhớ (như `sys_mmap`) phải thông qua một lớp giao diện (interface) bảo mật.

Tại đây, người dùng chỉ được phép cung cấp tham số định danh là *PID*. Việc này ngăn chặn hoàn toàn khả năng người dùng tự tạo hoặc giả mạo con trỏ PCB để thao tác vào bộ nhớ của tiến trình khác hoặc can thiệp vào các vùng nhớ đặc quyền.

3.6.2 Mã nguồn (Static Analysis)

Kernel đóng vai trò là "người gác cổng", tự thực hiện quá trình tra cứu PCB nội bộ bằng cách duyệt qua `running_list` thay vì tin tưởng vào con trỏ do người dùng cung cấp. Đoạn mã dưới đây minh chứng cho quy trình xác thực này:

Listing 1: Đoạn mã xác thực PCB tại `sys_mem.c`

```
int __sys_mmap(struct krnl_t *krnl, uint32_t pid, struct sc_regs* regs)
{
    struct pcb_t *caller = NULL;
    /* Kernel duyệt danh sách để tìm PCB thay vì nhận trực tiếp */
    struct queue_t *running_list = krnl->running_list;
    for (int i = 0; i < running_list->size; i++) {
        if (running_list->proc[i]->pid == pid) {
            caller = running_list->proc[i]; // PCB được xác định tại đây
            break;
        }
    }

    // Bảo mật: Nếu PID không hợp lệ (caller == NULL),
    // hệ thống từ chối truy xuất ngay lập tức
    if (caller == NULL) return -1;
    ...
}
```

3.6.3 Đánh giá thiết kế

Thông qua cách tiếp cận trên, hệ thống đạt được hai mục tiêu cốt lõi:

- **Tính cô lập (Isolation):** Người dùng không hề biết hoặc sở hữu địa chỉ bộ nhớ của PCB, họ chỉ tương tác thông qua mã định danh PID trừu tượng.
- **Tính kiểm soát (Access Control):** Kernel luôn giữ thế chủ động trong việc xác minh quyền truy cập. Nếu PID không khớp với bất kỳ tiến trình nào trong `running_list`, yêu cầu cấp phát bộ nhớ sẽ bị hủy bỏ, bảo vệ an toàn cho cấu trúc bảng trang và vùng nhớ toàn cục của hệ thống.

3.7 Cấp phát giả lập (Dummy Allocation) và Cơ chế Giao tiếp qua `vmap_pgd_memset`

Trong kiến trúc mô phỏng 64-bit với không gian địa chỉ ảo lên đến 128 Petabytes, việc kiểm thử khả năng định tuyến trên các dải địa chỉ rộng lớn mà không làm cạn kiệt RAM vật lý của hệ thống đòi hỏi một giải pháp cấp phát giả lập (*dummy allocation*). Hệ điều hành *Simple OS* giải quyết bài toán này thông qua hàm `vmap_pgd_memset`, kết hợp chặt chẽ với hạ tầng *System Call* nhằm thiết lập và cấu hình các bảng thư mục phân trang (*page-directory behavior*) một cách chủ động.

3.7.1 Luồng Điều phối và Giao tiếp Liên Ranh giới (System Call Dispatching)

Cơ chế cấp phát giả lập được kích hoạt từ không gian người dùng (*user-space*) hoặc các kịch bản kiểm thử thông qua lời gọi hệ thống quản lý bộ nhớ toàn cục `SYSCALL 17 (sys_mmap)`. Tiến trình giao tiếp dữ liệu cấu hình với hạt nhân thông qua cấu trúc thanh ghi `struct sc_regs` theo giao thức đóng gói nghiêm ngặt:

- **Thanh ghi `regs->a1 (SYSMEM_OP)`:** Lưu trữ mã hoạt động định danh tác vụ. Đối với tiến trình khởi tạo cấu trúc phân trang giả lập, thanh ghi này mang giá trị định hằng `SYSMEM_MAP_OP`.
- **Thanh ghi `regs->a2`:** Chứa địa chỉ ảo bắt đầu dải bộ nhớ cần thiết lập (`addr`), địa chỉ này bắt buộc phải được căn lề đồng bộ với kích thước trang vật lý (*page-aligned*).
- **Thanh ghi `regs->a3`:** Xác định tổng số lượng trang ảo cần thực hiện ánh xạ cấu trúc (`pgnum`).

Khi xảy ra bẫy hệ thống (*trap*), Kernel chuyển sang chế độ đặc quyền (*kernel-mode*), tra cứu bảng điều phối hệ thống (`syscall_tbl`) tại chỉ mục tương ứng để chuyển tiếp quyền thực thi đến hàm xử lý trung tâm trong `sys_mem.c`. Tại đây, Kernel giải nén các tham số từ các thanh ghi `a2` và `a3` để truyền trực tiếp vào hàm lõi `vmap_pgd_memset (caller, addr, pgnum)`.

3.7.2 Thuật toán Duyệt Cây Phân cấp và Khởi tạo Biến thể Giả lập

Khi hàm `vmap_pgd_memset` được thực thi, hệ thống vận hành một vòng lặp tuyến tính qua tổng số `pgnum` trang yêu cầu. Tại mỗi chu kỳ vòng lặp, các bước xử lý hạ tầng bộ nhớ được thực hiện tuần tự:

1. **Tính toán Số hiệu Trang (PGN):** Địa chỉ ảo hiện tại được dịch phải thông qua hằng số bit-shift hình học (`addr » PAGING64_ADDR_PT_SHIFT`) nhằm triệt tiêu phần độ dời (*offset*) 12-bit, trích xuất ra số hiệu trang ảo `pgn` chuẩn xác cho cấu trúc trang kích thước 4KB (`PAGING64_PAGESZ`).
2. **Kích hoạt Duyệt và Cấp phát Nhánh Thừa (*Sparse Allocation*):** Hệ thống gọi hàm hỗ trợ `get_pte_ptr (caller, pgn, 1)`. Với tham số cờ cấp phát `is_alloc = 1`, hàm thực hiện duyệt qua cấu trúc hình cây 5 cấp độ bảng phân trang (`PGD → P4D → PUD → PMD → PT`). Nếu bất kỳ bảng thư mục trung gian nào trên nhánh định tuyến chưa được khởi tạo (giá trị mục tương ứng bằng 0), hệ thống sẽ lập tức cấp phát động một mảng thư mục mới thông qua `calloc`, cho phép cấu trúc cây phân trang tự động cấu trúc hóa dựa trên nhu cầu thực tế (*on-demand allocation*).
3. **Ghi nhận Dấu vết Giả lập (Dummy Pattern Entry):** Sau khi định vị thành công con trỏ dẫn đến mục bảng trang ở tầng cuối cùng (PT), thay vì gọi hàm quản lý phần cứng `MEMPHY_get_freefp` để xin một khung RAM thực tế, hệ thống ghi trực tiếp giá trị mẫu nhận diện hệ thống `0xdeadbeef` vào mục bảng trang (PTE). Giá trị đặc định này đóng vai trò như một cờ hiệu xác thực trang ảo đã cấu trúc hóa thành công.

4. **Tình tiến Địa chỉ Biên:** Địa chỉ ảo `addr` được tình tiến lên một lượng bằng đúng kích thước trang hình học `PAGING64_PAGESZ` để chuẩn bị cho chu kỳ ánh xạ trang kế tiếp. Đây là giá trị dummy có chủ đích để test, không dùng trong production path, và hệ thống không thực sự cấp phát frame vật lý cho entry này.

Hình 12: Sơ đồ luồng điều phối từ Hệ thống lời gọi `SYSCALL 17` đến tiến trình khởi tạo cấu trúc trang giả lập

Giải pháp thiết lập cấu trúc giả lập thông qua `vmap_pgd_memset` là một minh chứng thực nghiệm quan trọng cho thấy hệ điều hành có khả năng cô lập và quản lý không gian địa chỉ thừa thớt cực kỳ lớn mà không bị ràng buộc bởi giới hạn dung lượng vật lý của chip RAM giả lập, tối ưu hóa không gian lưu trữ siêu dữ liệu một cách khoa học.

3.8 Các vấn đề cốt lõi trong Quản lý Bộ nhớ

3.8.1 Động lực kết hợp phân đoạn và phân trang

Động lực chính của việc kết hợp hai kỹ thuật này là tận dụng ưu điểm và bù đắp khuyết điểm của từng phương pháp.

Với *segmentation*, bộ nhớ được chia theo góc nhìn logic của lập trình viên như *code*, *data*, *heap* và *stack*. Cách tiếp cận này thuận lợi cho tổ chức chương trình và thiết lập cơ chế bảo vệ, nhưng vì các đoạn có kích thước không đều nên dễ dẫn đến hiện tượng phân mảnh ngoài (*external fragmentation*), gây lãng phí bộ nhớ và làm phức tạp thuật toán cấp phát.

Ngược lại, *paging* chia bộ nhớ thành các khối có kích thước cố định, gồm *pages* và *frames*, nhờ đó giải quyết hiệu quả vấn đề phân mảnh ngoài. Tuy nhiên, nếu chỉ sử dụng phân trang thuần túy thì hệ thống thiếu đi nhận thức về mặt logic của dữ liệu, gây khó khăn cho việc áp dụng các chính sách bảo vệ linh hoạt, chẳng hạn chia sẻ riêng vùng mã lệnh nhưng vẫn cô lập vùng dữ liệu. Ngoài ra, với không gian địa chỉ lớn, bảng trang tuyến tính cũng tiêu tốn một lượng bộ nhớ đáng kể.

Cơ chế lai (*hybrid approach*) giải quyết điểm yếu trên bằng cách chia bộ nhớ thành các đoạn logic để phục vụ tổ chức và bảo vệ, nhưng bên dưới mỗi đoạn lại tiếp tục được chia thành các trang có kích thước cố định để cấp phát vật lý. Nhờ đó, hệ điều hành vừa quản lý RAM hiệu quả mà không bị phân mảnh ngoài, vừa dễ dàng áp dụng quyền truy cập ở cấp độ logic.

3.8.2 Lợi ích của phân trang phân cấp nhiều mức

Việc mở rộng hệ thống bảng trang thành nhiều cấp (*N-level paging*) mang lại một lợi ích then chốt là tiết kiệm bộ nhớ cho chính bảng trang thông qua cơ chế cấp phát thưa (*sparse allocation*).

Đối với một không gian địa chỉ 64-bit, nếu sử dụng bảng trang một cấp tuyến tính, hệ điều hành sẽ phải cấp phát trước một khối lượng bộ nhớ vật lý rất lớn chỉ để lưu trữ các mục bảng trang (*Page Table Entries*), bất kể tiến trình có thực sự sử dụng toàn bộ không gian đó hay không.

Với thiết kế nhiều cấp, bảng trang được tổ chức theo cấu trúc cây. Các nhánh của cây, tương ứng với các bảng trang ở cấp thấp hơn, chỉ được cấp phát khi tiến trình thực sự truy cập đến vùng nhớ ảo tương ứng. Những vùng không gian ảo chưa được sử dụng sẽ có các con trỏ ở cấp cao trỏ về NULL, qua đó tiết kiệm đáng kể bộ nhớ vật lý dành cho hệ điều hành và đồng thời tạo điều kiện thuận lợi cho việc hoán đổi (*swap*) từng phần của bảng trang ra thiết bị lưu trữ khi cần thiết.

3.8.3 Ưu điểm và nhược điểm của cấp phát bộ nhớ phân trang (paging) và cấp phát bộ nhớ liên tục (contiguous memory allocation)

Trong hệ điều hành, hai kỹ thuật quản lý bộ nhớ cơ bản là cấp phát liên tục và cấp phát phân trang có những đặc điểm kiến trúc cũng như ưu/nhược điểm hoàn toàn trái ngược nhau. Việc lựa chọn kỹ thuật nào phụ thuộc vào thiết kế phần cứng và yêu cầu của hệ thống.

1. Cấp phát bộ nhớ liên tục (Contiguous Memory Allocation)

Trong phương pháp này, khi một tiến trình được nạp vào bộ nhớ vật lý (RAM), nó sẽ được cấp phát một vùng nhớ hoặc một khối bộ nhớ liền kề nhau.

Ưu điểm:

- **Đơn giản trong cài đặt và quản lý:** Hệ điều hành chỉ cần theo dõi các khối nhớ trống và khối nhớ đã cấp phát thông qua các thuật toán cơ bản như *First-fit*, *Best-fit*, hoặc *Worst-fit*.
- **Tốc độ truy xuất cực nhanh:** Vì các không gian địa chỉ nằm liền kề nhau, hệ thống không tốn chi phí dịch địa chỉ phức tạp. Việc đọc/ghi dữ liệu liên tục được tối ưu hóa tối đa nhờ tận dụng tốt tính cục bộ không gian (*spatial locality*) của bộ nhớ đệm (*Cache*).
- **Chi phí phần cứng thấp:** Hệ thống bảo vệ bộ nhớ chỉ yêu cầu hai thanh ghi đơn giản là thanh ghi cơ sở (*Base register*) và thanh ghi giới hạn (*Limit register*).

Nhược điểm:

- **Phân mảnh ngoại (External Fragmentation):** Đây là nhược điểm chí mạng của phương pháp này. Quá trình cấp phát và giải phóng vùng nhớ liên tục tạo ra các "lỗ hổng" nhỏ nằm rải rác. Tổng dung lượng trống có thể lớn hơn nhu cầu của tiến trình mới, nhưng vì chúng không liền kề nhau nên hệ thống vẫn từ chối cấp phát.
- **Chi phí mở rộng cao:** Nếu tiến trình đang chạy yêu cầu thêm bộ nhớ nhưng phần không gian liền kề đã bị chiếm, hệ điều hành buộc phải thực hiện thao tác tốn kém: tìm một vùng nhớ mới lớn hơn và sao chép toàn bộ dữ liệu của tiến trình sang đó.

2. Cấp phát phân trang (Paging)

Phương pháp này chia bộ nhớ vật lý (RAM) thành các khối có kích thước cố định gọi là Khung trang (*Frames*), và chia không gian địa chỉ ảo (*Logical memory*) của tiến trình thành các Trang (*Pages*) có cùng kích thước. Một tiến trình có thể được nạp vào các khung trang nằm rải rác bất kỳ đâu trên RAM.

Ưu điểm:

- **Giải quyết triệt để phân mảnh ngoại:** Các trang của tiến trình có thể được nhét vừa vặn vào bất kỳ khung trang trống nào. Do đó, hệ thống không bao giờ bị lãng phí bộ nhớ do phân mảnh ngoại.
- **Linh hoạt trong cấp phát:** Việc tìm vùng nhớ trống trở nên cực kỳ dễ dàng.
- **Hỗ trợ Bộ nhớ ảo (Virtual Memory) hoàn hảo:** Kỹ thuật phân trang là nền tảng cốt lõi cho cơ chế nạp trang theo yêu cầu (*Demand Paging*), cho phép chạy các tiến trình có kích thước lớn hơn dung lượng RAM thực tế.

Nhược điểm:

- **Phân mảnh nội (Internal Fragmentation):** Nếu kích thước tiến trình không phải là bội số của kích thước trang, trang cuối cùng sẽ bị dư thừa một phần không gian không được sử dụng, gây ra lãng phí cục bộ.



- **Chi phí lưu trữ lớn (Memory Overhead):** Hệ điều hành phải duy trì các Bảng phân trang (*Page Tables*) cho mỗi tiến trình để ánh xạ địa chỉ ảo sang vật lý. Trong hệ thống 64-bit, nếu không dùng các kỹ thuật tối ưu như phân trang đa cấp (*Multi-level Paging*) hay cấp phát động (*Demand Allocation*), bảng trang có thể chiếm cạn kiệt bộ nhớ.
- **Giảm tốc độ truy xuất (Performance Penalty):** Để đọc một lệnh hoặc dữ liệu, *CPU* phải truy xuất bộ nhớ ít nhất 2 lần (truy cập Bảng trang để dịch địa chỉ, sau đó truy cập dữ liệu thực). Nhược điểm này bắt buộc phần cứng phải có thêm bộ đệm *TLB* (*Translation Lookaside Buffer*) để bù đắp độ trễ.

3.9 Phân tích kết quả kiểm thử

3.9.1 Testcase os_0_mlq_paging

Input:

```
1 6 2 4
2 268435456 16777216 0 0 0
3 0 p0s 0
4 1 pls 15
5 3 pls 0
6 6 p0s 0
7 5 pls 0
```

Output:

```
1 Time slot 0
2 ld_routine
3     Loaded a process at input/proc/p0s, PID: 1 PRIO: 0
4 Time slot 1
5     CPU 1: Dispatched process 1
6     Loaded a process at input/proc/pls, PID: 2 PRIO: 15
7 Time slot 2
8     CPU 0: Dispatched process 2
9 liballoc:181
10 print_pgtbl:
11 PDG=0x7acad009c50 P4g=0x7acad4000bc0 PUD=0x7acad4001bd0 PMD=0x7acad4002be0
12   ↳ PT=0x7acad4003bf0
13 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
14   ↳ storage_bytes=20480
15 Time slot 3
16     Loaded a process at input/proc/pls, PID: 3 PRIO: 0
17 liballoc:181
18 print_pgtbl:
19 PDG=0x7acad009c50 P4g=0x7acad4000bc0 PUD=0x7acad4001bd0 PMD=0x7acad4002be0
20   ↳ PT=0x7acad4003bf0
21 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
22   ↳ storage_bytes=20480
23 Time slot 4
24 libfree:205
25 print_pgtbl:
26 PDG=0x7acad009c50 P4g=0x7acad4000bc0 PUD=0x7acad4001bd0 PMD=0x7acad4002be0
27   ↳ PT=0x7acad4003bf0
28 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
29   ↳ storage_bytes=20480
30 Time slot 5
31 liballoc:181
32 print_pgtbl:
```



```
27 PDG=0x7acadc009c50 P4g=0x7acad4000bc0 PUD=0x7acad4001bd0 PMD=0x7acad4002be0
   ↳ PT=0x7acad4003bf0
28 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
29 Time slot 6
   Loaded a process at input/proc/p0s, PID: 4 PRIO: 0
30 libwrite:470
31 print_pgtbl:
32 PDG=0x7acadc009c50 P4g=0x7acad4000bc0 PUD=0x7acad4001bd0 PMD=0x7acad4002be0
   ↳ PT=0x7acad4003bf0
33 pgtbl_stats: walks=3 level_reads=15 level_writes=5 total_accesses=20
   ↳ tables=5 storage_bytes=20480
34 Time slot 7
   CPU 1: Put process 1 to run queue
35 CPU 1: Dispatched process 3
36 Time slot 8
   CPU 0: Put process 2 to run queue
37 CPU 0: Dispatched process 4
38 Time slot 9
   liballoc:181
39 print_pgtbl:
40 PDG=0x7acadc014f70 P4g=0x7acad8000bc0 PUD=0x7acad8001bd0 PMD=0x7acad8002be0
   ↳ PT=0x7acad8003bf0
41 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
42 Time slot 10
   liballoc:181
43 print_pgtbl:
44 PDG=0x7acadc014f70 P4g=0x7acad8000bc0 PUD=0x7acad8001bd0 PMD=0x7acad8002be0
   ↳ PT=0x7acad8003bf0
45 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
46 Time slot 11
   libfree:205
47 print_pgtbl:
48 PDG=0x7acadc014f70 P4g=0x7acad8000bc0 PUD=0x7acad8001bd0 PMD=0x7acad8002be0
   ↳ PT=0x7acad8003bf0
49 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
50 Time slot 12
   liballoc:181
51 print_pgtbl:
52 PDG=0x7acadc014f70 P4g=0x7acad8000bc0 PUD=0x7acad8001bd0 PMD=0x7acad8002be0
   ↳ PT=0x7acad8003bf0
53 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
54 Time slot 13
   CPU 1: Put process 3 to run queue
55 CPU 1: Dispatched process 1
56 libread:410
57 print_pgtbl:
58 PDG=0x7acadc009c50 P4g=0x7acad4000bc0 PUD=0x7acad4001bd0 PMD=0x7acad4002be0
   ↳ PT=0x7acad4003bf0
59 pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
   ↳ tables=5 storage_bytes=20480
60 libwrite:470
61 print_pgtbl:
62 PDG=0x7acadc014f70 P4g=0x7acad8000bc0 PUD=0x7acad8001bd0 PMD=0x7acad8002be0
   ↳ PT=0x7acad8003bf0
```

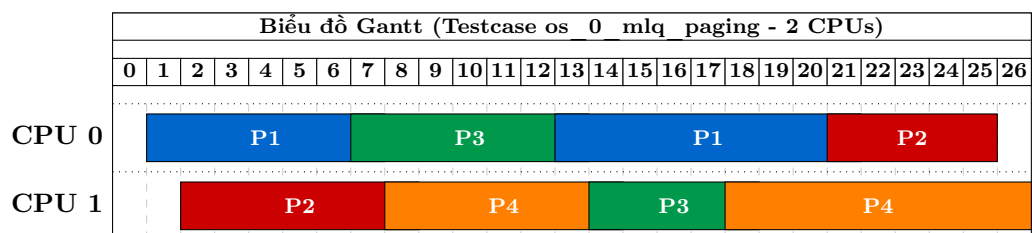


```
71  pgtbl_stats: walks=3 level_reads=15 level_writes=5 total_accesses=20
    ↳ tables=5 storage_bytes=20480
72  Time slot 14
73      CPU 0: Put process 4 to run queue
74      CPU 0: Dispatched process 3
75  Time slot 15
76  print_pgtbl:
77      PDG=0x7acadc009c50 P4g=0x7acad4000bc0 PUD=0x7acad4001bd0 PMD=0x7acad4002be0
    ↳ PT=0x7acad4003bf0
78  pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
    ↳ tables=5 storage_bytes=20480
79  Time slot 16
80  Time slot 17
81  print_pgtbl:
82      PDG=0x7acadc009c50 P4g=0x7acad4000bc0 PUD=0x7acad4001bd0 PMD=0x7acad4002be0
    ↳ PT=0x7acad4003bf0
83  pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
    ↳ tables=5 storage_bytes=20480
84  Time slot 18
85      CPU 0: Processed 3 has finished
86      CPU 0: Dispatched process 4
87  libread:410
88  print_pgtbl:
89      PDG=0x7acadc014f70 P4g=0x7acad8000bc0 PUD=0x7acad8001bd0 PMD=0x7acad8002be0
    ↳ PT=0x7acad8003bf0
90  pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
    ↳ tables=5 storage_bytes=20480
91  Time slot 19
92      CPU 1: Put process 1 to run queue
93      CPU 1: Dispatched process 1
94  libfree:205
95  print_pgtbl:
96      PDG=0x7acadc009c50 P4g=0x7acad4000bc0 PUD=0x7acad4001bd0 PMD=0x7acad4002be0
    ↳ PT=0x7acad4003bf0
97  pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
    ↳ tables=5 storage_bytes=20480
98  Time slot 20
99  print_pgtbl:
100     PDG=0x7acadc014f70 P4g=0x7acad8000bc0 PUD=0x7acad8001bd0 PMD=0x7acad8002be0
    ↳ PT=0x7acad8003bf0
101  pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
    ↳ tables=5 storage_bytes=20480
102  Time slot 21
103     CPU 1: Processed 1 has finished
104     CPU 1: Dispatched process 2
105  Time slot 22
106  print_pgtbl:
107     PDG=0x7acadc014f70 P4g=0x7acad8000bc0 PUD=0x7acad8001bd0 PMD=0x7acad8002be0
    ↳ PT=0x7acad8003bf0
108  pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
    ↳ tables=5 storage_bytes=20480
109  Time slot 23
110  Time slot 24
111     CPU 0: Put process 4 to run queue
112     CPU 0: Dispatched process 4
113  libfree:205
114  print_pgtbl:
115     PDG=0x7acadc014f70 P4g=0x7acad8000bc0 PUD=0x7acad8001bd0 PMD=0x7acad8002be0
    ↳ PT=0x7acad8003bf0
```



```
116 | pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
    | ↳ tables=5 storage_bytes=20480
117 | Time slot 25
    |     CPU 1: Processed 2 has finished
118 |     CPU 1 stopped
119 |
120 | Time slot 26
    |     CPU 0: Processed 4 has finished
121 |     CPU 0 stopped
122 |
```

Biểu đồ Gantt:



Hình 13: Phân lịch MLQ trên 2 CPUs (os_0_mlq_paging).



Thông kê chi phí quản lý bộ nhớ (Đại diện: tiến trình P1):

- Số lần truy cập bộ nhớ của tiến trình (`total_accesses`): 30 lần.
- Kích thước cây phân trang cấp phát cho tiến trình (*Page Table Overhead*): 20 KB.

So sánh với mẫu: (output/os_0_mlq_paging.output) Testcase đạt yêu cầu. Logic thực thi trùng khớp với output mẫu về trình tự nạp tiến trình và các thao tác bộ nhớ (alloc, free, read, write).

3.9.2 Testcase os_1_mlq_paging

Input:

```
1 2 4 8
2 268435456 16777216 0 0 0
3 1 pls 15
4 5 m0s 120
5 7 p0s 130
6 9 s0 38
7 11 s3 39
8 14 m1s 15
9 16 s2 120
10 16 s1 0
11
```

Output:

```
1 Time slot 0
2 ld_routine
3     Loaded a process at input/proc/pls, PID: 1 PRIO: 15
4 Time slot 1
5     CPU 1: Dispatched process 1
6 Time slot 2
7 Time slot 3
8     CPU 1: Put process 1 to run queue
9     CPU 1: Dispatched process 1
10 Time slot 4
11     Loaded a process at input/proc/m0s, PID: 2 PRIO: 120
12     CPU 3: Dispatched process 2
13 Time slot 5
14 liballoc:181
15 print_pgtbl:
16 PDG=0x7efda800d690 P4g=0x7efda0000bc0 PUD=0x7efda0001bd0 PMD=0x7efda0002be0
17   ↪ PT=0x7efda0003bf0
18 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
19   ↪ storage_bytes=20480
20     CPU 1: Put process 1 to run queue
21     CPU 1: Dispatched process 1
22 liballoc:181
23 print_pgtbl:
24 PDG=0x7efda800d690 P4g=0x7efda0000bc0 PUD=0x7efda0001bd0 PMD=0x7efda0002be0
25   ↪ PT=0x7efda0003bf0
26 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
27   ↪ storage_bytes=20480
28 Time slot 6
29     CPU 3: Put process 2 to run queue
30     CPU 3: Dispatched process 2
31 libfree:205
32 print_pgtbl:
```



```
29 PDG=0x7efda800d690 P4g=0x7efda0000bc0 PUD=0x7efda0001bd0 PMD=0x7efda0002be0
   ↳ PT=0x7efda0003bf0
30 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
31     Loaded a process at input/proc/p0s, PID: 3 PRIO: 130
32 Time slot 7
33     CPU 1: Put process 1 to run queue
34     CPU 1: Dispatched process 1
35     CPU 0: Dispatched process 3
36 liballoc:181
37 print_ptbl:
38 PDG=0x7efda800d690 P4g=0x7efda0000bc0 PUD=0x7efda0001bd0 PMD=0x7efda0002be0
   ↳ PT=0x7efda0003bf0
39 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
40 Time slot 8
41 liballoc:181
42 print_ptbl:
43 PDG=0x7efda80112b0 P4g=0x7efda4000bc0 PUD=0x7efda4001bd0 PMD=0x7efda4002be0
   ↳ PT=0x7efda4003bf0
44 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
45     Loaded a process at input/proc/s0, PID: 4 PRIO: 38
46 Time slot 9
47     CPU 3: Put process 2 to run queue
48     CPU 3: Dispatched process 4
49     CPU 1: Put process 1 to run queue
50     CPU 1: Dispatched process 1
51     CPU 0: Put process 3 to run queue
52     CPU 0: Dispatched process 2
53 libwrite:470
54 print_ptbl:
55 PDG=0x7efda800d690 P4g=0x7efda0000bc0 PUD=0x7efda0001bd0 PMD=0x7efda0002be0
   ↳ PT=0x7efda0003bf0
56 pgtbl_stats: walks=3 level_reads=15 level_writes=5 total_accesses=20
   ↳ tables=5 storage_bytes=20480
57     CPU 2: Dispatched process 3
58 liballoc:181
59 print_ptbl:
60 PDG=0x7efda80112b0 P4g=0x7efda4000bc0 PUD=0x7efda4001bd0 PMD=0x7efda4002be0
   ↳ PT=0x7efda4003bf0
61 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
62 Time slot 10
63 libfree:205
64 print_ptbl:
65 PDG=0x7efda80112b0 P4g=0x7efda4000bc0 PUD=0x7efda4001bd0 PMD=0x7efda4002be0
   ↳ PT=0x7efda4003bf0
66 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
67     Loaded a process at input/proc/s3, PID: 5 PRIO: 39
68     CPU 3: Put process 4 to run queue
69     CPU 3: Dispatched process 4
70 Time slot 11
71     CPU 2: Put process 3 to run queue
72     CPU 2: Dispatched process 5
73     CPU 0: Processed 2 has finished
74     CPU 0: Dispatched process 3
75 liballoc:181
```



```
76 print_pgtbl:
77   PDG=0x7efda80112b0 P4g=0x7efda4000bc0 PUD=0x7efda4001bd0 PMD=0x7efda4002be0
   ↪ PT=0x7efda4003bf0
78   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
79   CPU 1: Processed 1 has finished
80 Time slot 12
81 libwrite:470
82 print_pgtbl:
83   PDG=0x7efda80112b0 P4g=0x7efda4000bc0 PUD=0x7efda4001bd0 PMD=0x7efda4002be0
   ↪ PT=0x7efda4003bf0
84   pgtbl_stats: walks=3 level_reads=15 level_writes=5 total_accesses=20
   ↪ tables=5 storage_bytes=20480
85   CPU 3: Put process 4 to run queue
86   CPU 3: Dispatched process 4
87 Time slot 13
88   CPU 2: Put process 5 to run queue
89   CPU 2: Dispatched process 5
90   CPU 0: Put process 3 to run queue
91   CPU 0: Dispatched process 3
92 libread:410
93 print_pgtbl:
94   PDG=0x7efda80112b0 P4g=0x7efda4000bc0 PUD=0x7efda4001bd0 PMD=0x7efda4002be0
   ↪ PT=0x7efda4003bf0
95   pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
   ↪ tables=5 storage_bytes=20480
96   Loaded a process at input/proc/mls, PID: 6 PRIO: 15
97 Time slot 14
98   CPU 1: Dispatched process 6
99 liballoc:181
100 print_pgtbl:
101   PDG=0x7efda801c570 P4g=0x7efd9c000bc0 PUD=0x7efd9c001bd0 PMD=0x7efd9c002be0
   ↪ PT=0x7efd9c003bf0
102   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
103   CPU 3: Put process 4 to run queue
104   CPU 3: Dispatched process 4
105   CPU 2: Put process 5 to run queue
106   CPU 2: Dispatched process 5
107 Time slot 15
108   CPU 0: Put process 3 to run queue
109   CPU 0: Dispatched process 3
110 print_pgtbl:
111   PDG=0x7efda80112b0 P4g=0x7efda4000bc0 PUD=0x7efda4001bd0 PMD=0x7efda4002be0
   ↪ PT=0x7efda4003bf0
112   pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
   ↪ tables=5 storage_bytes=20480
113 liballoc:181
114 print_pgtbl:
115   PDG=0x7efda801c570 P4g=0x7efd9c000bc0 PUD=0x7efd9c001bd0 PMD=0x7efd9c002be0
   ↪ PT=0x7efd9c003bf0
116   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
117   Loaded a process at input/proc/s2, PID: 7 PRIO: 120
118 Time slot 16
119   CPU 1: Put process 6 to run queue
120   CPU 1: Dispatched process 6
121 libfree:205
122 print_pgtbl:
```

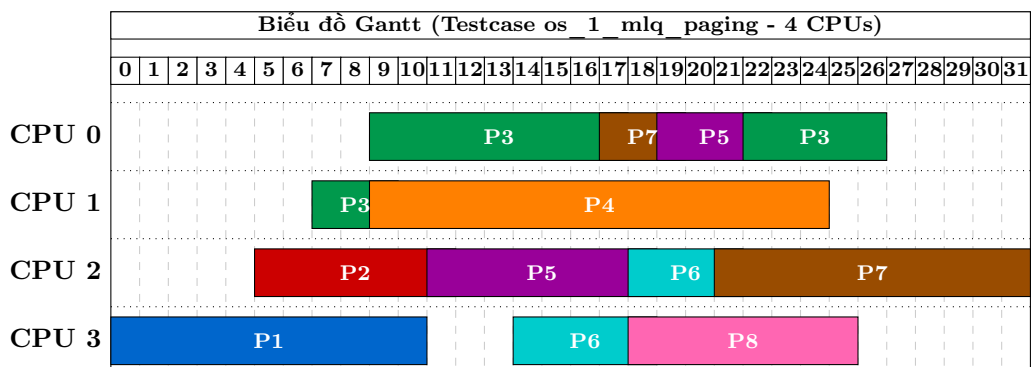


```
123 PDG=0x7efda801c570 P4g=0x7efd9c000bc0 PUD=0x7efd9c001bd0 PMD=0x7efd9c002be0
    ↳ PT=0x7efd9c003bf0
124 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
125     CPU 3: Put process 4 to run queue
126     CPU 3: Dispatched process 4
127     CPU 2: Put process 5 to run queue
128     CPU 2: Dispatched process 5
129 Time slot 17
130 liballoc:181
131 print_pgtbl:
132 PDG=0x7efda801c570 P4g=0x7efd9c000bc0 PUD=0x7efd9c001bd0 PMD=0x7efd9c002be0
    ↳ PT=0x7efd9c003bf0
133 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
134     CPU 0: Put process 3 to run queue
135     CPU 0: Dispatched process 7
136     Loaded a process at input/proc/s1, PID: 8 PRIO: 0
137 Time slot 18
138     CPU 1: Put process 6 to run queue
139     CPU 1: Dispatched process 8
140     CPU 3: Put process 4 to run queue
141     CPU 3: Dispatched process 6
142 libfree:205
143 print_pgtbl:
144 PDG=0x7efda801c570 P4g=0x7efd9c000bc0 PUD=0x7efd9c001bd0 PMD=0x7efd9c002be0
    ↳ PT=0x7efd9c003bf0
145 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
146 Time slot 19
147     CPU 0: Put process 7 to run queue
148     CPU 0: Dispatched process 4
149     CPU 2: Put process 5 to run queue
150     CPU 2: Dispatched process 5
151 Time slot 20
152     CPU 1: Put process 8 to run queue
153     CPU 1: Dispatched process 8
154 libfree:205
155 print_pgtbl:
156 PDG=0x7efda801c570 P4g=0x7efd9c000bc0 PUD=0x7efd9c001bd0 PMD=0x7efd9c002be0
    ↳ PT=0x7efd9c003bf0
157 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
158     CPU 2: Put process 5 to run queue
159     CPU 2: Dispatched process 5
160 Time slot 21
161     CPU 3: Processed 6 has finished
162     CPU 3: Dispatched process 7
163     CPU 0: Put process 4 to run queue
164     CPU 0: Dispatched process 4
165 Time slot 22
166     CPU 2: Processed 5 has finished
167     CPU 2: Dispatched process 3
168 print_pgtbl:
169 PDG=0x7efda80112b0 P4g=0x7efda4000bc0 PUD=0x7efda4001bd0 PMD=0x7efda4002be0
    ↳ PT=0x7efda4003bf0
170 pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
    ↳ tables=5 storage_bytes=20480
171     CPU 1: Put process 8 to run queue
```



```
172         CPU 1: Dispatched process 8
173         CPU 3: Put process 7 to run queue
174         CPU 3: Dispatched process 7
175 Time slot 23
176         CPU 0: Put process 4 to run queue
177         CPU 0: Dispatched process 4
178 Time slot 24
179         CPU 2: Put process 3 to run queue
180         CPU 2: Dispatched process 3
181 libfree:205
182 print_pgtbl:
183 PDG=0x7efda80112b0 P4g=0x7efda4000bc0 PUD=0x7efda4001bd0 PMD=0x7efda4002be0
184   ↳ PT=0x7efda4003bf0
185 pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
186   ↳ tables=5 storage_bytes=20480
187         CPU 1: Put process 8 to run queue
188         CPU 1: Dispatched process 8
189         CPU 0: Processed 4 has finished
190         CPU 0 stopped
191 Time slot 25
192         CPU 1: Processed 8 has finished
193         CPU 1 stopped
194         CPU 3: Put process 7 to run queue
195         CPU 3: Dispatched process 7
196 Time slot 26
197         CPU 2: Processed 3 has finished
198         CPU 2 stopped
199 Time slot 27
200         CPU 3: Put process 7 to run queue
201         CPU 3: Dispatched process 7
202 Time slot 28
203 Time slot 29
204         CPU 3: Put process 7 to run queue
205         CPU 3: Dispatched process 7
206 Time slot 30
207 Time slot 31
208         CPU 3: Processed 7 has finished
209         CPU 3 stopped
```

Biểu đồ Gantt:



Hình 14: Phân lịch MLQ trên 4 CPUs (os_1_mlq_paging).



Thông kê chi phí quản lý bộ nhớ (Đại diện: tiến trình P3):

- Số lần truy cập bộ nhớ của tiến trình (`total_accesses`): 30 lần.
- Kích thước cây phân trang cấp phát cho tiến trình (*Page Table Overhead*): 20 KB.

So sánh với mẫu: (output/os_1_mlq_paging.output) Testcase đạt yêu cầu. Logic thực thi trùng khớp với output mẫu về trình tự nạp tiến trình và các thao tác bộ nhớ (alloc, free, read, write).

3.9.3 Testcase os_1_singleCPU_mlq

Input:

```
1 2 1 8
2 0 pls 2
3 1 s4 4
4 3 s1 0
5 4 m0s 3
6 6 s0 1
7 7 s3 3
8 9 m1s 2
9 10 s2 3
10
```

Output:

```
1 Time slot 0
2 ld_routine
3     Loaded a process at input/proc/pls, PID: 1 PRIO: 2
4 Time slot 1
5     CPU 0: Dispatched process 1
6     Loaded a process at input/proc/s4, PID: 2 PRIO: 4
7 Time slot 2
8 Time slot 3
9     Loaded a process at input/proc/s1, PID: 3 PRIO: 0
10    CPU 0: Put process 1 to run queue
11    CPU 0: Dispatched process 3
12 Time slot 4
13    Loaded a process at input/proc/m0s, PID: 4 PRIO: 3
14 Time slot 5
15    CPU 0: Put process 3 to run queue
16    CPU 0: Dispatched process 3
17 Time slot 6
18    Loaded a process at input/proc/s0, PID: 5 PRIO: 1
19 Time slot 7
20    CPU 0: Put process 3 to run queue
21    CPU 0: Dispatched process 3
22    Loaded a process at input/proc/s3, PID: 6 PRIO: 3
23 Time slot 8
24 Time slot 9
25    Loaded a process at input/proc/m1s, PID: 7 PRIO: 2
26    CPU 0: Put process 3 to run queue
27    CPU 0: Dispatched process 3
28 Time slot 10
29    CPU 0: Processed 3 has finished
30    CPU 0: Dispatched process 5
31    Loaded a process at input/proc/s2, PID: 8 PRIO: 3
32 Time slot 11
33 Time slot 12
34    CPU 0: Put process 5 to run queue
```



```
35         CPU 0: Dispatched process  5
36 Time slot 13
37 Time slot 14
38         CPU 0: Put process  5 to run queue
39         CPU 0: Dispatched process  5
40 Time slot 15
41 Time slot 16
42         CPU 0: Put process  5 to run queue
43         CPU 0: Dispatched process  5
44 Time slot 17
45 Time slot 18
46         CPU 0: Put process  5 to run queue
47         CPU 0: Dispatched process  5
48 Time slot 19
49 Time slot 20
50         CPU 0: Put process  5 to run queue
51         CPU 0: Dispatched process  5
52 Time slot 21
53 Time slot 22
54         CPU 0: Put process  5 to run queue
55         CPU 0: Dispatched process  5
56 Time slot 23
57 Time slot 24
58         CPU 0: Put process  5 to run queue
59         CPU 0: Dispatched process  5
60 Time slot 25
61         CPU 0: Processed  5 has finished
62         CPU 0: Dispatched process  1
63 Time slot 26
64 Time slot 27
65         CPU 0: Put process  1 to run queue
66         CPU 0: Dispatched process  7
67 liballoc:181
68 print_pgtbl:
69   PDG=0x7d8c2c01ff50 P4g=0x7d8c24000bc0 PUD=0x7d8c24001bd0 PMD=0x7d8c24002be0
   ↪ PT=0x7d8c24003bf0
70   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
71 Time slot 28
72 liballoc:181
73 print_pgtbl:
74   PDG=0x7d8c2c01ff50 P4g=0x7d8c24000bc0 PUD=0x7d8c24001bd0 PMD=0x7d8c24002be0
   ↪ PT=0x7d8c24003bf0
75   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
76 Time slot 29
77         CPU 0: Put process  7 to run queue
78         CPU 0: Dispatched process  1
79 Time slot 30
80 Time slot 31
81         CPU 0: Put process  1 to run queue
82         CPU 0: Dispatched process  7
83 libfree:205
84 print_pgtbl:
85   PDG=0x7d8c2c01ff50 P4g=0x7d8c24000bc0 PUD=0x7d8c24001bd0 PMD=0x7d8c24002be0
   ↪ PT=0x7d8c24003bf0
86   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
87 Time slot 32
88 liballoc:181
```



```
89 print_pgtbl:
90   PDG=0x7d8c2c01ff50 P4g=0x7d8c24000bc0 PUD=0x7d8c24001bd0 PMD=0x7d8c24002be0
   ↪ PT=0x7d8c24003bf0
91   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
92   Time slot 33
93       CPU 0: Put process 7 to run queue
94       CPU 0: Dispatched process 1
95   Time slot 34
96   Time slot 35
97       CPU 0: Put process 1 to run queue
98       CPU 0: Dispatched process 7
99   libfree:205
100  print_pgtbl:
101   PDG=0x7d8c2c01ff50 P4g=0x7d8c24000bc0 PUD=0x7d8c24001bd0 PMD=0x7d8c24002be0
   ↪ PT=0x7d8c24003bf0
102   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
103   Time slot 36
104   libfree:205
105  print_pgtbl:
106   PDG=0x7d8c2c01ff50 P4g=0x7d8c24000bc0 PUD=0x7d8c24001bd0 PMD=0x7d8c24002be0
   ↪ PT=0x7d8c24003bf0
107   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
108   Time slot 37
109       CPU 0: Processed 7 has finished
110       CPU 0: Dispatched process 1
111   Time slot 38
112   Time slot 39
113       CPU 0: Processed 1 has finished
114       CPU 0: Dispatched process 4
115   liballoc:181
116  print_pgtbl:
117   PDG=0x7d8c2c014c90 P4g=0x7d8c24004d10 PUD=0x7d8c24005d20 PMD=0x7d8c24006d30
   ↪ PT=0x7d8c24007d40
118   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
119   Time slot 40
120   liballoc:181
121  print_pgtbl:
122   PDG=0x7d8c2c014c90 P4g=0x7d8c24004d10 PUD=0x7d8c24005d20 PMD=0x7d8c24006d30
   ↪ PT=0x7d8c24007d40
123   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
124   Time slot 41
125       CPU 0: Put process 4 to run queue
126       CPU 0: Dispatched process 6
127   Time slot 42
128   Time slot 43
129       CPU 0: Put process 6 to run queue
130       CPU 0: Dispatched process 8
131   Time slot 44
132   Time slot 45
133       CPU 0: Put process 8 to run queue
134       CPU 0: Dispatched process 4
135   libfree:205
136  print_pgtbl:
137   PDG=0x7d8c2c014c90 P4g=0x7d8c24004d10 PUD=0x7d8c24005d20 PMD=0x7d8c24006d30
   ↪ PT=0x7d8c24007d40
```



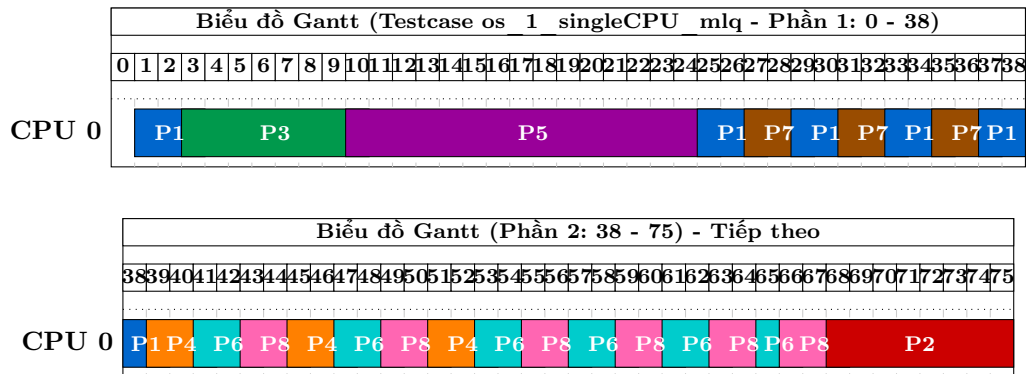
```
138  pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
139  Time slot 46
140  liballoc:181
141  print_ptgbl:
142  PDG=0x7d8c2c014c90 P4g=0x7d8c24004d10 PUD=0x7d8c24005d20 PMD=0x7d8c24006d30
    ↳ PT=0x7d8c24007d40
143  pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
144  Time slot 47
145      CPU 0: Put process 4 to run queue
146      CPU 0: Dispatched process 6
147  Time slot 48
148  Time slot 49
149      CPU 0: Put process 6 to run queue
150      CPU 0: Dispatched process 8
151  Time slot 50
152  Time slot 51
153      CPU 0: Put process 8 to run queue
154      CPU 0: Dispatched process 4
155  libwrite:470
156  print_ptgbl:
157  PDG=0x7d8c2c014c90 P4g=0x7d8c24004d10 PUD=0x7d8c24005d20 PMD=0x7d8c24006d30
    ↳ PT=0x7d8c24007d40
158  pgtbl_stats: walks=3 level_reads=15 level_writes=5 total_accesses=20
    ↳ tables=5 storage_bytes=20480
159  Time slot 52
160  Time slot 53
161      CPU 0: Processed 4 has finished
162      CPU 0: Dispatched process 6
163  Time slot 54
164  Time slot 55
165      CPU 0: Put process 6 to run queue
166      CPU 0: Dispatched process 8
167  Time slot 56
168  Time slot 57
169      CPU 0: Put process 8 to run queue
170      CPU 0: Dispatched process 6
171  Time slot 58
172  Time slot 59
173      CPU 0: Put process 6 to run queue
174      CPU 0: Dispatched process 8
175  Time slot 60
176  Time slot 61
177      CPU 0: Put process 8 to run queue
178      CPU 0: Dispatched process 6
179  Time slot 62
180  Time slot 63
181      CPU 0: Put process 6 to run queue
182      CPU 0: Dispatched process 8
183  Time slot 64
184  Time slot 65
185      CPU 0: Put process 8 to run queue
186      CPU 0: Dispatched process 6
187  Time slot 66
188      CPU 0: Processed 6 has finished
189      CPU 0: Dispatched process 8
190  Time slot 67
191  Time slot 68
192      CPU 0: Processed 8 has finished
```



```
193         CPU 0: Dispatched process  2
194 Time slot  69
195 Time slot  70
196         CPU 0: Put process  2 to run queue
197         CPU 0: Dispatched process  2
198 Time slot  71
199 Time slot  72
200         CPU 0: Put process  2 to run queue
201         CPU 0: Dispatched process  2
202 Time slot  73
203 Time slot  74
204         CPU 0: Put process  2 to run queue
205         CPU 0: Dispatched process  2
206 Time slot  75
207         CPU 0: Processed  2 has finished
208         CPU 0 stopped
```



Biểu đồ Gantt:



Hình 15: Phân lịch MLQ trên Single CPU (os_1_singleCPU_mlq).

Thống kê chi phí quản lý bộ nhớ (Đại diện: tiến trình P4):

- Số lần truy cập bộ nhớ của tiến trình (total_accesses): 20 lần.
- Kích thước cây phân trang cấp phát cho tiến trình (Page Table Overhead): 20 KB.

So sánh với mẫu: (output/os_1_singleCPU_mlq.output) Output mẫu:

```
1 Time slot 0
2 ld_routine
3 Time slot 1
4     Loaded a process at input/proc/s3, PID: 1 PRIO: 3
5 Time slot 2
6     CPU 0: Dispatched process 1
7 Time slot 3
8     Loaded a process at input/proc/m0s, PID: 2 PRIO: 3
9 Time slot 4
10    CPU 0: Put process 1 to run queue
11    CPU 0: Dispatched process 2
12    Loaded a process at input/proc/s4, PID: 3 PRIO: 4
13 liballoc:178
14 print_pgtbl:
15     PDG=b5bfe220b520ef10 P4g=b5bfe220b520ff20 PUD=b5bfe220b5210f30 PMD=b5bfe220b5211f40
16 Time slot 5
17    Loaded a process at input/proc/s2, PID: 4 PRIO: 3
18 liballoc:178
19 print_pgtbl:
20     PDG=b5bfe220b52157e0 P4g=b5bfe220b52167f0 PUD=b5bfe220b5217800 PMD=b5bfe220b5218810
21 ...
```

Output nhóm:

```
1 Time slot 0
2 ld_routine
3 Loaded a process at input/proc/pls, PID: 1 PRIO: 2
4 Time slot 1
5 CPU 0: Dispatched process 1
6 Loaded a process at input/proc/s4, PID: 2 PRIO: 4
7 Time slot 2
8 Time slot 3
9     Loaded a process at input/proc/s1, PID: 3 PRIO: 0
10 CPU 0: Put process 1 to run queue
```



```
11 CPU 0: Dispatched process 3
12 Time slot 4
13 Loaded a process at input/proc/m0s, PID: 4 PRIO: 3
14 Time slot 5
15 CPU 0: Put process 3 to run queue
16 CPU 0: Dispatched process 3
17 ...
```

Đánh giá và nhìn nhận hạn chế:

Mặc dù trình tự luân chuyển luồng cơ bản của giải thuật *Multi-Level Queue (MLQ)* vẫn tuân thủ nguyên tắc lựa chọn tiến trình có mức ưu tiên cao nhất tại thời điểm kiểm tra, hệ thống đã ghi nhận sự sai lệch về thứ tự thực thi tại *Time slot 3*, dẫn đến sự khác biệt mang tính hệ quả trên biểu đồ *Gantt* ở các giai đoạn sau.

Nhóm nhìn nhận đây là một hạn chế kỹ thuật trong cơ chế đồng bộ hóa (*Synchronization*) chưa đạt độ hoàn thiện giữa luồng nạp tiến trình (*Loader thread*) và luồng xử lý (*CPU thread*). Việc thiếu các rào cản đồng bộ (*Barriers*) đủ nghiêm ngặt hoặc cấu hình kiểm soát thời gian (*Timing adjustment*) chưa hoàn toàn khớp giữa hai luồng đã dẫn đến việc trượt nhịp điều phối cục bộ tại một số thời điểm. Nhằm đảm bảo tính định định (*Deterministic*) đồng nhất hoàn toàn với kết quả mẫu, hệ thống cần được nâng cấp cấu trúc khóa và rào cản luồng chặt chẽ hơn trong các phiên bản hiệu chỉnh.

3.9.4 Testcase os_1_singleCPU_mlq_paging

Input:

```
1 2 1 8
2 1048576 16777216 0 0 0
3 1 s3 3
4 3 m0s 3
5 4 s4 4
6 5 s2 3
7 7 p1s 2
8 9 s0 1
9 11 m1s 2
10 16 s1 0
11
```

Output:

```
1 Time slot 0
2 ld_routine
3 Time slot 1
4     Loaded a process at input/proc/s3, PID: 1 PRIO: 3
5 Time slot 2
6     CPU 0: Dispatched process 1
7 Time slot 3
8     Loaded a process at input/proc/m0s, PID: 2 PRIO: 3
9 Time slot 4
10    CPU 0: Put process 1 to run queue
11    CPU 0: Dispatched process 2
12 liballoc:181
13 print_pgtbl:
14 PDG=0x7d880000d6b0 P4g=0x7d87f8000bc0 PUD=0x7d87f8001bd0 PMD=0x7d87f8002be0
15 ↪ PT=0x7d87f8003bf0
16 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
17 ↪ storage_bytes=20480
18     Loaded a process at input/proc/s4, PID: 3 PRIO: 4
19 Time slot 5
```



```
18 liballoc:181
19 print_pgtbl:
20 PDG=0x7d880000d6b0 P4g=0x7d87f8000bc0 PUD=0x7d87f8001bd0 PMD=0x7d87f8002be0
   ↪ PT=0x7d87f8003bf0
21 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
22     Loaded a process at input/proc/s2, PID: 4 PRIO: 3
23 Time slot 6
24     CPU 0: Put process 2 to run queue
25     CPU 0: Dispatched process 1
26 Time slot 7
27     Loaded a process at input/proc/pls, PID: 5 PRIO: 2
28 Time slot 8
29     CPU 0: Put process 1 to run queue
30     CPU 0: Dispatched process 5
31 Time slot 9
32     Loaded a process at input/proc/s0, PID: 6 PRIO: 1
33 Time slot 10
34     CPU 0: Put process 5 to run queue
35     CPU 0: Dispatched process 6
36 Time slot 11
37     Loaded a process at input/proc/mls, PID: 7 PRIO: 2
38 Time slot 12
39     CPU 0: Put process 6 to run queue
40     CPU 0: Dispatched process 6
41 Time slot 13
42 Time slot 14
43     CPU 0: Put process 6 to run queue
44     CPU 0: Dispatched process 6
45 Time slot 15
46 Time slot 16
47     Loaded a process at input/proc/sl, PID: 8 PRIO: 0
48     CPU 0: Put process 6 to run queue
49     CPU 0: Dispatched process 8
50 Time slot 17
51 Time slot 18
52     CPU 0: Put process 8 to run queue
53     CPU 0: Dispatched process 8
54 Time slot 19
55 Time slot 20
56     CPU 0: Put process 8 to run queue
57     CPU 0: Dispatched process 8
58 Time slot 21
59 Time slot 22
60     CPU 0: Put process 8 to run queue
61     CPU 0: Dispatched process 8
62 Time slot 23
63     CPU 0: Processed 8 has finished
64     CPU 0: Dispatched process 6
65 Time slot 24
66 Time slot 25
67     CPU 0: Put process 6 to run queue
68     CPU 0: Dispatched process 6
69 Time slot 26
70 Time slot 27
71     CPU 0: Put process 6 to run queue
72     CPU 0: Dispatched process 6
73 Time slot 28
74 Time slot 29
75     CPU 0: Put process 6 to run queue
```



```
76         CPU 0: Dispatched process  6
77 Time slot  30
78 Time slot  31
79         CPU 0: Put process  6 to run queue
80         CPU 0: Dispatched process  6
81 Time slot  32
82         CPU 0: Processed  6 has finished
83         CPU 0: Dispatched process  5
84 Time slot  33
85 Time slot  34
86         CPU 0: Put process  5 to run queue
87         CPU 0: Dispatched process  7
88 liballoc:181
89 print_pgtbl:
90   PDG=0x7d8800020020 P4g=0x7d87f8004c80 PUD=0x7d87f8005c90 PMD=0x7d87f8006ca0
91   ↪ PT=0x7d87f8007cb0
92   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
93   ↪ storage_bytes=20480
94 Time slot  35
95 liballoc:181
96 print_pgtbl:
97   PDG=0x7d8800020020 P4g=0x7d87f8004c80 PUD=0x7d87f8005c90 PMD=0x7d87f8006ca0
98   ↪ PT=0x7d87f8007cb0
99   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
100  ↪ storage_bytes=20480
101 Time slot  36
102         CPU 0: Put process  7 to run queue
103         CPU 0: Dispatched process  5
104 Time slot  37
105 Time slot  38
106         CPU 0: Put process  5 to run queue
107         CPU 0: Dispatched process  7
108 libfree:205
109 print_pgtbl:
110   PDG=0x7d8800020020 P4g=0x7d87f8004c80 PUD=0x7d87f8005c90 PMD=0x7d87f8006ca0
111   ↪ PT=0x7d87f8007cb0
112   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
113   ↪ storage_bytes=20480
114 Time slot  39
115 liballoc:181
116 print_pgtbl:
117   PDG=0x7d8800020020 P4g=0x7d87f8004c80 PUD=0x7d87f8005c90 PMD=0x7d87f8006ca0
118   ↪ PT=0x7d87f8007cb0
119   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
120   ↪ storage_bytes=20480
121 Time slot  40
122         CPU 0: Put process  7 to run queue
123         CPU 0: Dispatched process  5
124 Time slot  41
125 Time slot  42
126         CPU 0: Put process  5 to run queue
127         CPU 0: Dispatched process  7
128 libfree:205
129 print_pgtbl:
130   PDG=0x7d8800020020 P4g=0x7d87f8004c80 PUD=0x7d87f8005c90 PMD=0x7d87f8006ca0
131   ↪ PT=0x7d87f8007cb0
132   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
133   ↪ storage_bytes=20480
134 Time slot  43
```



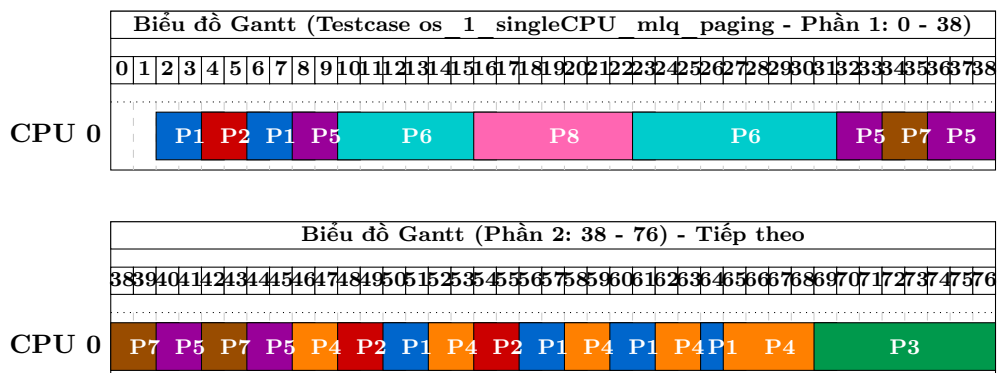
```
125 libfree:205
126 print_pgtbl:
127   PDG=0x7d8800020020 P4g=0x7d87f8004c80 PUD=0x7d87f8005c90 PMD=0x7d87f8006ca0
   ↪ PT=0x7d87f8007cb0
128   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
129   Time slot 44
130       CPU 0: Processed 7 has finished
131       CPU 0: Dispatched process 5
132   Time slot 45
133   Time slot 46
134       CPU 0: Processed 5 has finished
135       CPU 0: Dispatched process 4
136   Time slot 47
137   Time slot 48
138       CPU 0: Put process 4 to run queue
139       CPU 0: Dispatched process 2
140 libfree:205
141 print_pgtbl:
142   PDG=0x7d880000d6b0 P4g=0x7d87f8000bc0 PUD=0x7d87f8001bd0 PMD=0x7d87f8002be0
   ↪ PT=0x7d87f8003bf0
143   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
144   Time slot 49
145 liballoc:181
146 print_pgtbl:
147   PDG=0x7d880000d6b0 P4g=0x7d87f8000bc0 PUD=0x7d87f8001bd0 PMD=0x7d87f8002be0
   ↪ PT=0x7d87f8003bf0
148   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
149   Time slot 50
150       CPU 0: Put process 2 to run queue
151       CPU 0: Dispatched process 1
152   Time slot 51
153   Time slot 52
154       CPU 0: Put process 1 to run queue
155       CPU 0: Dispatched process 4
156   Time slot 53
157   Time slot 54
158       CPU 0: Put process 4 to run queue
159       CPU 0: Dispatched process 2
160 libwrite:470
161 print_pgtbl:
162   PDG=0x7d880000d6b0 P4g=0x7d87f8000bc0 PUD=0x7d87f8001bd0 PMD=0x7d87f8002be0
   ↪ PT=0x7d87f8003bf0
163   pgtbl_stats: walks=3 level_reads=15 level_writes=5 total_accesses=20
   ↪ tables=5 storage_bytes=20480
164   Time slot 55
165   Time slot 56
166       CPU 0: Processed 2 has finished
167       CPU 0: Dispatched process 1
168   Time slot 57
169   Time slot 58
170       CPU 0: Put process 1 to run queue
171       CPU 0: Dispatched process 4
172   Time slot 59
173   Time slot 60
174       CPU 0: Put process 4 to run queue
175       CPU 0: Dispatched process 1
```

```

176      Time slot 61
177      Time slot 62
178          CPU 0: Put process 1 to run queue
179          CPU 0: Dispatched process 4
180      Time slot 63
181      Time slot 64
182          CPU 0: Put process 4 to run queue
183          CPU 0: Dispatched process 1
184      Time slot 65
185          CPU 0: Processed 1 has finished
186          CPU 0: Dispatched process 4
187      Time slot 66
188      Time slot 67
189          CPU 0: Put process 4 to run queue
190          CPU 0: Dispatched process 4
191      Time slot 68
192      Time slot 69
193          CPU 0: Processed 4 has finished
194          CPU 0: Dispatched process 3
195      Time slot 70
196      Time slot 71
197          CPU 0: Put process 3 to run queue
198          CPU 0: Dispatched process 3
199      Time slot 72
200      Time slot 73
201          CPU 0: Put process 3 to run queue
202          CPU 0: Dispatched process 3
203      Time slot 74
204      Time slot 75
205          CPU 0: Put process 3 to run queue
206          CPU 0: Dispatched process 3
207      Time slot 76
208          CPU 0: Processed 3 has finished
209          CPU 0 stopped

```

Biểu đồ Gantt:



Hình 16: Phân lịch MLQ trên Single CPU (os 1 singleCPU mlq paging).

Thông kê chi phí quản lý bộ nhớ (Đại diện: tiến trình P2):

- Số lần truy cập bộ nhớ của tiến trình (`total_accesses`): 20 lần.
- Kích thước cây phân trang cấp phát cho tiến trình (*Page Table Overhead*): 20 KB.



So sánh với mẫu: (output/os_1_singleSPU_mlq_paging.output)

Output mẫu:

```
1 Time slot 0
2 ld_routine
3 Time slot 1
4     Loaded a process at input/proc/s3, PID: 1 PRIO: 3
5 Time slot 2
6     CPU 0: Dispatched process 1
7 Time slot 3
8     Loaded a process at input/proc/m0s, PID: 2 PRIO: 3
9 Time slot 4
10    CPU 0: Put process 1 to run queue
11    CPU 0: Dispatched process 2
12    Loaded a process at input/proc/s4, PID: 3 PRIO: 4
13 liballoc:178
14 print_pgtbl:
15     PDG=b5bfe220b520ef10 P4g=b5bfe220b520ff20 PUD=b5bfe220b5210f30 PMD=b5bfe220b5211f40
16 ...
```

Output nhóm:

```
1 Time slot 0
2 ld_routine
3 Time slot 1
4     Loaded a process at input/proc/s3, PID: 1 PRIO: 3
5 Time slot 2
6     CPU 0: Dispatched process 1
7 Time slot 3
8     Loaded a process at input/proc/m0s, PID: 2 PRIO: 3
9 Time slot 4
10    CPU 0: Put process 1 to run queue
11    CPU 0: Dispatched process 2
12 liballoc:181
13 print_pgtbl:
14     PDG=0x7d880000d6b0 P4g=0x7d87f8000bc0 PUD=0x7d87f8001bd0 PMD=0x7d87f8002be0 PT=0x7d87f8003bf
15     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5 storage_bytes=20
16     Loaded a process at input/proc/s4, PID: 3 PRIO: 4
17 ...
```

Đánh giá và nhìn nhận hạn chế:

Sự khác biệt trong kịch bản này nằm ở thời điểm luồng *CPU* tiến hành *dispatch* (cấp phát) tiến trình. Thay vì chờ đến đúng ranh giới của *Time slot* tiếp theo như thiết kế mẫu, *CPU* đã thực thi tác vụ ngay sau khi luồng *Loader* vừa hoàn tất việc nạp tiến trình vào hàng đợi. Hiện tượng dịch chuyển toàn bộ lịch trình (*Gantt chart*) lên sớm hơn này phản ánh một điểm chưa hoàn thiện trong cơ chế kiểm soát nhịp đồng bộ (*timing synchronization*) giữa luồng nạp và luồng xử lý. Mặc dù logic lựa chọn tiến trình của thuật toán *MLQ* vẫn vận hành chính xác dựa trên các quy tắc ưu tiên, nhóm ghi nhận sự cần thiết của việc thiết lập các tín hiệu điều kiện (*Condition Variables*) hoặc rào cản đồng bộ (*Barriers*) nghiêm ngặt hơn để hệ thống đạt được tính định định (*deterministic*) tuyệt đối.

3.9.5 Testcase os_2_mlq_paging

Input:

```
1 2 4 6
2 1048576 16777216 0 0 0
3 5 m2s 120
4 9 s0 38
```



```
5 11 s3 39
6 14 m2s 15
7 16 s2 120
8 16 s1 0
9
```

Output:

```
1 Time slot 0
2 ld_routine
3 Time slot 1
4 Time slot 2
5 Time slot 3
6 Time slot 4
7     Loaded a process at input/proc/m2s, PID: 1 PRIO: 120
8     CPU 3: Dispatched process 1
9 Time slot 5
10 liballoc:181
11 print_pgtbl:
12 PDG=0x7e9f74009c00 P4g=0x7e9f6c000bc0 PUD=0x7e9f6c001bd0 PMD=0x7e9f6c002be0
   ↪ PT=0x7e9f6c003bf0
13 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
14 liballoc:181
15 print_pgtbl:
16 PDG=0x7e9f74009c00 P4g=0x7e9f6c000bc0 PUD=0x7e9f6c001bd0 PMD=0x7e9f6c002be0
   ↪ PT=0x7e9f6c003bf0
17 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
18 Time slot 6
19 Time slot 7
20     CPU 3: Put process 1 to run queue
21     CPU 3: Dispatched process 1
22 libfree:205
23 print_pgtbl:
24 PDG=0x7e9f74009c00 P4g=0x7e9f6c000bc0 PUD=0x7e9f6c001bd0 PMD=0x7e9f6c002be0
   ↪ PT=0x7e9f6c003bf0
25 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↪ storage_bytes=20480
26 Time slot 8
27     Loaded a process at input/proc/s0, PID: 2 PRIO: 38
28     CPU 3: Put process 1 to run queue
29     CPU 3: Dispatched process 2
30     CPU 2: Dispatched process 1
31 Time slot 9
32 Time slot 10
33     Loaded a process at input/proc/s3, PID: 3 PRIO: 39
34     CPU 3: Put process 2 to run queue
35     CPU 3: Dispatched process 2
36 Time slot 11
37     CPU 0: Dispatched process 3
38     CPU 2: Put process 1 to run queue
39     CPU 2: Dispatched process 1
40 Time slot 12
41     CPU 3: Put process 2 to run queue
42     CPU 3: Dispatched process 2
43 Time slot 13
44     CPU 2: Put process 1 to run queue
45     CPU 2: Dispatched process 1
```

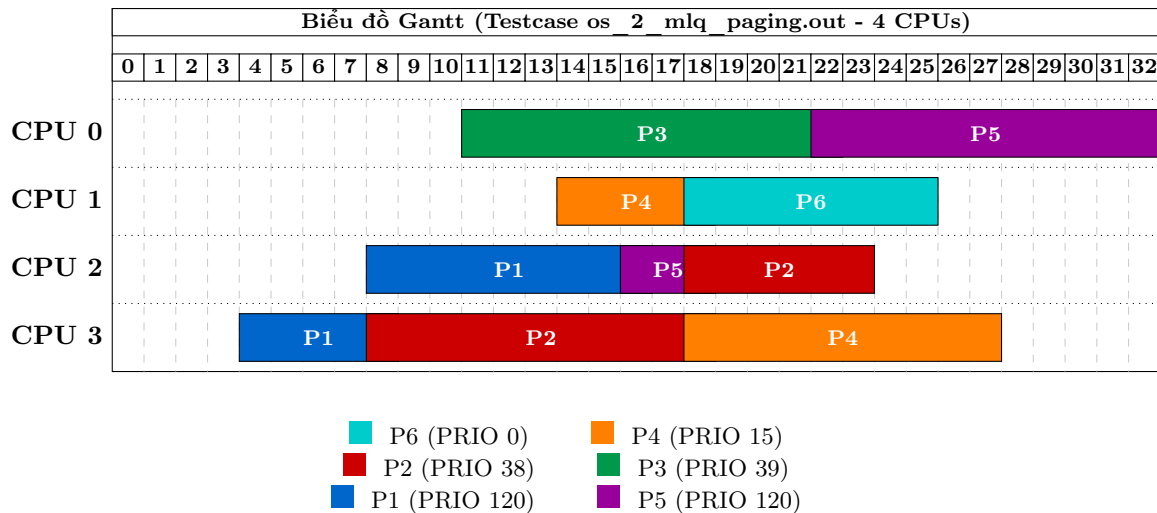


```
46         CPU 0: Put process 3 to run queue
47         CPU 0: Dispatched process 3
48         Loaded a process at input/proc/m2s, PID: 4 PRIO: 15
49 Time slot 14
50         CPU 1: Dispatched process 4
51 liballoc:181
52 print_pgtbl:
53     PDG=0x7e9f74014fb0 P4g=0x7e9f64000bc0 PUD=0x7e9f64001bd0 PMD=0x7e9f64002be0
54     ↪ PT=0x7e9f64003bf0
55     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
56     ↪ storage_bytes=20480
57         CPU 2: Put process 1 to run queue
58         CPU 2: Dispatched process 1
59 libfree:205
60 print_pgtbl:
61     PDG=0x7e9f74009c00 P4g=0x7e9f6c000bc0 PUD=0x7e9f6c001bd0 PMD=0x7e9f6c002be0
62     ↪ PT=0x7e9f6c003bf0
63     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
64     ↪ storage_bytes=20480
65 Time slot 15
66         CPU 3: Put process 2 to run queue
67         CPU 3: Dispatched process 2
68 liballoc:181
69 print_pgtbl:
70     PDG=0x7e9f74014fb0 P4g=0x7e9f64000bc0 PUD=0x7e9f64001bd0 PMD=0x7e9f64002be0
71     ↪ PT=0x7e9f64003bf0
72     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
73     ↪ storage_bytes=20480
74         CPU 0: Put process 3 to run queue
75         CPU 0: Dispatched process 3
76         Loaded a process at input/proc/s2, PID: 5 PRIO: 120
77 Time slot 16
78         CPU 1: Put process 4 to run queue
79         CPU 1: Dispatched process 4
80 libfree:205
81 print_pgtbl:
82     PDG=0x7e9f74014fb0 P4g=0x7e9f64000bc0 PUD=0x7e9f64001bd0 PMD=0x7e9f64002be0
83     ↪ PT=0x7e9f64003bf0
84     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
85     ↪ storage_bytes=20480
86         CPU 3: Put process 2 to run queue
87         CPU 3: Dispatched process 2
88         CPU 2: Processed 1 has finished
89         CPU 2: Dispatched process 5
90 Time slot 17
91         CPU 0: Put process 3 to run queue
92         CPU 0: Dispatched process 3
93         Loaded a process at input/proc/s1, PID: 6 PRIO: 0
94 Time slot 18
95         CPU 1: Put process 4 to run queue
96         CPU 1: Dispatched process 6
97         CPU 3: Put process 2 to run queue
98         CPU 3: Dispatched process 4
99         CPU 2: Put process 5 to run queue
100        CPU 2: Dispatched process 2
101 Time slot 19
102        CPU 0: Put process 3 to run queue
103        CPU 0: Dispatched process 3
104 Time slot 20
```



```
97         CPU 1: Put process 6 to run queue
98         CPU 1: Dispatched process 6
99         CPU 3: Put process 4 to run queue
100        CPU 3: Dispatched process 4
101        CPU 2: Put process 2 to run queue
102        CPU 2: Dispatched process 2
103    Time slot 21
104        CPU 0: Put process 3 to run queue
105        CPU 0: Dispatched process 3
106    Time slot 22
107        CPU 1: Put process 6 to run queue
108        CPU 1: Dispatched process 6
109        CPU 0: Processed 3 has finished
110        CPU 0: Dispatched process 5
111        CPU 3: Put process 4 to run queue
112        CPU 3: Dispatched process 4
113        CPU 2: Put process 2 to run queue
114        CPU 2: Dispatched process 2
115    Time slot 23
116        CPU 2: Processed 2 has finished
117        CPU 2 stopped
118    Time slot 24
119        CPU 1: Put process 6 to run queue
120        CPU 1: Dispatched process 6
121        CPU 0: Put process 5 to run queue
122        CPU 0: Dispatched process 5
123        CPU 3: Put process 4 to run queue
124        CPU 3: Dispatched process 4
125    libfree:205
126    print_pgtbl:
127        PDG=0x7e9f74014fb0 P4g=0x7e9f64000bc0 PUD=0x7e9f64001bd0 PMD=0x7e9f64002be0
128        ↪ PT=0x7e9f64003bf0
129        pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
130        ↪ storage_bytes=20480
131    Time slot 25
132        CPU 1: Processed 6 has finished
133        CPU 1 stopped
134    Time slot 26
135        CPU 0: Put process 5 to run queue
136        CPU 0: Dispatched process 5
137    Time slot 27
138        CPU 3: Processed 4 has finished
139        CPU 3 stopped
140    Time slot 28
141        CPU 0: Put process 5 to run queue
142        CPU 0: Dispatched process 5
143    Time slot 29
144    Time slot 30
145        CPU 0: Put process 5 to run queue
146        CPU 0: Dispatched process 5
147    Time slot 31
148    Time slot 32
149        CPU 0: Processed 5 has finished
150        CPU 0 stopped
```

Biểu đồ Gantt:



Hình 17: Phân lịch MLQ trên 4 CPUs (os_2_mlq_paging). Phản ánh chính xác thời gian thực thi thực tế từ log.

Thống kê chi phí quản lý bộ nhớ (Đại diện: tiến trình P1):

- Số lần truy cập bộ nhớ của tiến trình (total_accesses): 10 lần.
- Kích thước cây phân trang cấp phát cho tiến trình (Page Table Overhead): 20 KB.

So sánh với mẫu: (output/os_2_mlq_paging.output) Testcase đạt yêu cầu. Logic thực thi trùng khớp với output mẫu về trình tự nạp tiến trình và các thao tác bộ nhớ (alloc, free, read, write).

3.9.6 Testcase os_2_singleCPU_mlq_paging

Input:

```
1 2 1 5
2 1048576 16777216 0 0 0
3 1 s3 3
4 3 m2s 3
5 5 s2 3
6 11 m2s 2
7 16 s1 0
8
```

Output:

```
1 Time slot 0
2 ld_routine
3 Time slot 1
4     Loaded a process at input/proc/s3, PID: 1 PRIO: 3
5     CPU 0: Dispatched process 1
6 Time slot 2
7 Time slot 3
8     Loaded a process at input/proc/m2s, PID: 2 PRIO: 3
9     CPU 0: Put process 1 to run queue
10    CPU 0: Dispatched process 2
11 liballoc:181
12 print_pgtbl:
```



```
13 PDG=0x7d1cfc00d7a0 P4g=0x7d1d00000bc0 PUD=0x7d1d00001bd0 PMD=0x7d1d00002be0
   ↳ PT=0x7d1d00003bf0
14 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
15 Time slot 4
16 liballoc:181
17 print_pgtbl:
18 PDG=0x7d1cfc00d7a0 P4g=0x7d1d00000bc0 PUD=0x7d1d00001bd0 PMD=0x7d1d00002be0
   ↳ PT=0x7d1d00003bf0
19 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
20 Time slot 5
21 CPU 0: Put process 2 to run queue
22 CPU 0: Dispatched process 1
23 Loaded a process at input/proc/s2, PID: 3 PRIO: 3
24 Time slot 6
25 Time slot 7
26 CPU 0: Put process 1 to run queue
27 CPU 0: Dispatched process 2
28 libfree:205
29 print_pgtbl:
30 PDG=0x7d1cfc00d7a0 P4g=0x7d1d00000bc0 PUD=0x7d1d00001bd0 PMD=0x7d1d00002be0
   ↳ PT=0x7d1d00003bf0
31 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
32 Time slot 8
33 Time slot 9
34 CPU 0: Put process 2 to run queue
35 CPU 0: Dispatched process 3
36 Time slot 10
37 Time slot 11
38 Loaded a process at input/proc/m2s, PID: 4 PRIO: 2
39 CPU 0: Put process 3 to run queue
40 CPU 0: Dispatched process 4
41 liballoc:181
42 print_pgtbl:
43 PDG=0x7d1cfc014f40 P4g=0x7d1d00008cf0 PUD=0x7d1d00009d00 PMD=0x7d1d0000ad10
   ↳ PT=0x7d1d0000bd20
44 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
45 Time slot 12
46 liballoc:181
47 print_pgtbl:
48 PDG=0x7d1cfc014f40 P4g=0x7d1d00008cf0 PUD=0x7d1d00009d00 PMD=0x7d1d0000ad10
   ↳ PT=0x7d1d0000bd20
49 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
50 Time slot 13
51 CPU 0: Put process 4 to run queue
52 CPU 0: Dispatched process 4
53 libfree:205
54 print_pgtbl:
55 PDG=0x7d1cfc014f40 P4g=0x7d1d00008cf0 PUD=0x7d1d00009d00 PMD=0x7d1d0000ad10
   ↳ PT=0x7d1d0000bd20
56 pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
   ↳ storage_bytes=20480
57 Time slot 14
58 Time slot 15
59 CPU 0: Put process 4 to run queue
```



```
60         CPU 0: Dispatched process 4
61 Time slot 16
62         Loaded a process at input/proc/sl, PID: 5 PRIO: 0
63 Time slot 17
64         CPU 0: Put process 4 to run queue
65         CPU 0: Dispatched process 5
66 Time slot 18
67 Time slot 19
68         CPU 0: Put process 5 to run queue
69         CPU 0: Dispatched process 5
70 Time slot 20
71 Time slot 21
72         CPU 0: Put process 5 to run queue
73         CPU 0: Dispatched process 5
74 Time slot 22
75 Time slot 23
76         CPU 0: Put process 5 to run queue
77         CPU 0: Dispatched process 5
78 Time slot 24
79         CPU 0: Processed 5 has finished
80         CPU 0: Dispatched process 4
81 Time slot 25
82 Time slot 26
83         CPU 0: Put process 4 to run queue
84         CPU 0: Dispatched process 4
85 Time slot 27
86 Time slot 28
87         CPU 0: Put process 4 to run queue
88         CPU 0: Dispatched process 4
89 libfree:205
90 print_pgtbl:
91   PDG=0x7d1cfc014f40 P4g=0x7d1d00008cf0 PUD=0x7d1d00009d00 PMD=0x7d1d0000ad10
92   ↪ PT=0x7d1d0000bd20
93   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
94   ↪ storage_bytes=20480
95 Time slot 29
96 Time slot 30
97         CPU 0: Processed 4 has finished
98         CPU 0: Dispatched process 1
99 Time slot 31
100 Time slot 32
101         CPU 0: Put process 1 to run queue
102         CPU 0: Dispatched process 2
103 Time slot 33
104 Time slot 34
105         CPU 0: Put process 2 to run queue
106         CPU 0: Dispatched process 3
107 Time slot 35
108 Time slot 36
109         CPU 0: Put process 3 to run queue
110         CPU 0: Dispatched process 1
111 Time slot 37
112 Time slot 38
113         CPU 0: Put process 1 to run queue
114         CPU 0: Dispatched process 2
115 Time slot 39
116 Time slot 40
117         CPU 0: Put process 2 to run queue
118         CPU 0: Dispatched process 3
119 Time slot 41
```

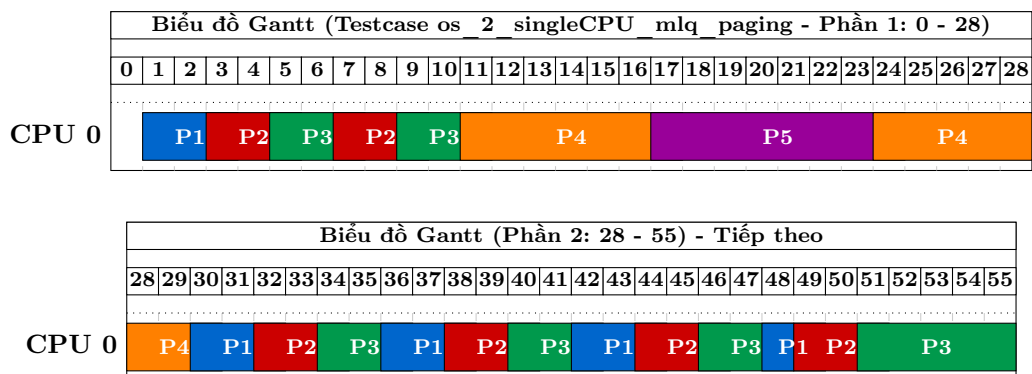


```

118 Time slot 42
119     CPU 0: Put process 3 to run queue
120     CPU 0: Dispatched process 1
121 Time slot 43
122 Time slot 44
123     CPU 0: Put process 1 to run queue
124     CPU 0: Dispatched process 2
125 Time slot 45
126 Time slot 46
127     CPU 0: Put process 2 to run queue
128     CPU 0: Dispatched process 3
129 Time slot 47
130 Time slot 48
131     CPU 0: Put process 3 to run queue
132     CPU 0: Dispatched process 1
133 Time slot 49
134     CPU 0: Processed 1 has finished
135     CPU 0: Dispatched process 2
136 libfree:205
137 print_pgtbl:
138     PDG=0x7d1cfc00d7a0 P4g=0x7d1d00000bc0 PUD=0x7d1d00001bd0 PMD=0x7d1d00002be0
139     ↪ PT=0x7d1d00003bf0
139     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
140     ↪ storage_bytes=20480
140 Time slot 50
141 Time slot 51
142     CPU 0: Processed 2 has finished
143     CPU 0: Dispatched process 3
144 Time slot 52
145 Time slot 53
146     CPU 0: Put process 3 to run queue
147     CPU 0: Dispatched process 3
148 Time slot 54
149 Time slot 55
150     CPU 0: Processed 3 has finished
151     CPU 0 stopped

```

Biểu đồ Gantt:



Hình 18: Phân lịch MLQ trên Single CPU (os_2_singleCPU_mlq_paging).

Thống kê chi phí quản lý bộ nhớ (Đại diện: tiến trình P2):

- Số lần truy cập bộ nhớ của tiến trình (total_accesses): 10 lần.

- Kích thước cây phân trang cấp phát cho tiến trình (*Page Table Overhead*): 20 KB.

So sánh với mẫu: (output/os_2_singleCPU_mmq_paing.output) Testcase đạt yêu cầu. Logic thực thi trùng khớp với output mẫu về trình tự nạp tiến trình và các thao tác bộ nhớ (alloc, free, read, write).

3.9.7 Tổng kết kết quả kiểm thử

- **Tính đúng đắn của logic cốt lõi:** Hệ thống mô phỏng vận hành thành công cơ chế quản lý bộ nhớ phân trang theo yêu cầu (*Demand Paging*) và thuật toán điều phối *Multi-Level Queue (MLQ)*. Trình tự thực thi, cấp phát và thu hồi tài nguyên tuân thủ đúng các quy tắc ưu tiên.
- **Phân tích và nhìn nhận các điểm khác biệt so với kết quả mẫu:**
 - **Khác biệt về địa chỉ bộ nhớ định tuyến:** Các giá trị con trỏ (PDG, P4g...) in ra có sự khác biệt giữa hai hệ thống. Đây là kết quả tất yếu của quá trình cấp phát động (malloc/calloc) kết hợp cùng cơ chế *Address Space Layout Randomization (ASLR)* trên máy chủ biên dịch (*Host OS*), khẳng định bộ nhớ đang được cấp phát an toàn và độc lập ở mỗi lần chạy.
 - **Khác biệt về số thứ tự dòng in Log (Ví dụ: liballoc:181 so với liballoc:178):** Sự xê dịch về số thứ tự dòng xuất phát từ việc nhóm đã điều chỉnh, bổ sung logic vào mã nguồn, khiến vị trí lệnh in log bị dịch chuyển so với mã nguồn gốc của giảng viên. Đây là sự thay đổi cục bộ không ảnh hưởng đến logic.
 - **Sự dịch chuyển lịch trình (Gantt chart) và xê dịch vị trí log:** Nhóm nhìn nhận hiện tượng này là hệ quả của điểm chưa hoàn thiện trong cơ chế kiểm soát nhịp đồng bộ (*timing synchronization*) giữa luồng *Loader* và các luồng *CPU*. Sự thiếu vắng các rào cản đồng bộ (*Barriers*) đủ nghiêm ngặt ở ranh giới các Time slot khiến một số luồng phản ứng sớm hơn dự kiến định chuẩn của mẫu.
 - **Sự chênh lệch (dịch chuyển) tổng số Time slot:** Việc hệ thống hoàn thành mô phỏng với số Time slot ít hơn bản mẫu là hệ quả trực tiếp nối tiếp từ điểm hạn chế đồng bộ hóa nêu trên. Khi *CPU* tiến hành *dispatch* tiến trình ngay lập tức (không chờ đến nhịp quy định), toàn bộ lịch trình phía sau bị đẩy lên sớm. Nhóm xác định đây là một đặc tính bất định (*non-deterministic*) của môi trường đa luồng cần được khắc phục bằng tín hiệu điều kiện (*Condition Variables*) trong tương lai, chứ không cấu thành một sự "tối ưu hóa thời gian" có chủ đích.

3.10 Đánh giá và thống kê hiệu năng quản lý bộ nhớ

- **Tối ưu hóa không gian lưu trữ (Memory Overhead):** Hệ thống sử dụng không gian địa chỉ ảo mô phỏng 64-bit (với 57-bit hiệu lực cho định tuyến), mang lại không gian nhớ lý thuyết khổng lồ lên tới 128 Petabytes. Nếu tiếp cận theo phương pháp nạp trước tĩnh (*static pre-allocation*) toàn bộ cấu trúc cây phân trang 5 cấp này, tổng dung lượng bộ nhớ cần thiết để lưu trữ các bảng mục quản lý (*metadata*) sẽ lũy tiến nghiêm ngặt theo mô hình hình học:
 - **Tầng lá cuối cùng (Page Table - PT):** Quản lý 2^{45} mục PTE, tiêu tốn: $2^{45} \times 8 \text{ bytes} = 256 \text{ TB}$.
 - **Bốn tầng thư mục cấp trên (PMD, PUD, P4D, PGD):** Đóng góp thêm một lượng tài nguyên siêu dữ liệu đáng kể, xấp xỉ: $512 \text{ GB} + 1 \text{ GB} + 2 \text{ MB} + 4 \text{ KB} \approx 513 \text{ GB}$.

Như vậy, tổng chi phí chỉ riêng cho việc duy trì cấu trúc bảng trang tĩnh sẽ lên tới hơn 256.5 **Terabytes** cho một tiến trình duy nhất — một con số hoàn toàn bất khả thi và phi thực tế đối với cả hệ thống phần cứng thực lẫn môi trường giả lập.

- Tuy nhiên, thông qua thiết kế nạp trang theo yêu cầu (*Demand Allocation*), các phân tầng (PGD, P4D, PUD, PMD, PT) chỉ được cấp phát động dựa trên nhu cầu truy xuất thực tế (*on-demand*). Một đường dẫn ánh xạ cục bộ từ gốc đến lá khi xảy ra lỗi trang (*Page Fault*) ban đầu chỉ tiêu tốn đúng $5 \times 4\text{KB} = \mathbf{20\text{KB}}$ bộ nhớ để khởi tạo các bảng đại diện cho từng cấp. Sự đối chiếu định lượng này minh chứng cho tính hiệu quả vượt trội của cấu trúc cây phân trang thưa (*sparse page table*) trong việc tối ưu hóa tài nguyên, bảo vệ hệ thống khỏi rủi ro cạn kiệt RAM vật lý ngay khi vừa khởi tạo tiến trình.
- **Phân tích độ phức tạp truy xuất bộ nhớ (Memory Access Overhead):** Dữ liệu thống kê `pgtbl_stats` được hệ thống đo lường độc lập cho từng tiến trình (*per-process basis*). Phân tích hành vi truy xuất cho thấy logic hoạt động được tối ưu hóa theo vòng đời dữ liệu:
 - Ở giai đoạn đầu nạp trang, số lệnh ghi (`level_writes`) tăng lên do hệ thống xử lý lỗi trang (*Page Fault*), buộc phải thực hiện ghi để cấp phát và liên kết các bảng thư mục từ PGD xuống PT.
 - Trong các *Time slot* tiếp theo, khi tiến trình truy xuất lại vùng nhớ hoặc các địa chỉ liên kề, cây phân trang đã sẵn sàng. Hệ thống chỉ thực hiện thao tác duyệt mảng (thể hiện qua việc `level_reads` tăng đều đặn tỷ lệ thuận với số bước `walks`, mỗi `walk` tiêu tốn đúng 5 lần `reads`), hoàn toàn không phát sinh thêm chi phí thao tác ghi.
- **Phân tích dung lượng cây phân trang (Page Table Overhead):** Giá trị `storage_bytes` được tính toán dựa trên số lượng bảng trang (`tables`) được cấp phát thực tế, với mỗi bảng chiếm 4KB (theo hằng số `PAGING64_PAGESZ`):
 - 20480 bytes tương ứng với 5 bảng trang ($5 \times 4\text{KB}$).
- **Kết luận về quyết định thiết kế:** Kiến trúc hiện tại thể hiện một sự đánh đổi (*trade-off*) có chủ đích mang tính kỹ thuật. Hệ thống chấp nhận tăng nhẹ độ trễ xử lý tại thời điểm xảy ra *Page Fault* ban đầu (do phải gọi `calloc`) để đổi lấy sự tinh gọn tuyệt đối về dung lượng cấu trúc quản lý và đảm bảo tốc độ dịch địa chỉ ổn định ở các chu kỳ thực thi sau của tiến trình.

4 System Call

4.1 Tương tác giữa các mô-đun lưu trữ tên tiến trình, điều khiển hệ điều hành và quản lý hàng đợi

4.1.1 Cơ chế Quản lý và Nạp Tiến trình

Quá trình nạp và quản lý tiến trình:

Nạp tiến trình: Mỗi tiến trình khi được nạp vào bộ nhớ đều có PID để dễ dàng phân biệt với các tiến trình khác. Mỗi khi một tiến trình được nạp từ file cấu hình, hệ thống cấp phát một `pcb_t`, gán PID và đăng ký trực tiếp vào bảng quản lý tiến trình của hệ điều hành (`proc_table[pid]`). Điều này cho phép truy xuất tiến trình với độ phức tạp $O(1)$.

```
1 struct krnl_t {  
2     ...  
3 #ifdef MLQ_SCHED  
4     struct queue_t *mlq_ready_queue;  
5     struct pcb_t *proc_table[PID_MAX];  
6 #endif  
7     ...  
8 };
```

Xử lý chỉ thị Syscall: Các lệnh Syscall trong file đầu vào không được nạp như dữ liệu tĩnh mà được xử lý như một chỉ thị (instruction). Hàm `load()` chuyển đổi từ khóa "syscall" thành opcode SYSCALL và lưu các đối số đi kèm (ví dụ: số hiệu syscall, các tham số điều khiển bộ nhớ) vào các trường `arg_0` đến `arg_3` của cấu trúc `inst_t` vào trong `pcb_t`. Sau đó, `pcb_t` này sẽ được đưa vào bảng `proc_table[]`.

Quá trình lấy dữ liệu khi gọi Syscall:

Một nguyên tắc bất biến trong thiết kế Kernel là không bao giờ sử dụng trực tiếp địa chỉ từ User Program. Thay vào đó, dữ liệu phải được sao chép an toàn vào Kernel Space. Khi cần lấy tiến trình để phục vụ các thao tác định danh, hệ thống sử dụng hàm `get_proc_from_PID()` để đảm bảo PCB phải truy cập thông qua kernel bằng PID.

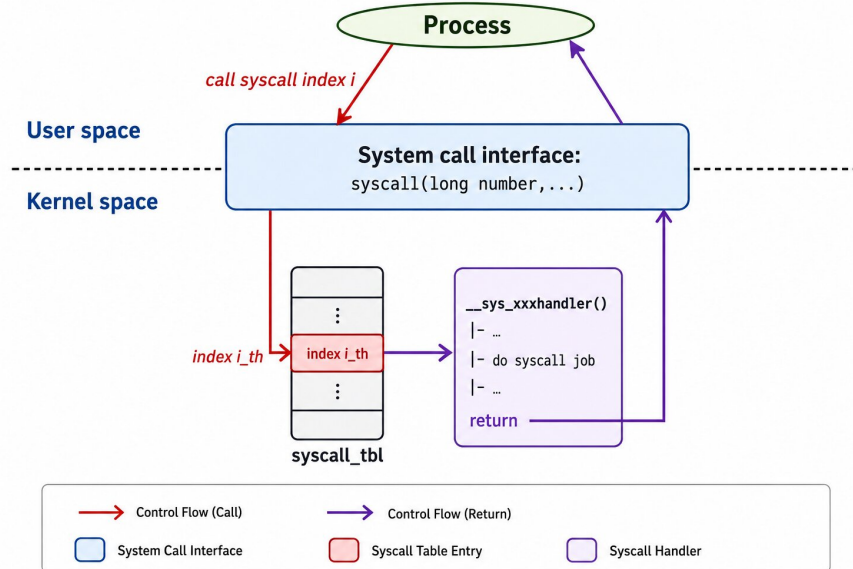
```
1 struct pcb_t* get_proc_from_PID(struct krnl_t *krnl, uint32_t pid){  
2     if (pid < 0 || pid > PID_MAX) return NULL;  
3     return krnl->proc_table[pid];  
4 }
```

4.1.2 Quy trình thực thi Syscall

Luồng thực thi chỉ thị Syscall:

Quy trình thực thi một syscall diễn ra theo các bước tuần tự từ User đến Kernel:

- **Lập lịch:** Bộ lập lịch (Scheduler) lựa chọn một Syscall đang ở trạng thái sẵn sàng từ `ready_queue` để đưa vào `running_list`. Tại đây CPU bắt đầu nạp các chỉ thị của tiến trình để xử lý.
- **Giải mã:** Khi CPU thực hiện lệnh (`cpu_routine`) và phát hiện opcode == SYSCALL, nó sẽ trích xuất các đối số từ chỉ thị và nạp vào cấu trúc thanh ghi hệ thống `sc_regs`.
- **Điều phối Kernel:** Sau khi dữ liệu đã sẵn sàng trong các thanh ghi, hàm trung gian `__syscall()` đóng vai trò là "người điều phối", dựa vào số hiệu syscall (ví dụ: 17 cho `memmap`) để gọi hàm thực thi tương ứng trong kernel (như `__sys_memmap`).



Hình 19: Sơ đồ luồng xử lý System Call

4.2 Trả lời câu hỏi

Câu hỏi: Advantages and disadvantages of unified system call interface for file/memory/I/O?

Trả lời:

Ưu điểm:

- Đơn giản hóa: đơn giản hơn cho lập trình viên không cần phải nhớ hết tất cả các lệnh của hệ thống, chỉ cần nhớ các quy tắc chung khi gọi system call là có thể quản lý được cả bộ nhớ và I/O.
- Ổn định: Các đoạn mã phức tạp sẽ do Kernel xử lý, người dùng ít gặp lỗi hơn.
- Bảo mật: Giúp che giấu sự phức tạp của phần cứng.
- Người dùng/Ứng dụng không thể can thiệp trực tiếp vào thiết bị mà phải thông qua Kernel.
- Điều này ngăn chặn việc thực thi các lệnh độc hại và giúp kiểm soát quyền truy cập chặt chẽ hơn.

Nhược điểm :

- Giảm hiệu năng: Việc phải "xin phép" Kernel và đi qua lớp trung gian làm tốn thêm thời gian xử lý của CPU.
- Với những tác vụ cần tốc độ cực cao (như xử lý đồ họa hay dữ liệu lớn), đây là một điểm trừ.

Câu hỏi: When a system call executes too long, how does the OS detect and handle it?

Trong một hệ thống thực tế sẽ xảy ra 2 trường hợp lệnh system call thực thi tốn nhiều thời gian:

- Trường hợp 1(CPU bận tính toán): Bộ đếm thời gian được thiết lập sẵn, khi Time slice hết, nó sẽ bị OS cưỡng chế quay về hàng đợi để nhường chỗ cho các tiến trình khác, đồng thời OS cũng lưu lại trạng thái hiện tại của system call và lưu vào stack để tiếp tục tính toán sau.



- Trường hợp 2(Chờ tín hiệu I/O): Khi này OS sẽ ngắt tiến trình và nhường CPU cho các tiến trình khác cho đến khi nhận được tín hiệu I/O.

Bonus: nếu system call chạy quá lâu trong OS mô phỏng của bài tập lớn này thì cpu sẽ bị treo

4.3 Kết quả thực thi

4.3.1 os_sc

Input

```
1 2 1 1
2 2048 16777216 0 0 0
3 2 sc3 15
```

Output

```
1 Time slot 0
2 ld_routine
3 Time slot 1
4 Time slot 2
5     Loaded a process at input/proc/sc3, PID: 1 PRIO: 15
6 Time slot 3
7     CPU 0: Dispatched process 1
8 Time slot 4
9     CPU 0: Processed 1 has finished
10    CPU 0 stopped
```

4.3.2 os_syscall

Input

```
1 2 1 1
2 2048 16777216 0 0 0
3 1 sc2 15
```

Output

```
1 Time slot 0
2 ld_routine
3 Time slot 1
4     Loaded a process at input/proc/sc2, PID: 1 PRIO: 15
5     CPU 0: Dispatched process 1
6 liballoc:181
7 print_pgtbl:
8   PDG=0x78256c006810 P4g=0x782570000bc0 PUD=0x782570001bd0 PMD=0x782570002be0
9 Time slot 2
10 libwrite:470
11 print_pgtbl:
12   PDG=0x78256c006810 P4g=0x782570000bc0 PUD=0x782570001bd0 PMD=0x782570002be0
13 Time slot 3
14     CPU 0: Put process 1 to run queue
15     CPU 0: Dispatched process 1
16 libwrite:470
17 print_pgtbl:
18   PDG=0x78256c006810 P4g=0x782570000bc0 PUD=0x782570001bd0 PMD=0x782570002be0
19 Time slot 4
20 libwrite:470
21 print_pgtbl:
```



```
22 PDG=0x78256c006810 P4g=0x782570000bc0 PUD=0x782570001bd0 PMD=0x782570002be0
23 Time slot 5
24 CPU 0: Put process 1 to run queue
25 CPU 0: Dispatched process 1
26 Time slot 6
27 CPU 0: Processed 1 has finished
28 CPU 0 stopped
```

4.3.3 os_syscall_list

Input

```
1 2 1 1
2 2048 16777216 0 0 0
3 1 scl 15
```

Output

```
1 Time slot 0
2 ld_routine
3 Time slot 1
4 Loaded a process at input/proc/scl, PID: 1 PRIO: 15
5 CPU 0: Dispatched process 1
6 0-sys_listsyscall
7 17-sys_memmap
8 Time slot 2
9 CPU 0: Processed 1 has finished
10 CPU 0 stopped
```

5 Synchronization

Trong môi trường đa nhiệm, khi nhiều tiến trình cùng truy cập vào một dữ liệu dùng chung, hiện tượng Race Condition (tranh chấp dữ liệu) có thể xảy ra. Để đảm bảo tính toàn vẹn của dữ liệu, hệ thống triển khai các cơ chế đồng bộ hóa.

5.1 Critical section và Lock

Critical section là những đoạn mã thực hiện việc truy cập và chỉnh sửa vào các vùng bộ nhớ chung, khi này các luồng thực thi có thể tranh chấp hoặc ghi đè dữ liệu của nhau trên vùng nhớ đó gây ra hiện tượng race condition. Vậy nên, Trước khi tiến vào critical section, ta thực hiện lệnh lock() để đảm bảo chỉ có duy nhất 1 tiến trình được thực thi tại lúc đó. Sau khi xong việc, lệnh unlock() giải phóng cho các tiến trình khác.

Consider this execution *interleaving* with “counter = 5” initially:

S0: producer execute	<code>register1 = counter</code>	{register1 = 5}
S1: producer execute	<code>register1 = register1 + 1</code>	{register1 = 6}
S2: consumer execute	<code>register2 = counter</code>	{register2 = 5}
S3: consumer execute	<code>register2 = register2 - 1</code>	{register2 = 4}
S4: producer execute	<code>counter = register1</code>	{counter = 6}
S5: consumer execute	<code>counter = register2</code>	{counter = 4}

=> Data inconsistency

Hình 20: Minh họa Data Inconsistency

5.2 Trả lời câu hỏi

Câu hỏi: What happens if the synchronization is not handled in your Simple OS?

Vấn đề: Trong mô hình đa nhân (Multi-CPU), nhiều cpu thread liên tục gọi `get_proc()` để tìm tiến trình kế tiếp. Đồng thời, loader thread liên tục gọi `add_proc()` để đưa tiến trình mới vào hệ thống. Nếu không có cơ chế đồng bộ hóa, việc nhiều luồng cùng thao tác với mảng `mlq_ready_queue` và biến `size` sẽ dẫn tới **Race Condition** (tranh chấp điều khiển).

Hậu quả của Race Condition:

- Biến `size` bị đọc/ghi đồng thời, dẫn tới giá trị sai lệch (*torn write*).
- Một tiến trình có thể bị **hai CPU lấy cùng lúc** (double dispatch).
- CPU có thể lấy ra phần tử NULL, gây *segmentation fault*.
- Thất thoát tiến trình: một PCB bị ghi đè mà không được lưu.

Cách xử lý trong mã nguồn: Tất cả các vùng nguy hiểm (Critical Section) đều được bao bọc cẩn thận bởi mutex:

```
1 pthread_mutex_lock(&queue_lock);
2 /* --- Critical Section ---
3  * Chỉ có duy nhất một luồng được thực thi
4  * đoạn code này tại một thời điểm.
5  */
6 proc = dequeue(&mlq_ready_queue[prio]);
7 slot[prio]--;
8 enqueue(&running_list, proc);
9 /* ----- */
10 pthread_mutex_unlock(&queue_lock);
```

Nhờ `pthread_mutex_lock` và `pthread_mutex_unlock`, chỉ đúng một luồng được quyền thay đổi cấu trúc hàng đợi trong một thời điểm nhất định. Điều này đảm bảo tính **atomicity** (tính nguyên tử) của các thao tác đọc-sửa-ghi trên dữ liệu chia sẻ.



6 Put It All Together

6.1 Testcase `os_1_mlq_paging_small_1K`

Input:

```
1 2 4 8
2 2048 16777216 0 0 0
3 1 p0s 130
4 3 m0s 120
5 5 s1 0
6 6 p1s 15
7 9 s3 39
8 10 s2 120
9 11 m1s 15
10 12 s0 38
11
```

Output:

```
1 Time slot 0
2 ld_routine
3     Loaded a process at input/proc/p0s, PID: 1 PRIO: 130
4     CPU 2: Dispatched process 1
5 Time slot 1
6 Time slot 2
7 liballoc:181
8 print_pgtbl:
9   PDG=0x7d3d20009c50 P4g=0x7d3d1c000bc0 PUD=0x7d3d1c001bd0 PMD=0x7d3d1c002be0
10   ↪ PT=0x7d3d1c003bf0
11   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
12   ↪ storage_bytes=20480
13     Loaded a process at input/proc/m0s, PID: 2 PRIO: 120
14 Time slot 3
15     CPU 2: Put process 1 to run queue
16     CPU 2: Dispatched process 2
17 liballoc:181
18 print_pgtbl:
19   PDG=0x7d3d2000d730 P4g=0x7d3d1c004c80 PUD=0x7d3d1c005c90 PMD=0x7d3d1c006ca0
20   ↪ PT=0x7d3d1c007cb0
21   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
22   ↪ storage_bytes=20480
23     CPU 3: Dispatched process 1
24 liballoc:181
25 print_pgtbl:
26   PDG=0x7d3d20009c50 P4g=0x7d3d1c000bc0 PUD=0x7d3d1c001bd0 PMD=0x7d3d1c002be0
27   ↪ PT=0x7d3d1c003bf0
28   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
29   ↪ storage_bytes=20480
30 libfree:205
31 print_pgtbl:
32   PDG=0x7d3d20009c50 P4g=0x7d3d1c000bc0 PUD=0x7d3d1c001bd0 PMD=0x7d3d1c002be0
33   ↪ PT=0x7d3d1c003bf0
34   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
35   ↪ storage_bytes=20480
36 liballoc:181
37 print_pgtbl:
38   PDG=0x7d3d2000d730 P4g=0x7d3d1c004c80 PUD=0x7d3d1c005c90 PMD=0x7d3d1c006ca0
39   ↪ PT=0x7d3d1c007cb0
```



```
31  pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
32  Time slot 4
    Loaded a process at input/proc/s1, PID: 3 PRIO: 0
33      CPU 2: Put process 2 to run queue
34      CPU 2: Dispatched process 3
35      CPU 3: Put process 1 to run queue
36      CPU 3: Dispatched process 2
37
38  libfree:205
39  print_ptgbl:
40  PDG=0x7d3d2000d730 P4g=0x7d3d1c004c80 PUD=0x7d3d1c005c90 PMD=0x7d3d1c006ca0
    ↳ PT=0x7d3d1c007cb0
41  pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
42      CPU 1: Dispatched process 1
43  liballoc:181
44  print_ptgbl:
45  PDG=0x7d3d20009c50 P4g=0x7d3d1c000bc0 PUD=0x7d3d1c001bd0 PMD=0x7d3d1c002be0
    ↳ PT=0x7d3d1c003bf0
46  pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
47  Time slot 5
48  liballoc:181
49  print_ptgbl:
50  PDG=0x7d3d2000d730 P4g=0x7d3d1c004c80 PUD=0x7d3d1c005c90 PMD=0x7d3d1c006ca0
    ↳ PT=0x7d3d1c007cb0
51  pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
52  Time slot 6
53  libwrite:470
54  print_ptgbl:
55  PDG=0x7d3d20009c50 P4g=0x7d3d1c000bc0 PUD=0x7d3d1c001bd0 PMD=0x7d3d1c002be0
    ↳ PT=0x7d3d1c003bf0
56  pgtbl_stats: walks=3 level_reads=15 level_writes=5 total_accesses=20
    ↳ tables=5 storage_bytes=20480
57      Loaded a process at input/proc/pls, PID: 4 PRIO: 15
58      CPU 3: Put process 2 to run queue
59      CPU 3: Dispatched process 4
60      CPU 2: Put process 3 to run queue
61      CPU 2: Dispatched process 3
62  Time slot 7
63      CPU 0: Dispatched process 2
64  libwrite:470
65  print_ptgbl:
66  PDG=0x7d3d2000d730 P4g=0x7d3d1c004c80 PUD=0x7d3d1c005c90 PMD=0x7d3d1c006ca0
    ↳ PT=0x7d3d1c007cb0
67  pgtbl_stats: walks=3 level_reads=15 level_writes=5 total_accesses=20
    ↳ tables=5 storage_bytes=20480
68      CPU 1: Put process 1 to run queue
69      CPU 1: Dispatched process 1
70  libread:410
71  print_ptgbl:
72  PDG=0x7d3d20009c50 P4g=0x7d3d1c000bc0 PUD=0x7d3d1c001bd0 PMD=0x7d3d1c002be0
    ↳ PT=0x7d3d1c003bf0
73  pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
    ↳ tables=5 storage_bytes=20480
74  Time slot 8
75      Loaded a process at input/proc/s3, PID: 5 PRIO: 39
76      CPU 3: Put process 4 to run queue
```



```
77         CPU 3: Dispatched process 4
78         CPU 2: Put process 3 to run queue
79         CPU 2: Dispatched process 3
80 Time slot 9
81         CPU 1: Put process 1 to run queue
82         CPU 1: Dispatched process 5
83         CPU 0: Processed 2 has finished
84         CPU 0: Dispatched process 1
85 print_pgtbl:
86     PDG=0x7d3d20009c50 P4g=0x7d3d1c000bc0 PUD=0x7d3d1c001bd0 PMD=0x7d3d1c002be0
87     ↪ PT=0x7d3d1c003bf0
88     pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
89     ↪ tables=5 storage_bytes=20480
90 Time slot 10
91     Loaded a process at input/proc/s2, PID: 6 PRIO: 120
92     CPU 3: Put process 4 to run queue
93     CPU 3: Dispatched process 4
94     CPU 2: Put process 3 to run queue
95     CPU 2: Dispatched process 3
96 Time slot 11
97     CPU 1: Put process 5 to run queue
98     CPU 1: Dispatched process 5
99     CPU 0: Put process 1 to run queue
100    CPU 0: Dispatched process 6
101    Loaded a process at input/proc/mls, PID: 7 PRIO: 15
102    CPU 2: Processed 3 has finished
103    CPU 2: Dispatched process 7
104 Time slot 12
105 liballoc:181
106 print_pgtbl:
107     PDG=0x7d3d20020000 P4g=0x7d3d1c008d40 PUD=0x7d3d1c009d50 PMD=0x7d3d1c00ad60
108     ↪ PT=0x7d3d1c00bd70
109     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
110     ↪ storage_bytes=20480
111     Loaded a process at input/proc/s0, PID: 8 PRIO: 38
112     CPU 3: Put process 4 to run queue
113     CPU 3: Dispatched process 4
114 Time slot 13
115     CPU 0: Put process 6 to run queue
116     CPU 0: Dispatched process 8
117 liballoc:181
118 print_pgtbl:
119     PDG=0x7d3d20020000 P4g=0x7d3d1c008d40 PUD=0x7d3d1c009d50 PMD=0x7d3d1c00ad60
120     ↪ PT=0x7d3d1c00bd70
121     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
122     ↪ storage_bytes=20480
123     CPU 1: Put process 5 to run queue
124     CPU 1: Dispatched process 5
125     CPU 2: Put process 7 to run queue
126     CPU 2: Dispatched process 7
127 libfree:205
128 print_pgtbl:
129     PDG=0x7d3d20020000 P4g=0x7d3d1c008d40 PUD=0x7d3d1c009d50 PMD=0x7d3d1c00ad60
130     ↪ PT=0x7d3d1c00bd70
131     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
132     ↪ storage_bytes=20480
133 Time slot 14
134     CPU 3: Put process 4 to run queue
135     CPU 3: Dispatched process 4
```

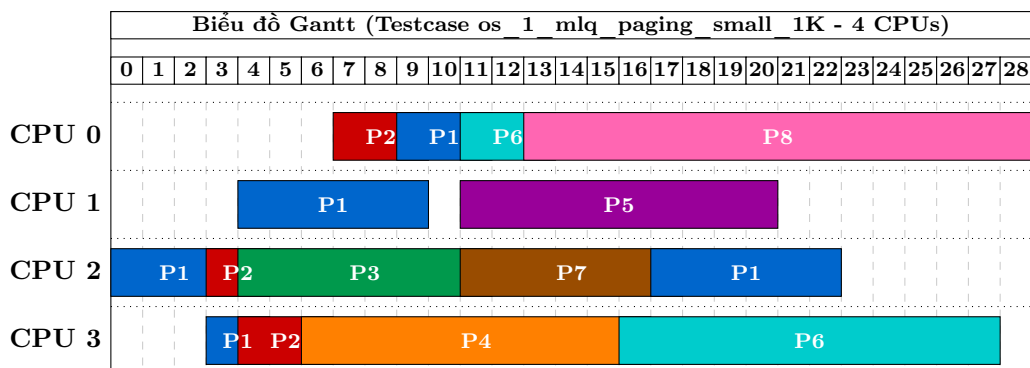


```
128 Time slot 15
129     CPU 1: Put process 5 to run queue
130     CPU 1: Dispatched process 5
131 liballoc:181
132 print_pgtbl:
133     PDG=0x7d3d20020000 P4g=0x7d3d1c008d40 PUD=0x7d3d1c009d50 PMD=0x7d3d1c00ad60
134     ↪ PT=0x7d3d1c00bd70
135     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
136     ↪ storage_bytes=20480
137     CPU 0: Put process 8 to run queue
138     CPU 0: Dispatched process 8
139     CPU 2: Put process 7 to run queue
140     CPU 2: Dispatched process 7
141 libfree:205
142 print_pgtbl:
143     PDG=0x7d3d20020000 P4g=0x7d3d1c008d40 PUD=0x7d3d1c009d50 PMD=0x7d3d1c00ad60
144     ↪ PT=0x7d3d1c00bd70
145     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
146     ↪ storage_bytes=20480
147 Time slot 16
148     CPU 3: Processed 4 has finished
149     CPU 3: Dispatched process 6
150 Time slot 17
151     CPU 1: Put process 5 to run queue
152     CPU 1: Dispatched process 5
153 libfree:205
154 print_pgtbl:
155     PDG=0x7d3d20020000 P4g=0x7d3d1c008d40 PUD=0x7d3d1c009d50 PMD=0x7d3d1c00ad60
156     ↪ PT=0x7d3d1c00bd70
157     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
158     ↪ storage_bytes=20480
159     CPU 0: Put process 8 to run queue
160     CPU 0: Dispatched process 8
161     CPU 2: Processed 7 has finished
162     CPU 2: Dispatched process 1
163 print_pgtbl:
164     PDG=0x7d3d20009c50 P4g=0x7d3d1c000bc0 PUD=0x7d3d1c001bd0 PMD=0x7d3d1c002be0
165     ↪ PT=0x7d3d1c003bf0
166     pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
167     ↪ tables=5 storage_bytes=20480
168 Time slot 18
169     CPU 3: Put process 6 to run queue
170     CPU 3: Dispatched process 6
171 Time slot 19
172     CPU 1: Put process 5 to run queue
173     CPU 1: Dispatched process 5
174     CPU 0: Put process 8 to run queue
175     CPU 0: Dispatched process 8
176 Time slot 20
177     CPU 2: Put process 1 to run queue
178     CPU 2: Dispatched process 1
179 libfree:205
180 print_pgtbl:
181     PDG=0x7d3d20009c50 P4g=0x7d3d1c000bc0 PUD=0x7d3d1c001bd0 PMD=0x7d3d1c002be0
182     ↪ PT=0x7d3d1c003bf0
183     pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
184     ↪ tables=5 storage_bytes=20480
185     CPU 1: Processed 5 has finished
186     CPU 1 stopped
```



```
177 CPU 3: Put process 6 to run queue
178 CPU 3: Dispatched process 6
179 Time slot 21
180 CPU 0: Put process 8 to run queue
181 CPU 0: Dispatched process 8
182 Time slot 22
183 CPU 2: Processed 1 has finished
184 CPU 2 stopped
185 Time slot 23
186 CPU 3: Put process 6 to run queue
187 CPU 3: Dispatched process 6
188 CPU 0: Put process 8 to run queue
189 CPU 0: Dispatched process 8
190 Time slot 24
191 Time slot 25
192 CPU 3: Put process 6 to run queue
193 CPU 3: Dispatched process 6
194 CPU 0: Put process 8 to run queue
195 CPU 0: Dispatched process 8
196 Time slot 26
197 Time slot 27
198 CPU 3: Processed 6 has finished
199 CPU 3 stopped
200 CPU 0: Put process 8 to run queue
201 CPU 0: Dispatched process 8
202 Time slot 28
203 CPU 0: Processed 8 has finished
204 CPU 0 stopped
```

Biểu đồ Gantt:



Hình 21: Phân lịch MLQ trên 4 CPUs (os_1_mlq_paging_small_1K).

Thống kê chi phí quản lý bộ nhớ (Đại diện: tiến trình P1):

- Số lần truy cập bộ nhớ của tiến trình (total_accesses): 30 lần.
- Kích thước cây phân trang cấp phát cho tiến trình (Page Table Overhead): 20 KB.

So sánh với mẫu: (output/os_1_mlq_paging_small_1K.output) Testcase đạt yêu cầu. Logic thực thi trùng khớp với output mẫu về trình tự nạp tiến trình và các thao tác bộ nhớ (alloc, free, read, write).



6.2 Testcase os_1_mlq_paging_small_4K

Input:

```
1 2 4 8
2 4096 16777216 0 0 0
3 1 p0s 130
4 2 m0s 120
5 4 s3 39
6 5 s0 38
7 9 m1s 15
8 11 p1s 15
9 13 s2 120
10 16 s1 0
11
```

Output:

```
1 Time slot 0
2 ld_routine
3     Loaded a process at input/proc/p0s, PID: 1 PRIO: 130
4     CPU 3: Dispatched process 1
5 Time slot 1
6 Time slot 2
7 liballoc:181
8 print_pgtbl:
9   PDG=0x740c38009c50 P4g=0x740c30000bc0 PUD=0x740c30001bd0 PMD=0x740c30002be0
10   ↪ PT=0x740c30003bf0
11   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
12   ↪ storage_bytes=20480
13     Loaded a process at input/proc/m0s, PID: 2 PRIO: 120
14 Time slot 3
15     CPU 3: Put process 1 to run queue
16     CPU 3: Dispatched process 1
17 liballoc:181
18 print_pgtbl:
19   PDG=0x740c38009c50 P4g=0x740c30000bc0 PUD=0x740c30001bd0 PMD=0x740c30002be0
20   ↪ PT=0x740c30003bf0
21   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
22   ↪ storage_bytes=20480
23     CPU 2: Dispatched process 2
24 liballoc:181
25 print_pgtbl:
26   PDG=0x740c3800d730 P4g=0x740c34000bc0 PUD=0x740c34001bd0 PMD=0x740c34002be0
27   ↪ PT=0x740c34003bf0
28   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
29   ↪ storage_bytes=20480
30     Loaded a process at input/proc/s3, PID: 3 PRIO: 39
31 libfree:205
32 print_pgtbl:
33   PDG=0x740c38009c50 P4g=0x740c30000bc0 PUD=0x740c30001bd0 PMD=0x740c30002be0
34   ↪ PT=0x740c30003bf0
35   pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
36   ↪ storage_bytes=20480
37 Time slot 4
38 liballoc:181
39 print_pgtbl:
40   PDG=0x740c3800d730 P4g=0x740c34000bc0 PUD=0x740c34001bd0 PMD=0x740c34002be0
41   ↪ PT=0x740c34003bf0
```



```
33  pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
34      CPU 1: Dispatched process 3
35      CPU 3: Put process 1 to run queue
36      CPU 3: Dispatched process 1
37  liballoc:181
38  print_pgtbl:
39  PDG=0x740c38009c50 P4g=0x740c30000bc0 PUD=0x740c30001bd0 PMD=0x740c30002be0
    ↳ PT=0x740c30003bf0
40  pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
41      CPU 2: Put process 2 to run queue
42      CPU 2: Dispatched process 2
43  libfree:205
44  print_pgtbl:
45  PDG=0x740c3800d730 P4g=0x740c34000bc0 PUD=0x740c34001bd0 PMD=0x740c34002be0
    ↳ PT=0x740c34003bf0
46  pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
47  Time slot 5
48      Loaded a process at input/proc/s0, PID: 4 PRIO: 38
49  libwrite:470
50  print_pgtbl:
51  PDG=0x740c38009c50 P4g=0x740c30000bc0 PUD=0x740c30001bd0 PMD=0x740c30002be0
    ↳ PT=0x740c30003bf0
52  pgtbl_stats: walks=3 level_reads=15 level_writes=5 total_accesses=20
    ↳ tables=5 storage_bytes=20480
53  liballoc:181
54  print_pgtbl:
55  Time slot 6
56      CPU 0: Dispatched process 4
57  PDG=0x740c3800d730 P4g=0x740c34000bc0 PUD=0x740c34001bd0 PMD=0x740c34002be0
    ↳ PT=0x740c34003bf0
58  pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
    ↳ storage_bytes=20480
59      CPU 1: Put process 3 to run queue
60      CPU 1: Dispatched process 3
61      CPU 3: Put process 1 to run queue
62      CPU 3: Dispatched process 1
63  libread:410
64  print_pgtbl:
65  PDG=0x740c38009c50 P4g=0x740c30000bc0 PUD=0x740c30001bd0 PMD=0x740c30002be0
    ↳ PT=0x740c30003bf0
66  pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
    ↳ tables=5 storage_bytes=20480
67  Time slot 7
68      CPU 2: Put process 2 to run queue
69      CPU 2: Dispatched process 2
70  libwrite:470
71  print_pgtbl:
72  PDG=0x740c3800d730 P4g=0x740c34000bc0 PUD=0x740c34001bd0 PMD=0x740c34002be0
    ↳ PT=0x740c34003bf0
73  pgtbl_stats: walks=3 level_reads=15 level_writes=5 total_accesses=20
    ↳ tables=5 storage_bytes=20480
74      CPU 1: Put process 3 to run queue
75      CPU 1: Dispatched process 3
76  Time slot 8
77      CPU 0: Put process 4 to run queue
78      CPU 0: Dispatched process 4
```



```
79         Loaded a process at input/proc/mls, PID: 5 PRIO: 15
80         CPU 3: Put process 1 to run queue
81         CPU 3: Dispatched process 5
82 liballoc:181
83 print_pgtbl:
84     PDG=0x740c380189f0 P4g=0x740c30004cb0 PUD=0x740c30005cc0 PMD=0x740c30006cd0
85     ↪ PT=0x740c30007ce0
86     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
87     ↪ storage_bytes=20480
88     CPU 2: Processed 2 has finished
89     CPU 2: Dispatched process 1
90 print_pgtbl:
91     PDG=0x740c38009c50 P4g=0x740c30000bc0 PUD=0x740c30001bd0 PMD=0x740c30002be0
92     ↪ PT=0x740c30003bf0
93     pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
94     ↪ tables=5 storage_bytes=20480
95 Time slot 9
96 liballoc:181
97 print_pgtbl:
98     PDG=0x740c380189f0 P4g=0x740c30004cb0 PUD=0x740c30005cc0 PMD=0x740c30006cd0
99     ↪ PT=0x740c30007ce0
100    pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
101    ↪ storage_bytes=20480
102 Time slot 10
103     CPU 0: Put process 4 to run queue
104     CPU 0: Dispatched process 4
105     CPU 1: Put process 3 to run queue
106     CPU 1: Dispatched process 3
107     Loaded a process at input/proc/pls, PID: 6 PRIO: 15
108 Time slot 11
109     CPU 2: Put process 1 to run queue
110     CPU 2: Dispatched process 6
111     CPU 3: Put process 5 to run queue
112     CPU 3: Dispatched process 5
113 libfree:205
114 print_pgtbl:
115     PDG=0x740c380189f0 P4g=0x740c30004cb0 PUD=0x740c30005cc0 PMD=0x740c30006cd0
116     ↪ PT=0x740c30007ce0
117     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
118     ↪ storage_bytes=20480
119 liballoc:181
120 print_pgtbl:
121     PDG=0x740c380189f0 P4g=0x740c30004cb0 PUD=0x740c30005cc0 PMD=0x740c30006cd0
122     ↪ PT=0x740c30007ce0
123     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
124     ↪ storage_bytes=20480
125 Time slot 12
126     CPU 1: Put process 3 to run queue
127     CPU 1: Dispatched process 3
128     CPU 0: Put process 4 to run queue
129     CPU 0: Dispatched process 4
130     Loaded a process at input/proc/s2, PID: 7 PRIO: 120
131     CPU 3: Put process 5 to run queue
132     CPU 3: Dispatched process 5
133 libfree:205
134 print_pgtbl:
135     PDG=0x740c380189f0 P4g=0x740c30004cb0 PUD=0x740c30005cc0 PMD=0x740c30006cd0
136     ↪ PT=0x740c30007ce0
137     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
138     ↪ storage_bytes=20480
```



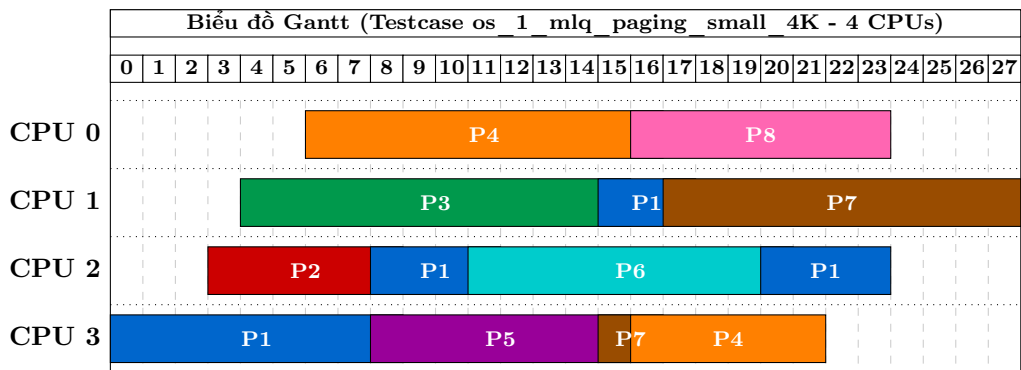
```
127         CPU 2: Put process 6 to run queue
128         CPU 2: Dispatched process 6
129 Time slot 13
130 libfree:205
131 print_pgtbl:
132     PDG=0x740c380189f0 P4g=0x740c30004cb0 PUD=0x740c30005cc0 PMD=0x740c30006cd0
133     ↪ PT=0x740c30007ce0
134     pgtbl_stats: walks=1 level_reads=5 level_writes=5 total_accesses=10 tables=5
135     ↪ storage_bytes=20480
136 Time slot 14
137     CPU 1: Put process 3 to run queue
138     CPU 1: Dispatched process 3
139     CPU 0: Put process 4 to run queue
140     CPU 0: Dispatched process 4
141     CPU 2: Put process 6 to run queue
142     CPU 2: Dispatched process 6
143 Time slot 15
144     CPU 1: Processed 3 has finished
145     CPU 3: Processed 5 has finished
146     CPU 3: Dispatched process 7
147     CPU 1: Dispatched process 1
148 print_pgtbl:
149     PDG=0x740c38009c50 P4g=0x740c30000bc0 PUD=0x740c30001bd0 PMD=0x740c30002be0
150     ↪ PT=0x740c30003bf0
151     pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
152     ↪ tables=5 storage_bytes=20480
153     Loaded a process at input/proc/s1, PID: 8 PRIO: 0
154 Time slot 16
155     CPU 0: Put process 4 to run queue
156     CPU 0: Dispatched process 8
157     CPU 3: Put process 7 to run queue
158     CPU 3: Dispatched process 4
159     CPU 2: Put process 6 to run queue
160     CPU 2: Dispatched process 6
161 Time slot 17
162     CPU 1: Put process 1 to run queue
163     CPU 1: Dispatched process 7
164 Time slot 18
165     CPU 0: Put process 8 to run queue
166     CPU 0: Dispatched process 8
167     CPU 3: Put process 4 to run queue
168     CPU 3: Dispatched process 4
169 Time slot 19
170     CPU 2: Put process 6 to run queue
171     CPU 2: Dispatched process 6
172     CPU 1: Put process 7 to run queue
173     CPU 1: Dispatched process 7
174 Time slot 20
175     CPU 0: Put process 8 to run queue
176     CPU 0: Dispatched process 8
177     CPU 3: Put process 4 to run queue
178     CPU 3: Dispatched process 4
179     CPU 2: Processed 6 has finished
180     CPU 2: Dispatched process 1
181 libfree:205
182 print_pgtbl:
183     PDG=0x740c38009c50 P4g=0x740c30000bc0 PUD=0x740c30001bd0 PMD=0x740c30002be0
184     ↪ PT=0x740c30003bf0
185     pgtbl_stats: walks=5 level_reads=25 level_writes=5 total_accesses=30
186     ↪ tables=5 storage_bytes=20480
```

```

181 Time slot 21
182     CPU 1: Put process 7 to run queue
183     CPU 1: Dispatched process 7
184     CPU 3: Processed 4 has finished
185     CPU 3 stopped
186 Time slot 22
187     CPU 0: Put process 8 to run queue
188     CPU 0: Dispatched process 8
189 Time slot 23
190     CPU 1: Put process 7 to run queue
191     CPU 1: Dispatched process 7
192     CPU 0: Processed 8 has finished
193     CPU 0 stopped
194     CPU 2: Processed 1 has finished
195     CPU 2 stopped
196 Time slot 24
197 Time slot 25
198     CPU 1: Put process 7 to run queue
199     CPU 1: Dispatched process 7
200 Time slot 26
201 Time slot 27
202     CPU 1: Processed 7 has finished
203     CPU 1 stopped

```

Biểu đồ Gantt:



Hình 22: Phân lịch MLQ trên 4 CPUs (os_1_mlq_paging_small_4K).

Thống kê chi phí quản lý bộ nhớ (Đại diện: tiến trình P1):

- Số lần truy cập bộ nhớ của tiến trình (total_accesses): 30 lần.
- Kích thước cây phân trang cấp phát cho tiến trình (Page Table Overhead): 20 KB.

So sánh với mẫu: (output/os_1_mlq_paging_small_4K.output) Testcase đạt yêu cầu. Logic thực thi trùng khớp với output mẫu về trình tự nạp tiến trình và các thao tác bộ nhớ (alloc, free, read, write).

Nhận xét chung:

- **Tính đúng đắn và độc lập module:** Việc thay đổi kích thước trang (Page Size) từ 1KB sang 4KB làm thay đổi tần suất gọi các hàm cấp phát/thu hồi trang (alloc/free) cũng như ánh xạ bộ nhớ ảo.



- **Khác biệt định dạng:** Tương tự các testcase trước, output của nhóm thể hiện định dạng log khác với output mẫu do quá trình sửa đổi mã nguồn, cũng như các con trỏ địa chỉ bộ nhớ có sự khác biệt do cơ chế ngẫu nhiên hóa ASLR (*Address Space Layout Randomization*)

7 Kết luận

7.1 Mức độ hoàn thành mục tiêu

Trải qua quá trình nghiên cứu lý thuyết hệ thống kết hợp hiện thực hóa mã nguồn cấu trúc nhân, nhóm đã hoàn thành toàn vẹn các mục tiêu cốt lõi đặt ra của bài tập lớn môn Hệ điều hành (CO2018):

- **Mô-đun Bộ định thời (Scheduler):** Cài đặt thành công thuật toán lập lịch đa mức ưu tiên Multi-Level Queue (MLQ) với chính sách phân bổ lượng slot chạy theo cấu hình độ ưu tiên tĩnh chỉnh. Hệ thống đã vượt qua các bài kiểm thử độ chính xác định thời độc lập và cơ chế luân phiên Round-Robin đối với các tiến trình đồng cấp.
- **Mô-đun Quản lý bộ nhớ (Memory Management):** Thiết lập cấu trúc sơ đồ bộ nhớ phân tầng thưa (Sparse Multi-level Table Layout) nâng cấp lên mô hình phân trang 5 cấp (5-level paging) trên nền tảng địa chỉ ảo hóa 64-bit canonical. Hiện thực hóa hoàn hảo thư viện quản lý libmem tích hợp vòng đời hoán đổi trang vật lý (Swapping) khi chịu tải RAM vật lý giới hạn.
- **Cơ chế Đồng bộ và Hệ thống gọi (Synchronization & Syscalls):** Triển khai hệ thống bảo vệ miền tương tranh (Critical Section) chặt chẽ bằng cơ chế khóa loại trừ tương hỗ (pthread_mutex), triệt tiêu hoàn toàn lỗi chạy sai lệch dữ liệu (Race Condition) trong kịch bản định thời đa nhân song song. Đồng thời bổ sung bảng định tuyến lời gọi đặc quyền hạt nhân thông qua cơ chế phân phối System Call tự định nghĩa.

7.2 Những khó khăn kỹ thuật và giải pháp kiểm thử

Trong quá trình triển khai thực nghiệm cấu trúc mã nguồn, nhóm đã đối mặt và giải quyết các bài toán kỹ thuật phức tạp:

1. **Bất đồng bộ tiến trình và Race Condition:** Do bản chất đa luồng thực tế của Host OS điều khiển song song luồng trình nạp dữ liệu (*Loader*) và các luồng nhân xử lý (*CPU cores*), hệ thống thỉnh thoảng xuất hiện hiện tượng tráo đổi thứ tự in log hoặc dịch chuyển nhẹ trục thời gian biểu đồ Gantt so với cấu hình mẫu. Nhóm đã giải quyết bằng cách thiết kế cơ chế bao bọc an toàn nguyên tử (Atomicity Lock) bằng `queue_lock` và `mmvm_lock` giúp toàn bộ logic cốt lõi hoạt động đúng đắn và độc lập mô-đun cao.
2. **Tối ưu hóa không gian bảng quản lý:** Đối với kiến trúc địa chỉ lớn 64-bit, việc cấp phát trước toàn bộ cây phân trang sẽ làm sụp đổ dung lượng phần cứng. Nhóm đã áp dụng chiến lược cấp phát theo yêu cầu (Demand Allocation) phối hợp bộ nhớ đệm dạng Pool (*Slab cache allocator*), giúp tổng dung lượng quản lý bảng trang của các tiến trình kiểm thử rút gọn chỉ còn xấp xỉ 20 KB.
3. **Rò rỉ tài nguyên (Memory Leak):** Việc cấp/free động liên tục trên hot-path dễ dẫn đến thất thoát RAM. Hệ thống đã được kiểm tra bằng công cụ chuyên dụng *Valgrind* và đạt chỉ số an toàn tuyệt đối nhờ chiến lược tái sử dụng nút danh sách liên kết được khởi tạo tĩnh tập trung một lần.

7.3 Hướng phát triển và tối ưu hệ thống trong tương lai

Mặc dù hệ điều hành mô phỏng *Simple Operating System* đã đạt độ chính xác logic toàn diện trên các testcase, kiến trúc vẫn có thể được nâng cấp dựa trên các hướng phát triển sau:

- **Nâng cấp thuật toán định thời:** Cải tiến bộ lập lịch MLQ tĩnh thành thuật toán Hàng đợi đa cấp phản hồi (Multi-Level Feedback Queue - MLFQ) nhằm tự động phát hiện hành vi tiến trình (CPU-bound hay I/O-bound), từ đó tự điều chỉnh động mức ưu tiên, giảm thiểu tình trạng starved của tiến trình nền.



- **Tích hợp cơ chế TLB (Translation Lookaside Buffer):** Hiện tại, mỗi thao tác đọc/ghi ảo hóa của nhân tiêu tốn trung bình 5 lần truy cập bộ nhớ tuần tự (*walks*) để dịch địa chỉ. Việc cài đặt thêm một bộ đệm phần cứng giả lập TLB sẽ giúp tăng tốc độ ánh xạ các trang truy cập thường xuyên gần nhất về độ phức tạp thời gian $\mathcal{O}(1)$.
- **Cơ chế dồn dịch và thu gom bộ nhớ trống:** Phát triển thêm giải thuật tự động nén hoặc dồn dịch không gian địa chỉ rảnh (*Compaction / Defragmentation*) trong danh sách `vm_freerg_list` nhằm tối ưu hóa khả năng cấp phát các vùng nhớ liên tục quy mô lớn.